

PROPOSAL PROYEK AKHIR

**RANCANG BANGUN *BACK-END* SISTEM INFORMASI
KLINIK KECANTIKAN MISCHE BERBASIS *WEBSITE*
DENGAN *LARAVEL* DAN METODE *INCREMENTAL***

Politeknik Caltex Riau

Disusun oleh:

MUHAMMAD AS SHAFF

NIM 2257301088

PROGRAM STUDI SITEM INFORMASI

POLITEKNIK CALTEX RIAU

2025

RINGKASAN

Klinik kecantikan Mische menghadapi tantangan dalam melakukan pengelolaan data reservasi, promosi dan transaksi yang dilakukan selama ini secara terpisah dan belum terintegrasi dalam satu platform digital. Kondisi ini dapat menimbulkan resiko akan kesalahan pencatatan dan menghambat kinerja operasional klinik. Untuk mengatasi permasalahan tersebut, penelitian ini bertujuan mengembangkan *backend* sistem informasi berbasis website yang dapat mengakomodasi layanan klinik secara digital. Sistem dibangun dengan menggunakan *framework Laravel* dengan penerapan *Rest API* sehingga memungkinkan untuk berkomunikasi dengan frontend. Pada pengembangan sistem ini menerapkan metode *Incremental*, yang dimana dalam pengembangannya dilakukan secara bertahap sesuai dengan prioritas dan kebutuhan. *Backend* yang dibangun berupa modul manajemen pengguna, dokter, reservasi, promosi, event, testimoni, produk, laporan penjualan, hingga profil klinik. Kemudian pada bagian pengujian menerapkan *Unit Testing*, *Integration Testing*, *Performance Testing*, dan *Security Testing* untuk memastikan sistem yang akan digunakan dapat berjalan dengan baik, stabil, aman, dan sesuai dengan fungsinya. Hasil dari penelitian ini diharapkan dapat menyelesaikan permasalahan dalam menjalankan operasional sehari hari, serta dapat mendukung pelayanan yang lebih profesional dan modern di klinik kecantikan Mische.

Kata Kunci: *Backend, Sistem Informasi Klinik, Laravel, Rest API, Incremental.*

DAFTAR ISI

RINGKASAN	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL.....	v
I. PENDAHULUAN	1
1.1 LATAR BELAKANG	1
1.2 RUMUSAN MASALAH	3
1.3 BATASAN MASALAH	3
1.4 TUJUAN PENELITIAN	3
1.5 MANFAAT PENELITIAN.....	4
1.6 METODOLOGI PENELITIAN	4
1.7 SISTEM PENULISAN	5
II. TINJAUAN PUSTAKA.....	6
2.1 REVIEW PENELITIAN	6
2.2 LANDASAN TEORI	9
2.2.1 Klinik Kecantikan Mische	9
2.2.2 Sistem Informasi	10
2.2.3 Metode Pengembangan <i>Incremental</i>	10
2.2.4 <i>Entity Relation Diagram (ERD)</i>	12
2.2.5 <i>Database</i>	13
2.2.6 <i>MySQL</i>	14
2.2.7 <i>Backend</i>	15
2.2.8 <i>Laravel</i>	15
2.2.9 <i>PHP</i>	16
2.2.10 <i>Laragon</i>	18
2.2.11 <i>Rest API</i>	19
2.2.12 <i>Postman</i>	20
2.2.13 <i>Unit Testing</i>	21
2.2.14 <i>Integration Testing</i>	22
2.2.15 <i>Performance Testing</i>	22
2.2.16 <i>Security Testing</i>	23
III. PERANCANGAN	25
3.1 ANALISIS KEBUTUHAN PENGGUNA	25

3.1.1 Pengumpulan Kebutuhan Pengguna	25
3.1.2 Kebutuhan Sistem	25
3.1.3 Proses Bisnis <i>As-Is</i>	26
3.1.4 <i>Increment Plan</i>	27
3.2 DESAIN	28
3.2.1 <i>Proses Bisnis To-Be</i>	28
3.2.2 <i>Arsitektur Sistem</i>	29
3.2.3 <i>Use Case Diagram</i>	29
3.2.4 <i>Class Diagram</i>	32
3.2.5 <i>Perancangan Database</i>	33
3.2.6 <i>Perancangan API</i>	34
3.3 TESTING.....	35
3.3.1 <i>Rancangan Unit Testing</i>	35
3.3.2 <i>Rancangan Integration Testing</i>	35
3.3.3 <i>Rancangan Performance Testing</i>	36
3.3.4 <i>Rancangan Security Testing</i>	37
IV. JADWAL DAN ANGGARAN BIAYA	38
4.1 JADWAL	38
4.2 ANGGARAN BIAYA	38
V. DAFTAR PUSTAKA	39
LAMPIRAN A – SURAT PERNYATAAN.....	A-1
LAMPIRAN B – FOTO WAWANCARA	B-1
LAMPIRAN C – HASIL WAWANCARA	C-1
LAMPIRAN D – PROSES BISNIS <i>AS-IS</i>	D-1
LAMPIRAN E – PROSES BISNIS <i>TO-BE</i>	E-1
LAMPIRAN F – STRUKTUR TABEL.....	F-1
LAMPIRAN G – PERANCANGAN <i>API</i>	G-1
LAMPIRAN H – RANCANGAN UNIT TESTING	H-5
LAMPIRAN I – RANCANGAN <i>INTEGRATION TESTING</i>	I-1
LAMPIRAN J – RANCANGAN <i>PERFORMANCE TESTING</i>	J-1
LAMPIRAN K – RANCANGAN <i>SECURITY TESTING</i>	K-1
LAMPIRAN L – DOKUMENTASI <i>INCREMENT</i>	L-1

DAFTAR GAMBAR

Gambar 2. 1 Logo Klinik Kecantikan Mische	9
Gambar 2. 2 Tahapan Metode <i>Incremental</i>	11
Gambar 2. 3 Simbol <i>Entity</i> , <i>Atribut</i> , dan <i>Relasi</i>	12
Gambar 2. 4 Logo MySQL	14
Gambar 2. 5 Logo Laravel	15
Gambar 2. 6 Arsitektur <i>MVC</i>	16
Gambar 2. 7 Logo <i>PHP</i>	17
Gambar 2. 8 Logo <i>Laragon</i>	18
Gambar 2. 9 Arsitektur <i>Rest API</i>	19
Gambar 2. 10 Logo <i>Postman</i>	20
Gambar 2. 11 Siklus <i>Unit Testing</i>	21
Gambar 2. 12 <i>Integration Testing</i>	22
Gambar 2. 13 Alur Pengujian <i>Peformance Testing</i>	23
Gambar 3. 1 Proses Bisnis <i>As Is</i> Profil Klinik	26
Gambar 3. 2 Proses Bisnis <i>To-Be</i> Profil Klinik	28
Gambar 3.3 Arsitektur Sistem.....	29
Gambar 3.4 <i>Use Case Diagram</i>	30
Gambar 3. 5 <i>Class Diagram</i>	32
Gambar 3. 6 Rancangan <i>ERD</i>	33

DAFTAR TABEL

Tabel 2. 1 Perbandingan Penelitian Terdahulu	8
Tabel 3. 1 Kebutuhan Sistem	25
Tabel 3. 2 <i>Increment Plan</i>	27
Tabel 3.3 Struktur Tabel <i>user</i>	34
Tabel 3.4 Perancangan <i>API</i>	34
Tabel 3. 5 Rancangan <i>Unit Testing</i> Autentikasi.....	35
Tabel 3. 6 Rancangan <i>Integration Testing</i> Autentikasi.....	36
Tabel 3. 7 Rancangan <i>Peformance Testing</i> Autentikasi	36
Tabel 3. 8 Rancangan <i>Security Testing</i> Autentikasi.....	37

I. PENDAHULUAN

1.1 Latar Belakang

Klinik kecantikan Mische adalah salah satu klinik kecantikan yang berdiri sejak tahun 2014 di kota Pekanbaru. Klinik ini menerima berbagai macam perawatan wajah dimulai dari jerawat, bopeng, flek hitam dan pelayanan kecantikan lainnya yang masih berkaitan dengan kesehatan wajah. Sehingga klinik ini memiliki *customer* dari berbagai kalangan dimulai dari remaja hingga dewasa dikarenakan didukung dengan terdapatnya produk *skincare* dan juga *treatment* yang berkualitas. Berdasarkan hasil wawancara dengan Bapak Fauzy Ramadhan selaku *Supervisor*, rincian lengkapnya dapat dilihat pada **Lampiran C**, didapatkan permasalahan yang memerlukan pembuatan sistem informasi sebagai bentuk untuk meningkatkan kinerja operasional dari klinik kecantikan Mische. Oleh karena itu, pada penelitian ini akan berfokus pada pengembangan sistem informasi bagian *backend* yang berguna untuk mengelola logika sistem dan integrasi fitur-fitur yang digunakan.

Backend berperan penting sebagai penghubung antara sisi pengguna (*Frontend*) dengan sisi server yang melakukan pengelolaan data pada sistem informasi melalui *API* (*Application Programming Interface*), dan juga penggunaan *API* ini dapat meningkatkan efisiensi pengembangan sebagaimana dijelaskan oleh Bachtiar dkk. (2024). Sehingga dalam melakukan pengembangan *backend* memiliki aspek yang perlu diperhatikan, salah satunya adalah sisi keamanan data. Karena aspek ini merupakan kebutuhan utama dalam melakukan pengembangan sistem informasi terutama dalam melakukan perlindungan integritas layanan sebagaimana yang dipaparkan oleh Prasetya & Suharjo (2025). Selain keamanan, Menurut Maulana dkk. (2024) aspek lain yang perlu diperhatikan juga di bagian penyimpanan data, pengelolaan data, pemrosesan, permintaan data dan menyediakan layanan menjadi aspek yang perlu diperhatikan dalam pengembangan *backend*. Seluruh aspek tersebut berpengaruh besar terhadap kinerja sistem informasi secara keseluruhan, karena *frontend* sangat bergantung pada layanan yang disediakan oleh *backend* agar dapat berjalan dengan baik dan optimal.

Untuk melakukan pengembangan sistem informasi klinik kecantikan mische terutama di bagian *backend* diperlukan beberapa teknologi, diantaranya bahasa pemrograman dan *framework* yang mendukung bahasa pemrograman pada sisi server (*server-side*). Bahasa pemrograman yang banyak digunakan dan berjalan pada sisi server salah satunya adalah *PHP* (*Hypertext Preprocessor*). Dalam pembuatan *backend* memerlukan sebuah

framework yang dapat memudahkan dalam pengembangan agar lebih terstruktur dan mudah untuk dipahami logika pemograman yang dibuat. Sehingga dalam penelitian kali ini akan menggunakan *framework* yang mendukung bahasa pemograman *PHP*, dan *framework* paling sering digunakan oleh pihak pengembang *backend* salah satunya adalah *laravel*. Kemudian didukung dengan data pada tahun 2021, *laravel* menjadi *framework PHP* yang paling banyak digunakan dengan jumlah persentase mencapai 67% seperti yang dikemukakan oleh Kansha (2023). Dari data ini dapat di ambil kesimpulan bahwa tingginya tingkat komunitas dan juga kemudahan menjadi poin tambahan dan alasan pada penelitian ini menggunakan *framework laravel*.

Berdasarkan aspek penting dan permasalahan yang sudah diuraikan sebelumnya dapat disimpulkan bahwa dibutuhkannya sebuah sistem informasi untuk klinik kecantikan mische dalam mendukung operasionalnya. Kemudian pada penelitian ini difokuskan dibagian *backend* sehingga memberikan sebuah sistem *backend* yang sesuai dengan kebutuhan *frontend* baik dari segi keamanan data, pengiriman data, pemrosesan data, serta pengelolaan data yang optimal untuk mendukung performa sistem secara keseluruhan. Selain itu, sistem backend juga dirancang agar mempermudah dalam melakukan pengembangan dan pemeliharaan dimasa depan, sehingga dapat mengikuti teknologi yang dibutuhkan oleh pihak klinik kecantikan mische.

Sehingga harapan nya dengan penelitian ini berlangsung dapat memberikan sebuah kemudahan dalam pengolahan data yang akurat, aman, cepat dan terstruktur. Sehingga memudahkan untuk melakukan pertukaran data antara *frontend* dan *backend* baik dalam menampilkan data yang tersimpan pada *database* maupun mengirim data yang di *input*-kan oleh *user* melalui antar muka sistem. Kemudian dengan adanya sistem *backend* yang dirancang secara terstruktur diharapkan juga dapat mendukung kinerja *frontend* yang berjalan dengan stabil dan aman sehingga mampu memenuhi kebutuhan *frontend* secara menyeluruh dalam proses bisnis dan pelayanan di klinik kecantikan mische.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang sudah di paparkan sebelumnya, sehingga didapatkan rumusan masalah di antaranya sebagai berikut ini:

1. Bagaimana cara membuat sistem *backend* yang dapat mengakomodasi akan kebutuhan *frontend* untuk menampilkan data dengan baik?
2. Bagaimana memastikan aspek keamanan, pengolahan, dan penyimpanan data pada sistem *backend* agar sesuai dengan kebutuhan *frontend* dan mendukung performa sistem secara keseluruhan?

1.3 Batasan Masalah

Agar penelitian ini terarah dan tidak menyimpang dari tujuan utama, maka ruang lingkup penelitian dibatasi pada beberapa hal berikut:

1. Penelitian ini hanya berfokus pada pengembangan sistem informasi pada sisi *backend* untuk Klinik Kecantikan Mische.
2. Pengembangan *backend* menggunakan *framework laravel* berbasis *PHP* dengan pendekatan *Rest API* yang dapat diakses oleh *frontend*.
3. Fokus utama pengembangan *backend* meliputi pengelolaan data, keamanan sistem, pengiriman data antar sistem, dan optimasi performa sistem.

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Mengembangkan sistem backend untuk mendukung sistem informasi pada Klinik Kecantikan Mische yang mampu mengelola data layanan dan operasional secara aman, cepat, dan terstruktur.
2. Membangun backend berbasis *framework laravel* dengan pendekatan *Rest API* yang dapat terintegrasi dengan *frontend* dan menunjang performa sistem informasi secara keseluruhan.
3. Memastikan aspek-aspek penting dalam pengembangan *backend* seperti keamanan data, pemrosesan data, pengiriman data, serta manajemen data dapat berjalan secara optimal.

1.5 Manfaat Penelitian

Adapun manfaat yang diharapkan dari penelitian ini adalah:

1. Memberikan kemudahan bagi sisi *frontend* dalam melakukan pertukaran data dengan *backend*, baik dalam menampilkan data dari *database* maupun menyimpan data yang dikirimkan *user* melalui tampilan antar muka sistem.
2. Membantu *frontend* dalam menjaga keamanan dan kecepatan respon data, sehingga menciptakan sistem dengan performa yang baik dalam pengolahan data.

1.6 Metodologi Penelitian

Metode penelitian yang digunakan dalam pembuatan proyek akhir ini adalah metode *incremental*, yang dilakukan secara bertahap berdasarkan kebutuhan sistem dan umpan balik dari pengguna. Adapun tahapan metodologi penelitian ini adalah sebagai berikut:

1. Studi Literatur

Tahap ini dilakukan untuk mengumpulkan referensi dari berbagai sumber seperti buku, jurnal, dan penelitian terdahulu yang berkaitan dengan pengembangan *backend*, keamanan data, integrasi sistem, serta penggunaan *framework Laravel* sebagai teknologi utama dalam pengembangan *backend* sistem informasi.

2. Analisis Kebutuhan Pengguna

Analisis dilakukan berdasarkan hasil wawancara dengan pihak klinik untuk mengidentifikasi kebutuhan *backend* dan kebutuhan integrasi dengan antarmuka pengguna (*frontend*). Temuan dari analisis ini kemudian dirumuskan dalam bentuk *user stories* untuk memudahkan proses perancangan sistem.

3. Perancangan Sistem *Backend*

Pada tahap ini dilakukan perancangan struktur sistem backend, termasuk perancangan *database*, *endpoint API*, serta skema autentikasi dan *otorisasi* pengguna. Seluruh perancangan dilakukan dengan mempertimbangkan *security* dan *performance*.

4. Pengembangan Sistem

Tahap implementasi dilakukan dengan menggunakan *framework Laravel* sebagai *tools* utama dalam membangun *backend* dan menggunakan *PHP* sebagai basis Bahasa pemrogramannya. Pengembangan dilakukan secara bertahap berdasarkan modul atau fitur yang telah dirancang.

5. Pengujian Sistem

Pengujian dilakukan untuk memastikan bahwa fungsi *backend* berjalan sesuai dengan kebutuhan dan dapat terintegrasi dengan baik ke sisi *frontend*. Pengujian meliputi *unit testing*, *Integration testing*, *performance testing* dan *security testing*.

6. Evaluasi Sistem

Setelah pengujian dilakukan, sistem dievaluasi berdasarkan hasil pengujian dan *feedback* dari pengguna internal (staf klinik). Evaluasi ini bertujuan untuk mengetahui efektivitas *backend* dalam mendukung operasional klinik dan memastikan bahwa sistem dapat berjalan secara stabil, aman, dan optimal.

1.7 Sistem Penulisan

Sistematika penulisan dalam proposal penelitian ini disusun agar memberikan gambaran yang terstruktur dan memudahkan dalam memahami isi keseluruhan dokumen. Adapun sistematika penulisan laporan ini adalah sebagai berikut:

BAB I PENDAHULUAN

Bab ini menjelaskan latar belakang permasalahan, rumusan masalah, tujuan penelitian, manfaat penelitian, ruang lingkup atau batasan masalah, dan sistematika penulisan.

BAB II TINJAUAN PUSTAKA

Berisi teori-teori dan literatur yang relevan dengan penelitian, termasuk pengertian sistem informasi, konsep *backend*, *framework Laravel*, serta penelitian terdahulu yang mendukung.

BAB III PERANCANGAN

Pada bab ini menjelaskan tentang perancangan sistem yang terdiri dari arsitektur sistem, *ERD*, *Endpoint API*, test plan dan hal yang berkaitan lain dalam perancangan.

BAB IV JADWAL DAN ANGGARAN BIAYA

Berisi informasi mengenai anggaran yang akan digunakan selama tahapan penelitian ini, kemudian penjadwalan dari kegiatan penelitian yang dilakukan.

II. TINJAUAN PUSTAKA

2.1 Review Penelitian

Penelitian pertama yang dilakukan oleh Zulkifli & Rahmalisa (2022) membahas mengenai permasalahan dalam melakukan pengelolaan data pasien dalam Klinik Kecantikan Qatrunnada yang masih menerapkan pembukuan secara manual. Hal ini menciptakan keterlambatan dalam melakukan pencatatan, kesalahan pencatatan, duplikasi data sehingga merugikan efektifitas klinik. Untuk itu, dilakukan perancangan dan implementasi sistem informasi berbasis website dengan menggunakan *framework Laravel* yang berbasis bahasa pemrograman *PHP*, kemudian didukung dengan penggunaan *MySQL* sebagai *database*. Penelitian ini juga menggunakan metode pengembangan sistem waterfall untuk memastikan tahap perancangan sampai implementasi berjalan secara sistematis dan terstruktur. Hasil implementasi sistem informasi ini menunjukkan peningkatan efektifitas dalam pengolahan data dibandingkan dengan metode manual. Data tersimpan dengan aman, mudah diakses, dan meminimalisasi kehilangan data. Klinik juga dapat mengakses laporan kunjungan secara realtime dan mencetak laporan bulanan yang sebelumnya tidak tersedia.

Penelitian kedua dilakukan oleh Amrina dkk. (2021) yang mengangkat masalah tentang pengelolaan data yang masih konvensional, sehingga menimbulkan ketidak efisiensi dalam menjalankan kegiatan operasional, semisal seorang dokter ketika mencari data pasien yang diinginkan namun membutuhkan waktu yang lama. Sehingga hal ini melandasi dalam melakukan pengembangan sistem informasi klinik kecantikan yang menerapkan *framework Codeigniter* dengan basis bahasa pemrogramannya *PHP* serta didukung dengan *MySQL* sebagai *database*-nya. Untuk metode pengembangan sistem yang digunakan adalah *Scrum*. Sistem ini dirancang untuk mampu mengelola berbagai aspek klinik kecantikan seperti pasien, rekam medis, antrian pasien, dan manajemen logistik. Hasil dari implementasi sistem informasi menghasilkan fitur-fitur seperti pencatatan data pasien yang cepat, pencarian data yang mudah, pengelolaan antrian, hingga pembuatan laporan operasional, sehingga dengan fitur tersebut dapat memberikan efisiensi dan efektifitas operasional klinik.

Kemudian untuk penelitian ketiga adalah penelitian yang dilakukan oleh Agustin (2025) pada penelitian ini dilandasi dengan permasalahan dalam melakukan pengelolaan jadwal treatment pasien di Klinik Kecantikan *Ramdani Skincare* yang masih dilakukan secara manual dan konvensional, sehingga seringkali terjadi keterlambatan dan risiko

kelupaan pengingat jadwal pasien. Sehingga untuk mengatasi permasalahan yang terjadi pada penelitian ini adalah dengan melakukan pengembangan sebuah sistem informasi yang dapat melakukan pengingatan secara otomatis terkait jadwal *treatment* berbasis *SMS Gateway*. Pada implementasinya diterapkan beberapa teknologi didalamnya yang meliputi bahasa pemrograman *PHP* dengan *framework* *Laravel* yang mendukung pengembangan aplikasi *web* secara efisien dan *MySQL* sebagai basis data untuk penyimpanan informasi pasien dan jadwal *treatment* yang terstruktur, dan metode pengembangan yang diterapkan adalah *waterfall*. Dengan sistem yang dikembangkan ini dihasilkan sebuah fitur yang melakukan pengingatan secara otomatis kepada pasien yang terjadwal *treatment* melalui *SMS*, sehingga dapat mengurangi resiko kelupaan dan memastikan pasien mendapatkan layanan tepat waktu. Sehingga dengan penerapan sistem pengingat pada Klinik Kecantikan *Ramdani Skincare* menunjukkan peningkatan dalam manajemen jadwal pasien serta efektivitas pelayanan klinik secara keseluruhan.

Penelitian yang keempat dilakukan oleh Badri & Sari (2023) penelitian ini memiliki tujuan untuk melakukan pengembangan sistem informasi terutama berfokus pada bagian manajemen penjualan produk obat untuk apotek, yang dimana sebelumnya masih menggunakan metode pengelolaan secara tradisional dengan kendala dalam melakukan pengelolaan data stok dan transaksi. Sehingga untuk mengatasi permasalahan yang terjadi, peneliti melakukan perancangan sistem yang tidak hanya mampu melakukan pencatatan terhadap penjualan, namun juga mempermudah dalam melakukan pengelolaan inventaris obat dan memberikan laporan penjualan. Dalam penelitian ini digunakan metode *incremental* yang dimana pada pengembangannya dilakukan secara bertahap melalui beberapa iterasi didalamnya. Kemudian penelitian ini dilakukan sebanyak dua iterasi di dalamnya, pada iterasi pertama dilakukan pengembangan di bagian pengelola data master obat dan transaksi penjualan harian, untuk iterasi kedua untuk pembuatan laporan penjualan dan analisis data. Metode *incremental* dipilih karena memiliki fleksibilitas yang tinggi, dapat dilakukan penyesuaian kembali dengan bantuan umpan balik dari pengguna, serta dapat melakukan pengembangan menghadirkan fitur yang siap pakai dibandingkan dengan metode lain seperti *waterfall*. Hasil dari implementasi metode ini menunjukkan bahwa sistem dapat dilakukan pengembangan secara efektif dan efisien, dengan dari setiap tahapan memberikan evaluasi dan penyempurnaan yang sesuai dengan kebutuhan dari pengguna.

Penelitian yang kelima adalah penelitian yang dilakukan oleh Filiana dkk. (2022) pada penelitian ini mengangkat dari permasalahan penting terkait dengan ketiadaanya sistem

informasi terpusat yang menyediakan dan mengelola dari pasar tradisional di kota Yogyakarta. Sehingga permasalahan tersebut melatar belakangi dilakukan nya pengembangan *Rest API* untuk sistem informasi pasar tradisional kota Yogyakarta. Metode pengembangan yang diterapkan pada penelitian ini menggunakan *incremental* yang dimana terdiri dari tiga iterasi, dimulai dari iterasi pertama membuat pengelolaan data utama pasar, iterasi kedua pengelolaan deskripsi dan sejarah pasar, kemudian iterasi terakhir tentang pengelolaan fasilitas dan kekhasan pasar. Kemudian teknologi yang diaplikasikan pada penelitian ini meliputi penggunaan *laravel* sebagai *framework* dengan basis bahasa pemogramannya adalah *PHP*, dan di dukung dengan penggunaan *MySQL* sebagai *database*-nya. Penggunaan metode *incremental* dipilih karena metode ini memudahkan pengembangan yang sesuai dengan kebutuhan pengguna, serta didukung oleh proses evaluasi dan masukan pada setiap tahapannya.

Berdasarkan penelitian-penelitian yang sudah dipaparkan sebelumnya, berikut merupakan tabel 2.1 perbandinganya sebagai berikut ini :

Tabel 2. 1 Perbandingan Penelitian Terdahulu

Peneliti	Metode	Teknologi	Hasil / Fitur
Zulkifli & Rahmalisa (2022)	<i>Waterfall</i>	<i>Laravel, PHP, MySQL</i>	Digitalisasi pencatatan data pasien dan pembuatan laporan bulanan.
Amrina dkk. (2021)	<i>Scrum</i>	<i>PHP, MySQL</i>	Sistem pencatatan pasien, antrian, pengelolaan produk, dan rekam medis.
Agustin (2025)	<i>Waterfall</i>	<i>Laravel, PHP, MySQL, SMS Gateway</i>	Pengingat jadwal <i>treatment</i> otomatis melalui <i>SMS</i> .
Murdani dkk. (2022)	<i>Incremental</i>	<i>PHP, MySQL</i>	Otomatisasi laporan rugi laba & neraca, fitur rekalkulasi saldo.
Badri & Sari (2023)	<i>Incremental</i>	<i>PHP, MySQL</i>	Sistem Informasi manajemen penjualan obat, pengelolaan inventaris, serta otomatisasi laporan penjualan
Penelitian yang di ajukan	<i>Incremental</i>	<i>Laravel, My SQL</i>	Sistem Informasi <i>profile company</i> , penjualan produk serta mengelola promo dan <i>event</i> .

2.2 Landasan Teori

Terdapat beberapa landasan teori yang digunakan dalam pembuatan penelitian ini, diantaranya adalah sebagai berikut :

2.2.1 Klinik Kecantikan Mische

Klinik kecantikan merupakan fasilitas pelayanan kesehatan yang menyelenggarakan pelayanan medis dasar dan/atau spesialisasi yang berfokus pada estetika dan perawatan kulit. Menurut Permenkes No. 9 Tahun 2014, klinik adalah fasilitas pelayanan kesehatan perorangan yang dapat diselenggarakan oleh pemerintah, pemerintah daerah, atau swasta, dan menyediakan pelayanan promotif, preventif, kuratif, serta rehabilitatif. Klinik kecantikan biasanya memberikan layanan seperti perawatan wajah, pengobatan jerawat, terapi anti-aging, serta penjualan produk perawatan kulit. Dalam perkembangannya, klinik kecantikan membutuhkan sistem informasi yang dapat mendukung pelayanan tersebut agar berjalan dengan baik, profesional, dan mampu membangun kepercayaan pelanggan.



Gambar 2. 1 Logo Klinik Kecantikan Mische

Dalam penelitian ini, penerapan dilakukan pada Klinik Kecantikan Mische, sebuah klinik kecantikan yang telah berdiri sejak tahun 2014 di Kota Pekanbaru, Riau, Indonesia, yang memiliki berbagai layanan estetika dan perawatan kulit yang profesional. Berdasarkan hasil wawancara yang terdapat pada **Lampiran C**, dalam menjalankan operasionalnya, klinik kecantikan Mische menyediakan berbagai produk *skincare* yang disesuaikan dengan kebutuhan perawatan kulit *customer*. Kemudian terdapat juga pelayanan treatment kulit yang dapat dilakukan reservasi, sehingga setiap customer dapat menyesuaikan dengan waktu yang dimilikinya. Selain itu, klinik ini juga sering membuat sebuah *event* di berbagai lokasi sebagai sarana untuk melakukan promosi dan edukasi serta dilengkapi dengan penawaran promo baik kode voucher maupun diskon khusus yang menarik. Strategi ini bertujuan untuk menarik lebih banyak customer serta meningkatkan loyalitas customer terhadap klinik kecantikan mische.

2.2.2 Sistem Informasi

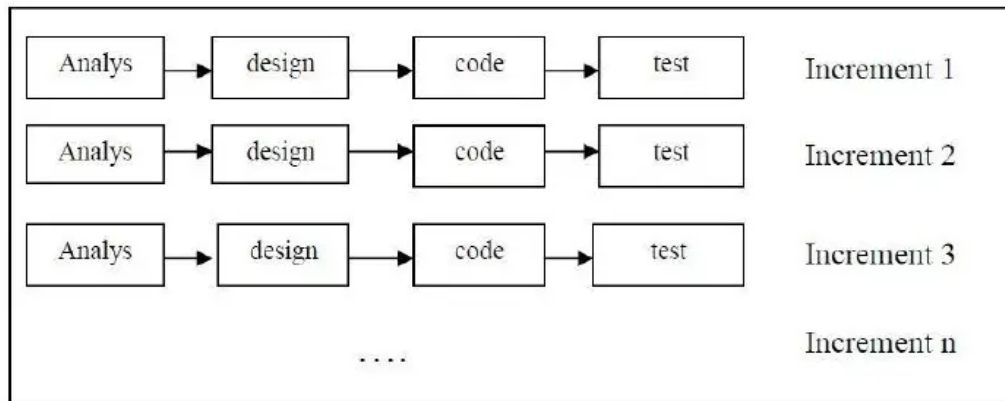
Sistem informasi merupakan suatu sistem yang dirancang untuk mengumpulkan, menyimpan, mengelola, dan mendistribusikan informasi guna mendukung proses pengambilan keputusan dan aktivitas operasional dalam suatu organisasi. Menurut Andini dkk. (2023), sistem informasi merupakan sebuah sistem yang melakukan sebuah pengumpulan, menyimpan, mengelola, memproses, dan penyebaran informasi dalam sebuah instansi atau organisasi. Sistem informasi juga berguna untuk meningkatkan efisiensi dalam melaksanakan operasional instansi atau organisasi serta memberikan dukungan strategi dalam melakukan pengambilan sebuah keputusan. Sehingga sistem informasi dapat berguna untuk memenuhi akan kebutuhan user dalam menanggulangi permasalahan yang di hadapi, contohnya memberikan kebijakan dan startegi bisnis yang dapat diterapkan selama melaksanakan operasional perusahaan, dan hal ini memberikan dampak dan keuntungan berupa meningkatkan efisiensi dalam sebuah instansi atau organisasi.

Menurut Oktaviyana (2023) sistem informasi mengandung tiga aktivitas dasar di dalamnya, yaitu terdapat aktivitas untuk melakukan sebuah masukan (*input*), kemudian aktivitas untuk melakukan sebuah pemrosesan (*processing*), kemudian yang terakhir terdapat aktivitas keluaran (*output*). Kemudian dari ketiga aktivitas ini akan menghasilkan sebuah informasi yang dibutuhkan oleh organisasi untuk melakukan pengambilan keputusan, strategi pemasaran ataupun menciptakan produk baru . Masukan (*input*) berperan untuk melakukan pengumpulan data mentah. Kemudian pemrosesan (*Processing*) merupakan tahapan pengolahan data dari *input*-an menjadi sebuah informasi yang dapat digunakan pada organisasi. Dan yang terakhir keluaran (*output*) yaitu hasil dari proses yang dapat digunakan oleh perusahaan. Sehingga sistem informasi sangat penting dalam organisasi terutama dalam pengambilan keputusan organisasi untuk kedepannya.

2.2.3 Metode Pengembangan *Incremental*

Metode *Incremental* adalah salah satu metode pengembangan perangkat lunak (*Software Development*) yang dilakukan dengan pendekatan secara bertahap (*increment*) serta melibatkan iterasi yang dilakukan secara berulang selama pengembangan sistem, sebagaimana dijelaskan oleh Sitepu dkk. (2023). Metode ini juga melakukan penerapan dari elemen elemen yang ada pada metode *waterfall* dengan di bantu filosofi iteratif dari metode *prototyping*. Pada metode ini dibagi menjadi bagian bagian kecil yang dimana disebut *increment*. Untuk disetiap *increment* dilakukan penambahan fitur atau fungsi yang sudah dapat dilakukan pengujian, setelah satu *increment* selesai dan sudah memenuhi akan

kebutuhan dari *user* maka tahapan pengembangannya dilanjutkan ke *increment* yang berikutnya hingga sistem akan jadi sepenuhnya. Sehingga dengan metode ini fitur-fitur yang sudah dibuat dan sesuai dengan kebutuhan pengguna nantinya bisa langsung digunakan oleh *user*. Selain itu, metode *incremental* juga memberikan kemudahan dalam melakukan penyesuaian baik itu perbaikan maupun penambahan kebutuhan dalam pengembangan sehingga dengan metode ini dapat langsung menyesuaikan dengan kebutuhan dari *user*.



Gambar 2. 2 Tahapan Metode *Incremental*

Sumber: Habib & Kartika (2020)

Pada metode pengembangan *Incremental* memiliki empat tahapan di dalamnya, seperti yang dijelaskan oleh Sutjiadi dkk. (2022) sebagai berikut ini:

a) Analisis

Tahapan analisis pada metode *incremental* adalah dimana dilakukannya identifikasi terhadap masalah yang terjadi kepada calon pengguna sistem agar menemukan kebutuhan utama dan juga kebutuhan pendukung pada sistem tersebut.

b) Desain

Tahapan desain merupakan tahapan melakukan perancangan untuk sistem yang akan dikembangkan berdasarkan kebutuhan yang telah didapatkan pada tahapan analisis.

c) Implementasi

Tahapan implementasi pada metode *incremental* yaitu tahapan dimana dilakukan nya implementasi ide desain ke dalam sebuah baris kode sistem yang akan dibangun.

d) Testing

Tahapan testing merupakan tahapan validasi dari hasil pembuatan pada tahapan sebelumnya apakah penerapan kode sudah sesuai dengan kebutuhan pengguna atau belum.

2.2.4 Entity Relation Diagram (ERD)

Menurut Akbar & Haryanti (2023) *Entity Relation Diagram (ERD)* adalah sebuah diagram yang dibentuk secara terstruktur yang digunakan untuk merancang sebuah basis data dari sistem yang sedang di bangun. Dengan kata lain, *ERD* menjadi suatu model untuk menjelaskan hubungan antar data dalam basis data berdasarkan objek-objek dasar data yang mempunyai hubungan antar relasi. Selain itu *ERD* juga menjelaskan bagaimana ketentuan, batasan agar tetap konsisten dan valid selama diproses oleh sistem.

Dalam melakukan perancangan *ERD* perlu diperhatikan tiga konsep didalamnya, sebagai mana di jelaskan oleh Togatorop dkk. (2021) *ERD* memiliki tiga konsep utama sebagai berikut ini:

a) *Entitas*

Entitas merupakan objek penting dalam sebuah organisasi seperti orang, tempat, benda ataupun kejadian yang harus dicatat pada basis data. *Entitas* juga terbagi menjadi dua jenis, yaitu *strong entity* (tidak bergantung pada entitas lain) dan *weak entity* (yang bergantung pada entitas lain).

b) *Atribut*

Pada *ERD* terdapat sebuah atribut yang dimana atribut adalah sebuah bagian yang mendeskripsikan *entitas*. Atribut terbagi atas beberapa jenis yaitu, *simple attribute* dan *composite attribute*, *single valued attribute* dan *multi value attribute*, *derived attribute*, *key attribute*. *Primary key* merupakan sebuah nama dari atribut yang bersifat unik dan tidak memiliki kemiripan dengan *atribut* lain yang berguna untuk memberitahu lokasi sesuatu data.

c) *Relasi*

Relasi adalah sebuah hubungan antara satu entitas dengan entitas lainnya yang saling berkaitan yang dilambangkan dengan bentuk belah ketupat (*diamond*). *Relasi* juga terbagi atas tiga jenis yaitu *unary*, *binary*, *ternary*.

Dari ketiga konsep tersebut yaitu *Entity*, *Atribut*, dan *Relasi* memiliki simbol dalam pembuatannya seperti pada gambar 2.3 sebagai berikut ini:



Gambar 2. 3 Simbol *Entity*, *Atribut*, dan *Relasi*

Sumber: Togatorop dkk. (2021)

2.2.5 Database

Database adalah sekumpulan data yang terorganisir secara sistematis untuk memudahkan penyimpanan, pengelolaan, dan pengambilan informasi. Menurut penelitian, *database* berfungsi sebagai alat yang memungkinkan pengguna untuk mengakses dan memanipulasi data secara efisien melalui sistem manajemen basis data (*DBMS*). Keunggulan utama *database* terletak pada kemampuannya untuk mengurangi redundansi data, meningkatkan integritas informasi, dan mendukung analisis data yang kompleks. Penelitian menunjukkan bahwa penerapan *database* dalam berbagai bidang, seperti bisnis dan pendidikan, dapat meningkatkan efisiensi operasional serta mendukung pengambilan keputusan yang berbasis data sebagaimana dijelaskan oleh Fahzirah & Nasution (2024).

Database juga memiliki berbagai macam jenis nya, menurut Daud Pujas dkk. (2024) terdapat dua jenis *database* yang paling banyak digunakan oleh *software developert* dalam mengembangkan sebuah sistemnya, kedua jenis *database* tersebut adalah:

a) *Database SQL*

Database SQL (Structured Query Language) adalah *database* yang memiliki bentuk yang terstruktur dan saling berelasi satu tabel dengan tabel lain nya berdasarkan Primary Key.

b) *Database NoSQL*

Database NoSQL (*No Structured Query Language*) adalah *database* memiliki konsep tidak berelasi (*non-relational*) yang dapat melakukan penyimpanan data tanpa memerlukan definisi struktur tabel terlebih dahulu. *NoSQL* dapat menyimpan berbagai macam bentuk data baik itu dalam bentuk dokumen, *graph*, dan *key-value*.

Kedua *database* tersebut memiliki kelebihan dan kekurangan nya masing masing, sehingga diperlukan analisis akan kebutuhan terlebih dahulu sebelum memilih dan menggunakannya.

2.2.6 MySQL

Menurut Hermiati dkk. (2021) *MySQL* adalah salah satu sistem manajemen basis data relasional *RDBMS (Relational Database Management System)* yang cukup terkenal, yang dimana dapat melakukan pengolahan terhadap data yang terstruktur karena bahasa yang digunakan adalah bahas *SQL (Strukture Query Language)*. *Database* relasional menyimpan data pada tabel-tabel yang terpisah, bukan menyimpan data dalam ruang Menurut Musliyana & Helinda (2022) *MySQL* merupakan sistem *database* yang banyak digunakan untuk pengembangan aplikasi *web* karena pengolahan datanya sederhana dan memiliki keamanan yang baik. Kemampuannya untuk menyimpan data dalam jumlah besar dan memberikan akses cepat terhadap data melalui *query SQL* menjadikannya pilihan utama dalam pengolahan data.



Gambar 2. 4 Logo MySQL

Kemudian *MySQL* bersifat *open source* dan juga memiliki kinerja yang cukup baik karena *MySQL* ini tidak memerlukan penggunaan *RAM* yang besar, serta juga mendukung dalam penggunaan *multiuser*, sebagaimana dikemukakan oleh Silalahi (2022). Keunggulan keunggulan ini menjadikan *MySQL* sebagai pilihan yang banyak digunakan oleh *developer* dalam melakukan pengembangan sistem yang membutuhkan sebuah tampilan yang dinamis. Selain itu *MySQL* memiliki kompetabilitas akan bahasa yang cukup banyak dan juga terhadap *framework modern* sehingga mempermudah dalam melakukan proses integrasi terutama terhadap sisi *backend* yang langsung berkomunikasi dengan *database*. Kemampuan *MySQL* dalam menangani beban kerja yang tinggi secara efisien, serta kemudahan integrasinya dengan berbagai teknologi dan *framework*, semakin memperkuat posisinya sebagai pilihan utama bagi para *developer* dalam membangun aplikasi yang handal dan skalabel.

2.2.7 Backend

Backend merupakan sebuah kumpulan dari beberapa program dan *script* yang saling terintegrasi kemudian dijalankan pada server dan bekerja di belakang dari sebuah sistem yang digunakan oleh pengguna. *Backend* dapat dikatakan juga sebagai sebuah penampung dari logika inti dan fungsional pada sebuah sistem, sebagaimana dikemukakan oleh Herdiyatomoko (2022). Sistem *backend* akan memastikan proses melakukan penampilan dan juga pengiriman data ke *frontend* dilakukan melalui metode terprogram. Kemudian *backend* terdiri dari beberapa komponen dimulai dari server, *database*, dan *API* (*Application Programming Interface*) yang membantu untuk melakukan interaksi dengan *frontend* secara aman dan terstruktur. Dengan kata lain *backend* memastikan dari semua interaksi yang dilakukan *user* seperti melakukan pemesanan produk, penjadwalan dan lain lainnya yang dilakukan pada sistem melalui *frontend* dapat berjalan dengan baik sesuai dengan logika yang sudah dirancang.

2.2.8 Laravel

Laravel adalah *framework* merupakan salah satu *framework* yang berbasis bahasa pemrograman *PHP* yang dimana dapat digunakan untuk melakukan pengembangan sistem terutama sistem *backend*, sebagaimana dikemukakan oleh Melyani dkk. (2023). Kemudian pada *framework laravel* juga memiliki berbagai macam fitur didalam yang dapat membantu *developer* dalam melakukan pengembangan sistem. Untuk fitur yang disediakan oleh *framework laravel* diantaranya adalah *Bundles*, *Eloquent ORM* (*Object-Relational-Mapping*), *Query Builder*, *Resource Controller*, *Blade*, *Migration*, *Middleware*, dan *Automatic Pagination*. Sehingga hal ini menjadikan keunggulan bagi *framework laravel* dari segi penulisan kode yang lebih singkat, mudah untuk dipahami, dan ekspresif. Sehingga *framework laravel* sangat cocok digunakan untuk melakukan pengembangan sistem *backend* karena didukung penuh dengan fitur dari *framework laravel*.



Gambar 2. 5 Logo Laravel

Laravel adalah salah satu *framework* yang menerapkan konsep MVC (*Model-View-Controller*) di dalamnya, seperti yang tampak pada gambar 2.6. Menurut Salam dkk. (2023) MVC membagi aplikasi menjadi tiga komponen utama yang terpisah, dengan pembagian fungsi sebagai berikut:

a) *Model*

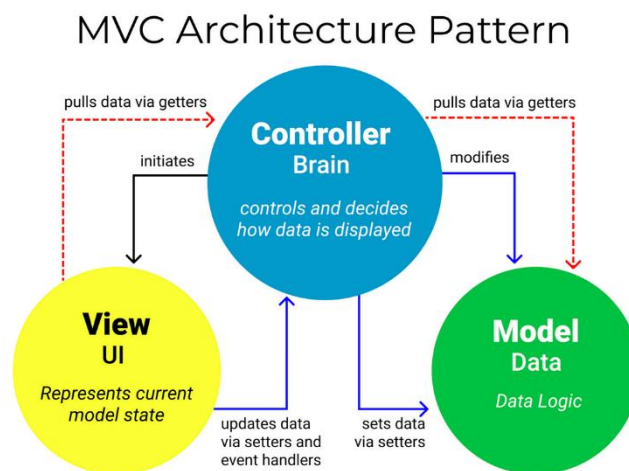
Model adalah kelas logika dari sistem yang berguna untuk melakukan penghubung antar sistem dengan *database*, serta digunakan juga untuk melakukan manipulasi data.

b) *View*

View adalah kelas yang mewakili fungsi antarmuka suatu sistem dan menjadi penghubung interaksi antara sistem dengan pengguna.

c) *Controller*

Controller berfungsi sebagai penghubung antara *View* dan *Model*. *Controller* menerima *input* dari *View*, memproses data tersebut, kemudian mengirimkannya ke *Model* untuk disimpan ke dalam *database*, dan sebaliknya mengatur data dari *Model* agar dapat ditampilkan kembali ke *View*.



Gambar 2. 6 Arsitektur MVC

Sumber: Rahmawati & Sumarsono (2024)

2.2.9 PHP

PHP adalah singkatan dari *Hypertext Preprocessor* merupakan bahasa yang berbasis *server-side* yang dimana berarti dilakukan pengolahan di bagian server dan di desain untuk melakukan pengembangan *web*, sebagaimana dikemukakan oleh Noviana (2022). *PHP* merupakan bahasa skrip yang ditambahkan ke dalam *HTML* kemudian digunakan untuk membuat *website* yang memiliki tampilan yang dinamis dan dapat disesuaikan

dengan kebutuhan *user*, sehingga informasi yang diterima oleh *client* selalu *up to date*, seperti dijelaskan oleh Apandi & Istini (2023). *PHP* dapat menjalankan kode secara langsung di server sebelum konten dikirimkan ke browser, sehingga memungkinkan interaksi yang lebih kompleks dengan pengguna. Bahasa ini juga mendukung berbagai protokol komunikasi yang penting dalam pengembangan aplikasi *web* modern.

PHP memiliki *sintaks* yang sederhana, fleksibel dan didukung dengan komunitas yang cukup besar. Sehingga bahasa pemrograman *PHP* dapat digunakan untuk melakukan pengembangan sistem *backend* karena mampu melakukan pengolahan bisnis serta dapat menjembatani pertukaran data antara server dengan klien dengan baik.



Gambar 2. 7 Logo *PHP*

PHP juga memiliki beberapa kelebihan seperti yang dipaparkan oleh Sitanggang dkk. (2022) sebagai berikut ini:

a) Bahasa *multiplatform*

PHP merupakan bahasa yang dapat di jalankan di berbagai macam platform yang ada di dunia mulai dari *Linux*, *Unix*, *Macintos* dan juga *Windows*.

b) *Open source*

PHP bersifat *open source* yang berarti dapat digunakan oleh siapa saja dan gratis tanpa harus membayarkan royalti kepada pihak pengembang bahasa *PHP*.

c) Memiliki banyak dukungan *web server*

Ketersediaan *web server* yang mendukung *PHP* sangat banyak di dunia ini mulai dari *apche*, *IIS*, *Lighttpd*, *nginx*, hingga *Xitami* dengan konfigurasi yang cukup mudah dilakukan.

d) Mudah dalam pengembangan

Dari sisi pengembangan *PHP* sudah memiliki komunitas *developer* yang besar sehingga dapat saling bertukar solusi mengenai *PHP*.

e) Mendukung banyak *database*

PHP juga memiliki berbagai macam dukungan terhadap *database* yang ada mulai dari *MySQL*, *Oracle* dan *MS-SQL*.

2.2.10 Laragon

Menurut pendapat dari Fanani dkk. (2023) *Laragon* merupakan perangkat yang digunakan oleh pihak *developer* dalam melakukan pengembangan sistem, yang dimana berguna untuk menyediakan lingkungan pengembangan secara lokal (*localhost*) bisa juga dikatakan sebagai penyedia *server virtual* yang dijalankan pada sistem operasi yang digunakan pihak pengembang. Hal ini sangat penting terutama dalam fase awal pembangunan aplikasi, karena *developer* dapat melakukan pengujian secara langsung tanpa harus bergantung pada server produksi. Dengan adanya *Laragon*, proses konfigurasi yang biasanya kompleks menjadi lebih sederhana dan otomatis, sehingga menghemat waktu dan meminimalkan kesalahan konfigurasi manual yang umum terjadi saat *setup server* secara manual.



Gambar 2. 8 Logo *Laragon*

Kemudian laragon memiliki banyak fungsi seperti yang di sampaikan oleh Fanani dkk. (2023) di dalam *Laragon* terdapat fungsi yang berguna untuk mendukung proses pengembangan sistem seperti apache, *PHP Server*, *PhpMyAdmin*, *MySQL*, *Composer*. Keberadaan komponen komponen tersebut dapat memudahkan pengembang tanpa melakukan pengaturan dan instalasi masing masing *tools* yang akan digunakan dalam pengembangan secara terpisah-pisah. Sehingga dengan fitur fitur yang disajikan tersebut menjadikan laragon populer dalam melakukan pengembangan sistem. Selain itu, *Laragon* juga mendukung penambahan modul atau stack tambahan seperti *Node.js*, *Python*, atau *database* lain secara fleksibel sesuai kebutuhan proyek. Keunggulan lainnya adalah antarmuka pengguna yang bersih dan intuitif, serta kemampuan *auto-start service* yang membuat *workflow* pengembangan menjadi lebih mudah dan stabil, bahkan saat sistem di-*reboot* sekalipun.

2.2.11 Rest API

Menurut Shofyan & Shita (2024) *Rest API* adalah sebuah *API (Application Programming Interface)* yang didalamnya menggunakan arsitektur *REST* untuk melakukan komunikasi antara *client* dengan *server* sistem. Sehingga membuat sistem menampilkan data, mengakses, mengolah menggunakan *HTTP (Hyper Text Transfer Protocol)* request dan di bantu dengan *URLs* dalam mengelola sumber data yang digunakan. Pendekatan ini dapat mempermudah bagi pengembang untuk melakukan pemeliharaan karena disetiap endpoint karena memiliki struktur yang konsisten. Selain itu, penggunaan *Rest API* juga bersifat stateless yang mana artinya setiap kali permintaan dari client harus dapat menyajikan semua informasi yang dibutuhkan.

Bila menggunakan *Rest API* terdapat beberapa permintaan *HTTP* yang umum digunakan seperti yang di sampaikan oleh Nurjaman dkk. (2024) pada arsitektur *REST* seperti menggunakan *GET, POST, PUT, dan DELETE* untuk dapat melakukan operasi *CRUD (Create, Read, Update, Delete)* terhadap data yang disimpan pada *server*. Penggunaan *Rest API* menjadikan komunikasi antar *client* dan juga *server* menjadi lebih sederhana dan mudah, efisien dan dapat terintegrasi dengan sistem yang berbeda *platform*. Selain itu *Rest API* mempresentasikan data nya menggunakan format yang ringan untuk dibaca seperti *JSON (JavaScript Object Notation)* dan juga *XML*. Hal ini menjadikan *Rest API* banyak digunakan pada web pada saat ini. Dukungan format data yang fleksibel ini sangat membantu integrasi antar sistem, khususnya ketika satu aplikasi membutuhkan data dari layanan lain tanpa harus melakukan parsing kompleks.



Gambar 2. 9 Arsitektur *Rest API*

Sumber: Mohidin (2022)

2.2.12 Postman

Menurut Raharja & Wijayanto (2023) Postman merupakan aplikasi yang digunakan oleh developer untuk membantu dalam proses melakukan pengujian, kemudian digunakan juga untuk melakukan dokumentasi, kolaborasi, dan juga pemantauan *API* yang dibuat secara *real-time*. *Developer* dapat menggunakan *postman* untuk melakukan pengecekan terhadap hasil *response* yang diberikan oleh *API*, yang dimana umumnya untuk *response* nya itu berbentuk *JSON*. Sehingga *Postman* sangat membantu dalam melakukan *debugging* terhadap *API* yang sedang dibuat oleh *developer* agar mengetahui *API* yang dibuat berjalan sesuai dengan fungsinya. Dalam pengembangan sistem terutama dibagian *backend* untuk menguji kestabilan dan keamanan *API* sebelum akan digunakan oleh *frontend*.



Gambar 2. 10 Logo Postman

Postman memiliki beberapa fitur unggulan yang dimiliki dan dapat membantu *developer* dalam proses melakukan pengujian dan didukung juga dengan pemaparan dari Herdiputra dkk. (2021) fitur unggulan yang terdapat pada *postman* sebagai berikut ini:

a) *Collection*

Fitur ini berguna untuk melakukan pengaturan untuk *API* dilakukan pengelompokkan atau secara dalam folder.

b) *Envirorment*

Fitur ini berguna untuk melakukan pengembangan dalam menyimpan attribute yang akan di manfaatkan pada proses request *API*.

c) *Response*

Fitur ini digunakan untuk melakukan visualisasi terhadap *API* sebelum dilakukan implementasikan pada produk asli.

d) *Mock Server*

Fitur ini akan berguna untuk membuat *API* menjadi sebuah layanan yang telah di deploy pada *server*.

e) *Script Test*

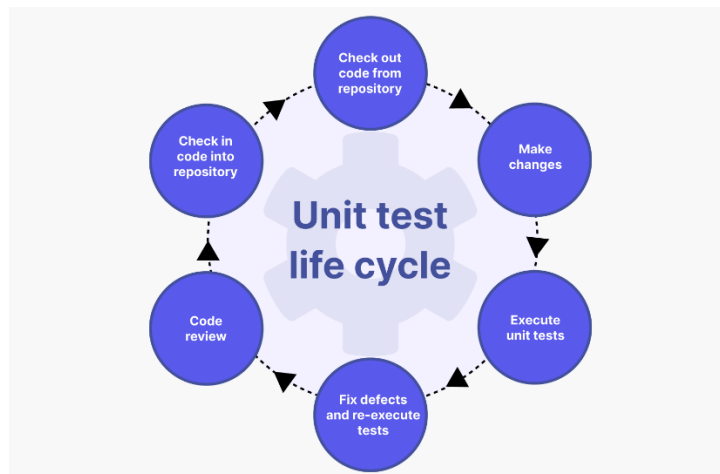
Fitur ini dapat dimanfaatkan untuk melakukan verifikasi respon dan test yang dibutuhkan pada pengembangan aplikasi.

f) *Automated Test*

Fitur *Automated Test* merupakan fitur yang dilakukan secara otomatis dengan script test yang digunakan untuk melakukan request dalam *collection*.

2.2.13 Unit Testing

Menurut Hasibuan & Dirgahayu (2021) pengujian unit testing merupakan teknik pengujian untuk memastikan kode program perangkat lunak apakah sudah sesuai dengan dan bebas dari kesalahan. Pengujian ini dilakukan pada lingkup terkecil dari perangkat lunak, yaitu pada unit atau komponen terkecil yang dapat berdiri sendiri, seperti fungsi, prosedur, atau kelas. Hal ini sejalan dengan pendapat yang disampaikan oleh Abraham dkk. (2021) juga memaparkan mengenai unit testing, yang dimana berguna untuk melakukan pengetesan dari setiap kode secara individu dimulai dari fungsi hingga methods. Penerapan unit testing juga memberikan manfaat dalam meningkatkan keandalan perangkat lunak, mempermudah proses perawatan (*maintenance*), serta mendukung praktik pengembangan modern, di mana kode diuji terlebih dahulu sebelum diintegrasikan dalam sistem yang lebih kompleks.

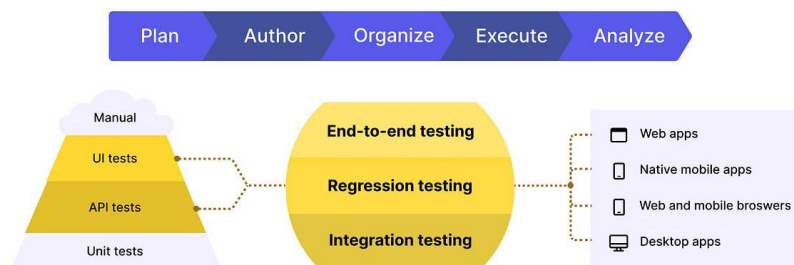


Gambar 2. 11 Siklus *Unit Testing*

Karakteristik unit test yang baik adalah cepat, sederhana, dan terisolasi. Pola *AAA* (*Arrange, Act, Assert*) digunakan untuk mengatur struktur unit test agar jelas dan mudah dibaca, di mana persiapan data, pemanggilan fungsi, dan verifikasi *output* dilakukan secara terstruktur. Implementasi unit testing dapat dilakukan dengan pendekatan *Test-First Development* (*TFD*) atau siklus iterasi yang melibatkan penulisan kode uji, eksekusi pengujian (berhasil dan gagal), serta refactoring Rizkyana dkk. (2021). Manfaat unit testing meliputi mempermudah perbaikan desain kode, mengurangi waktu *debugging*, memastikan kualitas komponen perangkat lunak, dan menyediakan pengujian menyeluruh untuk memastikan kode berkualitas baik. Dengan demikian, unit testing memainkan peran penting dalam memastikan keandalan dan efisiensi pengembangan perangkat lunak.

2.2.14 Integration Testing

Integration testing adalah salah satu testing yang biasa di gunakan untuk penggunaan pengembangan sistem. Menurut Setiawan & Risal (2024) *integration testing* merupakan pengujian yang berfokus pada bagian beberapa komponen sekaligus dan memastikan dapat berjalan dengan baik sesuai dengan kebutuhan pengguna. Pengujian ini penting walaupun kode program sudah melewati bagian unit testing, namun interaksi antar komponen belum dilakukan sehingga berkemungkinan dapat menimbulkan masalah baru yang tidak terlihat pada bagian pengujian sebelumnya. Hal ini menjadi sangat penting mengingat setiap komponen dalam sebuah sistem seringkali dikembangkan secara terpisah oleh tim yang berbeda dan memiliki dependensi data maupun fungsi yang saling terhubung.



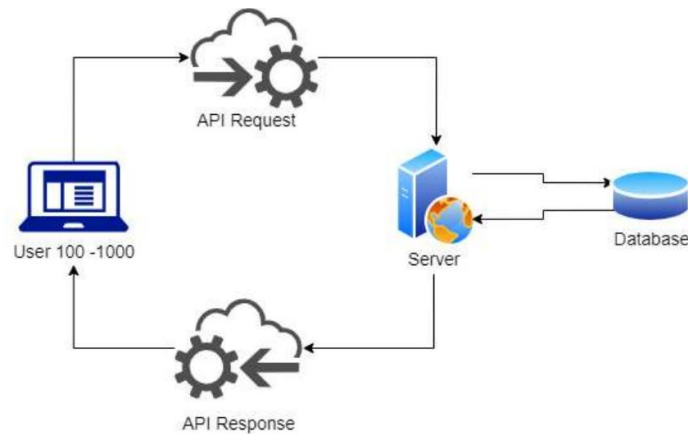
Gambar 2. 12 *Integration Testing*

Selain itu *Integration testing* memiliki tujuan khusus seperti yang di sampaikan oleh Aprila & Victor (2025) yaitu untuk mengetahui ketidak kesesuaian antara komponen satu dengan komponen lain yang saling berinteraksi atau masalah antar dua komponen yang telah di gabungan menjadi satu kesatuan. Melalui *integration testing*, masalah-masalah seperti kesalahan pada format data yang dipertukarkan, ketidakcocokan logika antar komponen, hingga kegagalan pada mekanisme komunikasi dapat diidentifikasi dan diperbaiki lebih awal sebelum sistem diimplementasikan secara penuh. Dengan demikian, pengujian ini berperan penting dalam memastikan integrasi komponen sistem yang handal, stabil, dan sesuai dengan kebutuhan pengguna. Dengan kata lain, *integration testing* membantu memastikan bahwa sistem tidak hanya berfungsi dengan baik pada masing-masing modul, tetapi juga mampu bekerja secara satu kesatuan yang utuh.

2.2.15 Performance Testing

Menurut Sianturi dkk. (2021) *performance testing* merupakan teknik pengujian perangkat lunak untuk mengetahui sejauh mana perangkat lunak dapat berjalan dibawah beban kerja yang di inginkan, atau bisa disebut katakana juga evaluasi kemampuan program

untuk beroperasi dengan benar dipandang dari sisi acuan kebutuhan, misalnya ukuran pemakaian memori. Pada pengujian ini biasanya mencakup dari pengukuran waktu respon, tingkat pemanfaatan kemampuan *CPU*, *memory*, *bandwidth* jaringan, serta kestabilan dalam mengatasi beban lain. Kemudian testing ini dilakukan dengan kondisi ketika sudah *deploy* dari sistem agar mengetahui kinerja setelah di implementasi pada lingkungan produksi.



Gambar 2. 13 Alur Pengujian *Peformance Testing*

Sumber: Mulana dkk. (2022a)

Tujuan dari *peformance testing* menurut Mulana dkk. (2022b) menyatakan bahwa performance testing memiliki tujuan untuk menguji scalability, availability dan kinerja beban baik dari sisi perangkat keras ataupun dari sisi perangkat lunak. Sehingga dengan menggunakan performance testing ini pengembang dapat memastikan sistem yang dikembangkan tidak hanya memenuhi kebutuhan fungsional tetapi juga memberikan pengalaman yang stabil dan dapat di akses dalam kondisi beban yang cukup besar. Dampak lain dari penerapan pengujian ini juga berupa meningkatkan keandalan dan kepuasan pengguna karena sistem mampu menampilkan respon yang cepat dan konsisten dalam berbagai macam kondisi.

2.2.16 *Security Testing*

Securiy Testing menurut Siregar (2021) mengatakan bahwa *security testing* adalah pengujian keamanan untuk menilai sejauh mana sistem dapat dikatakan aman. Pengujian ini dilakukan untuk mengetahui sejauh mana sistem ini dapat menjaga keamanan data yang digunakan dan dikelolanya. Tujuan dari testing ini untuk memastikan dari mekanisme sistem dapat berjalan dengan baik dan lancar serta tidak mudah untuk di tembus oleh pihak yang tidak berwenang. Dalam penggunaan *security testing* dapat membantu pengembang

untuk mengetahui tingkat keamanan dari sistem yang dibuat dan di uji. Dengan demikian penggunaan security testing akan menjadi tindakan yang baik dalam proses pengembangan perangkat lunak yang menekankan pada aspek keamanan sebagai salah satu prioritas utama.

Menurut BPMPP UMA (2022) diantara nya sebagai berikut ini:

a) *Vulnerability Scanning*

Pengujian ini merupakan jenis pengujian yang mencari kelemahan pada sebuah sistem seperti *Cross Site Scripting* atau *SQL Injection*.

b) *Security scanning*

Jenis pengujian ini dilakukan dengan cara pemindaian pada sistem yang ingin kita testing, yang dimana akan berfokus dari beberapa area seperti *applications, ports, websites, services, networks*.

c) *Penetration Testing*

Penetration testing merupakan teknik pengujian perangkat lunak dengan menerapkan simulasi serangan *cyber* di dalamnya.

d) *Risk Assessment*

Identifikasi dan analisis risiko keamanan pada perangkat dan jaringan, lalu mengelompokkan risiko berdasarkan tingkat bahaya (tinggi, sedang, rendah). Biasanya dilakukan oleh tim IT internal.

e) *Security Auditing*

Merupakan testing yang dilakukan dengan cara melakukan evaluasi pada langkah langkah keamanan pada dalam perusahaan dan dilakukan audit secara rutin.

f) *Ethical hacking*

Ethical hacking ialah pengujian keamanan yg dilakukan menggunakan semua teknik peretasan dan teknik serangan personal komputer terkait lainnya.

g) *Posture Assessment*

Testing yang berjalan dengan tes keamanan untuk mengevaluasi dan meningkatkan kemampuan manajemen risiko di perusahaan.

III. PERANCANGAN

3.1 Analisis Kebutuhan Pengguna

3.1.1 Pengumpulan Kebutuhan Pengguna

Pada tahapan awal dalam melakukan pengembangan sistem *backend* untuk klinik kecantikan mische ini, dilakukan pengumpulan kebutuhan pengguna untuk mengetahui informasi dan permasalahan secara spesifik. Dalam pengumpulan kebutuhan pengguna dilakukan menggunakan metode observasi dan juga wawancara dengan pihak terkait dari Klinik Kecantikan Mische yang menjadikan inisialisasi dalam melakukan pengembangan sistem *backend* ini. Pengembangan sistem *backend* ini menggunakan metode *incremental* yang dimana dilakukan secara bertahap yang dibagi menjadi beberapa modul didalam pengembangannya agar mencapai sebuah kepuasan dan menjawab akan kebutuhan pengguna.

3.1.2 Kebutuhan Sistem

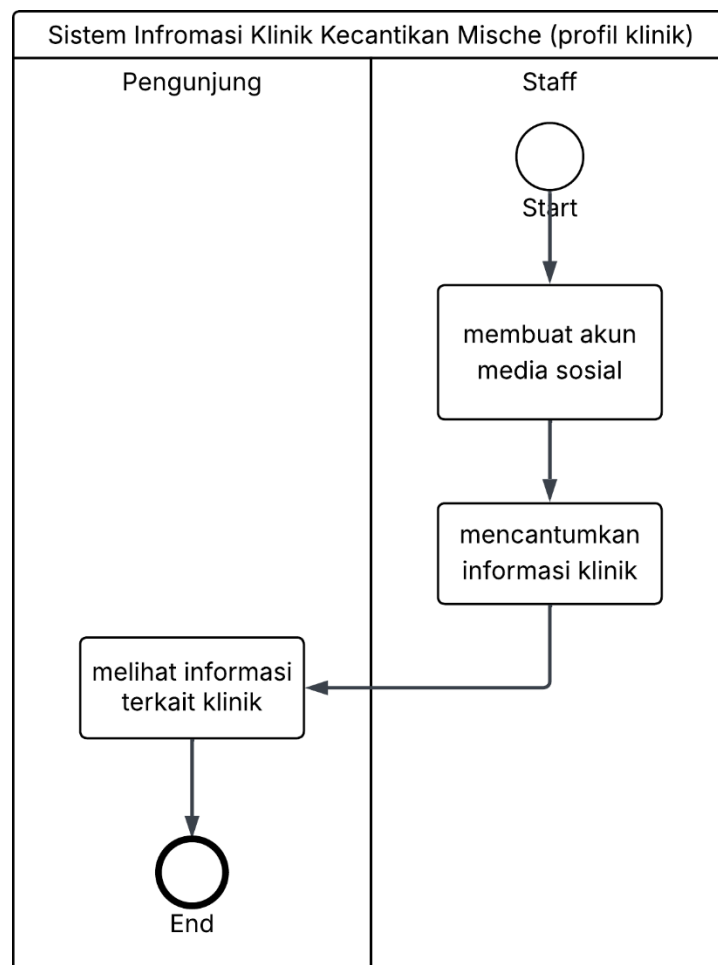
Setelah melakukan wawancara dan observasi dengan salah satu staf dari Klinik Kecantikan Mische, Bapak Fauzi Ramadhan S.Ak., pada tanggal 26 Maret 2025 yang dapat dilihat pada **Lampiran C** mendapatkan beberapa permasalahan yang terjadi pada klinik kecantikan mische, yang dimana di antaranya seperti yang dipaparkan pada tabel 3.1 sebagai berikut ini:

Tabel 3. 1 Kebutuhan Sistem

No	Masalah	Solusi Yang Diterapkan
1	Klinik Kecantikan Mische belum memiliki media resmi yang menampilkan profil perusahaan, sehingga informasi mengenai identitas dan layanan klinik sulit diakses oleh calon pelanggan.	Membuat <i>profile company</i> yang dinamis agar bisa disesuaikan dengan perkembangan klinik.
2	Tidak tersebarnya informasi promo dan <i>event</i> kepada <i>customer</i> dengan baik.	Membuat fitur penyebaran informasi promo dan <i>event</i> dengan nomor <i>customer</i> .
3	Sulit mengatur jadwal <i>reservasi treatment</i> dan masih dilakukan secara manual.	Membuat fitur penjadwalan <i>treatment</i> yang disesuaikan dengan jadwal dokter yang tersedia.
4	Tidak adanya informasi yang jelas dari produk yang dijual baik dari segi stok dan juga harga.	Membuat fitur penjualan produk dari klinik.

3.1.3 Proses Bisnis *As-Is*

Hasil wawancara yang telah dilakukan memberikan gambaran mengenai proses bisnis yang berlangsung di Klinik Kecantikan Mische. Alur proses bisnis profil klinik ditampilkan pada Gambar 3.1. Alur proses bisnis profil klinik dimulai dari penyampaian informasi oleh pihak Klinik Kecantikan Mische melalui media sosial. Dalam proses ini, staf membuat akun resmi klinik, kemudian mencantumkan informasi terkait layanan dan profil klinik. Informasi tersebut dapat diakses oleh pengunjung melalui media sosial sehingga mereka dapat mengenal dan mengetahui layanan yang tersedia di klinik.



Gambar 3. 1 Proses Bisnis *As Is* Profil Klinik

Untuk proses bisnis *as-is reservasi*, penjualan produk, penyebaran informasi *event* dan promo dapat dilihat pada **Lampiran D**.

3.1.4 Increment Plan

Tahapan perancangan sistem pada penelitian ini disusun berdasarkan metode *Incremental* yang dilakukan secara bertahap atau secara *Increment* berdasarkan prioritas dari kebutuhan yang didapatkan pada analisis kebutuhan pengguna. Perancangan ini dilakukan agar sesuai dengan kebutuhan dari pengguna dan memiliki alur pengerjaan yang terstruktur antar *backend* dan *frontend*, maka dari itu dibuatlah *increment plan* seperti pada tabel 3.3.

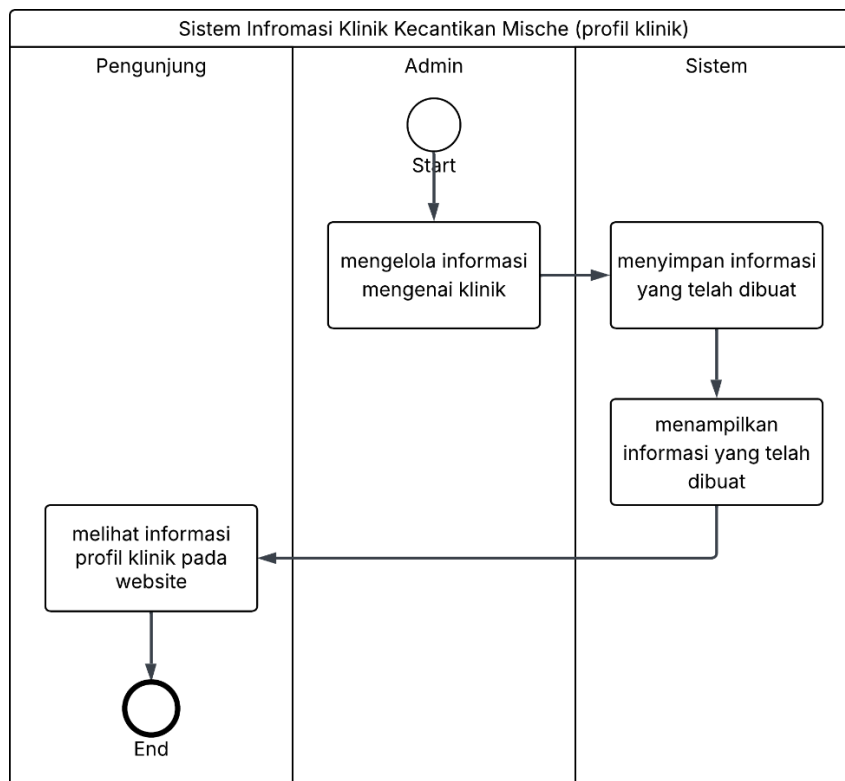
Tabel 3. 2 *Increment Plan*

<i>Increment</i>	Modul yang Dikembangkan	Estimasi Waktu	Alasan Prioritas
1	Modul <i>login admin</i> dan manajemen <i>user</i> , modul profil klinik (visi, misi, deskripsi, kontak, jam operasional), serta tampilan profil klinik di sisi <i>customer</i> .	12 Hari	Fondasi awal sistem dan akses utama
2	Modul registrasi dan <i>login customer</i> serta modul profil <i>customer</i> (kelola/edit data akun).	10 Hari	Diperlukan sebelum layanan reservasi.
3	Modul pengelolaan jadwal reservasi (<i>admin</i> menambah/mengatur jadwal dokter dan <i>treatment</i>).	9 Hari	Jadwal diperlukan sebelum reservasi dilakukan.
4	Modul pemesanan reservasi oleh <i>customer</i> berdasarkan jadwal yang tersedia (hanya setelah <i>login</i>).	11 Hari	Lanjutan dari jadwal, memungkinkan <i>customer</i> melakukan reservasi.
5	Modul kelola informasi dokter oleh <i>admin</i> dan tampilkan profil dokter ke <i>customer</i> .	8 Hari	Melengkapi reservasi dengan informasi dokter.
6	Modul <i>promo</i> dan <i>event</i> (<i>admin</i> mengelola <i>promo/event</i> dan <i>customer</i> dapat melihat informasi tersebut).	9 Hari	Mendukung layanan dengan informasi promosi.
7	Modul distribusi <i>promo/event</i> via <i>WhatsApp</i> serta modul testimoni (<i>admin</i> kelola testimoni, <i>customer</i> dapat melihat testimoni).	10 Hari	Lanjutan dari promo, mempercepat penyebaran informasi.
8	Modul produk dan kategori produk (<i>admin</i> mengelola data produk, <i>customer</i> dapat melihat produk).	9 Hari	Modul penting untuk mendukung transaksi penjualan produk.
9	Modul transaksi penjualan produk yang terintegrasi dengan kode <i>voucher</i> dan diskon otomatis.	14 Hari	Lanjutan dari produk, mendukung transaksi klinik.
10	Modul laporan reservasi dan penjualan (ekspor ke <i>Excel</i>), modul <i>dashboard</i> ringkasan data, serta tahap penyempurnaan sistem.	14 Hari	Tahap akhir untuk pelaporan dan evaluasi sistem.

3.2 Desain

3.2.1 Proses Bisnis To-Be

Setelah dilakukan analisis kebutuhan pengguna, maka dirancang sebuah proses bisnis baru yang dapat mengatasi permasalahan yang terjadi melalui implementasi backend. Untuk menggambarkan dari proses bisnis baru ini dapat dilihat pada gambar 3.2 yang menggambarkan alur proses bisnis profil klinik yang baru. Proses bisnis profil klinik dimulai dari *admin* melakukan pengelolaan informasi klinik seperti tentang klinik, visi, misi, deskripsi klinik dan hal lainnya. Kemudian setelah dilakukan pengelolaan, maka sistem akan secara otomatis akan menyimpan informasi tersebut dan menampilkan di halaman profil klinik yang akan berguna untuk memberikan informasi kepada pengunjung yang ingin tau lebih banyak tentang klinik kecantikan mische.

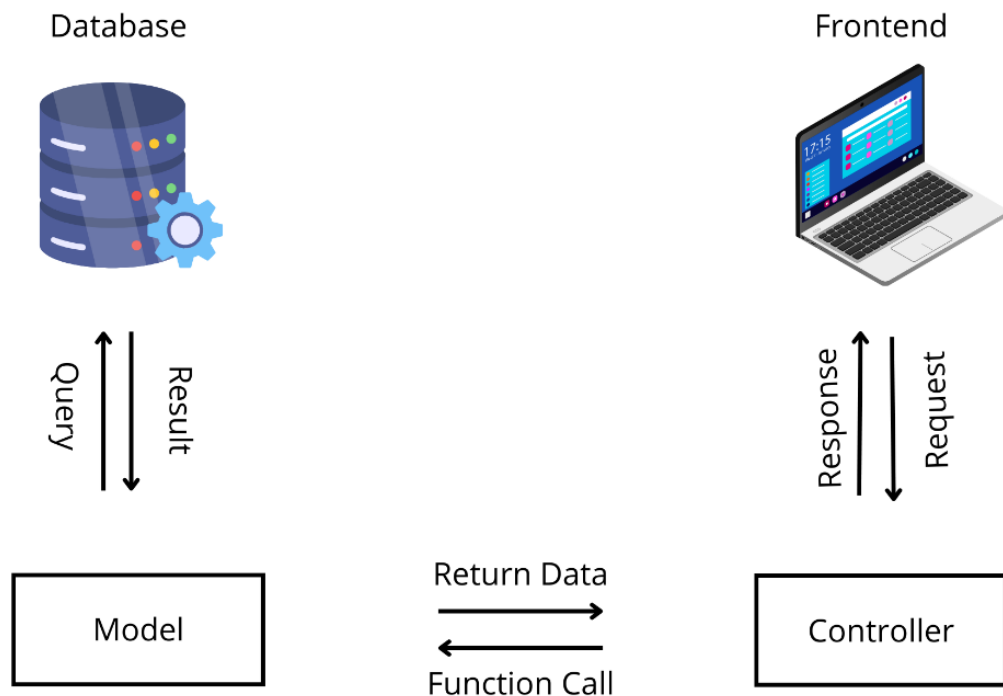


Gambar 3. 2 Proses Bisnis To-Be Profil Klinik

Untuk proses bisnis *to-be reservasi online*, *reservasi offline*, penjualan produk, penyebaran informasi *event* dan promo dapat dilihat pada **Lampiran E**.

3.2.2 Arsitektur Sistem

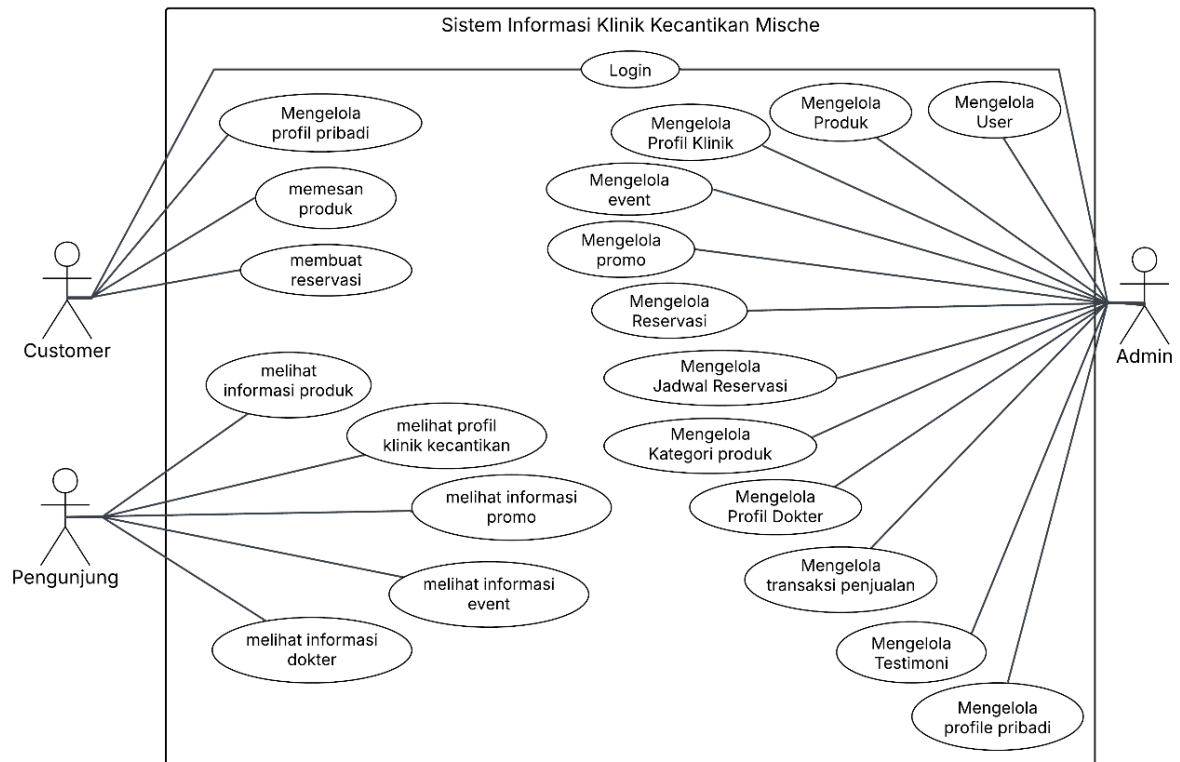
Arsitektur sistem berguna untuk memberikan gambaran dari sebuah alur kerja dari sebuah sistem yang dikembangkan. Pada pengembangan sistem hanya berfokus pada bagian *backend* saja sehingga dapat dilihat arsitektur sistem penelitian ini pada gambar 3.3. Pada gambar tersebut menjelaskan akan alur dari sistem *backend* yang dibuat yang dimulai dari *request* dari *frontend* yang kemudian di arahkan *controller* untuk mengelola logika sistem, kemudian setelah dikelola di *controller* akan diarahkan ke model yang berguna untuk menghubungkan dengan *database*, setelah itu database akan memberikan *response* sesuai dengan *request* yang diberikan hingga nantinya akan sampai *response* ke bagian *frontend* kembali.



Gambar 3.3 Arsitektur Sistem

3.2.3 Use Case Diagram

Dalam melakukan perancangan sebuah sistem dibutuhkan sebuah gambaran aktor dan interaksi antar komponen sistem, maka dari itu dibuatlah *use case* pada penelitian ini seperti yang di paparkan pada gambar 3.4.



Gambar 3.4 Use Case Diagram

Berdasarkan gambar 3.4 terdapat 3 aktor yang berperan didalam sistem informasi klinik kecantikan mische yaitu *Admin*, *Customer*, dan *Pengunjung*. Berikut ini adalah penjelasan lebih mengenai *use case diagram* tersebut:

a) *Customer*

- i. *Customer* dapat melakukan login terlebih dahulu sebelum melakukan pemesanan produk dan *reservasi treatment*.
- ii. *Customer* juga dapat melakukan pemesanan produk serta dapat menggunakan *voucher* apabila memiliki dan dilanjutkan dengan pembayaran terhadap produk nya.
- iii. *Customer* dapat melakukan *reservasi* sesuai dengan jadwal yang sudah ditetapkan oleh pihak klinik pada sistem.
- iv. *Customer* dapat mengelola profil pribadi yang dan diharuskan melakukan login terlebih dahulu.

b) *Admin*

- i. *Admin* dapat melakukan *login* sebelum mengakses halaman *admin*.
- ii. *Admin* dapat mengelola *user* yang dapat *login* baik itu *customer* dan juga para *admin* yang lain.

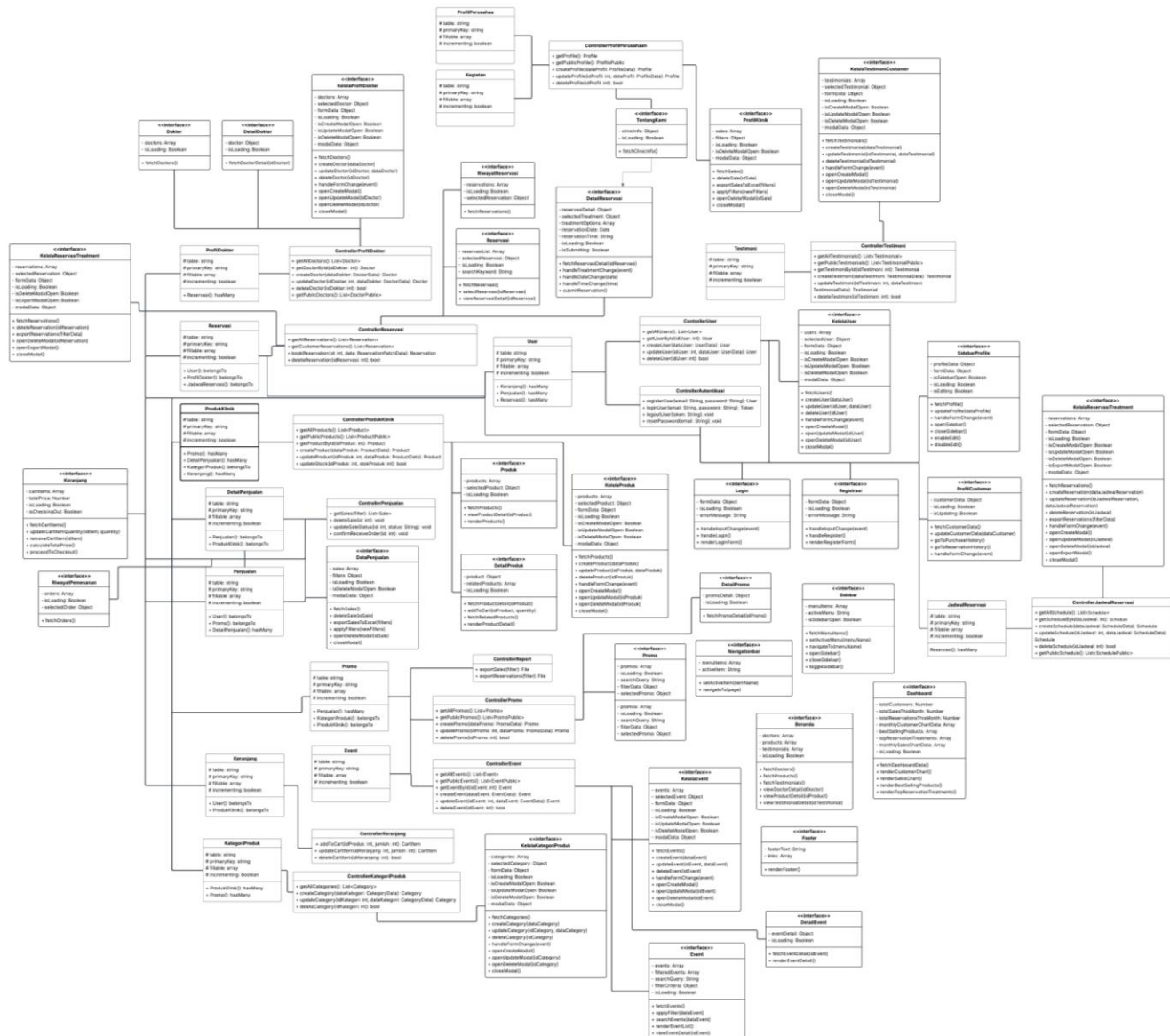
- iii. *Admin* dapat melakukan pengelolaan terhadap produk yang akan dijual pada sistem.
- iv. *Admin* juga dapat melakukan pengelolaan terhadap profil klinik yang nantinya dapat dilihat oleh customer.
- v. *Admin* dapat melakukan pengelolaan informasi *event* yang akan diselenggarakan oleh klinik dan di bantu dengan menyebarkan nya melalui *WhatsApp*.
- vi. *Admin* juga melakukan pengelolaan terhadap promo yang ada pada klinik dan di bantu dengan menyebarkan nya melalui *WhatsApp*.
- vii. *Admin* melakukan pengelolaan terhadap *reservasi customer* dan dapat mengexport laporan *reservasi* kedalam bentuk *excel*.
- viii. *Admin* melakukan pengelolaan terhadap jadwal *reservasi*.
- ix. *Admin* dapat melakukan pengelolaan terhadap profil dokter yang bekerja pada klinik.
- x. *Admin* dapat mengelola kategori produk yang akan membagi produk menjadi kelompok-kelompoknya.
- xi. *Admin* dapat mengelola transaksi penjualan dan dapat dapat mengexport laporan *reservasi* kedalam bentuk *excel*.
- xii. *Admin* dapat mengelola testimoni dari *customer* yang pernah membeli produk ataupun melakukan treatment pada klinik.
- xiii. *Admin* dapat mengelola profil pribadi sesuai dengan data diri dari *admin*.

c) Pengunjung

- i. Pengunjung dapat melihat informasi promo yang ditawarkan oleh klinik pada halaman promo di sistem.
- ii. Pengunjung dapat melihat informasi *event* yang dibuat oleh pihak klinik kemudian akan di tampilkan pada halaman *event*.
- iii. Pengunjung dapat melihat informasi dokter yang berada pada klinik dan memberikan informasi lebih dari dokter tersebut.
- iv. Pengunjung dapat melihat informasi produk yang ditawarkan oleh pihak klinik.
- v. Pengunjung dapat melihat informasi Perusahaan seperti visi, misi, kontak dan alamat perusahaan kemudian juga galeri dari klinik.

3.2.4 Class Diagram

Tahapan perancangan *class diagram* akan berguna untuk menggambarkan kelas-kelas yang terlibat beserta atribut dan hubungan antar kelas. Adapun perancangan *class diagram* untuk penelitian ini dapat dilihat pada gambar 3.5.



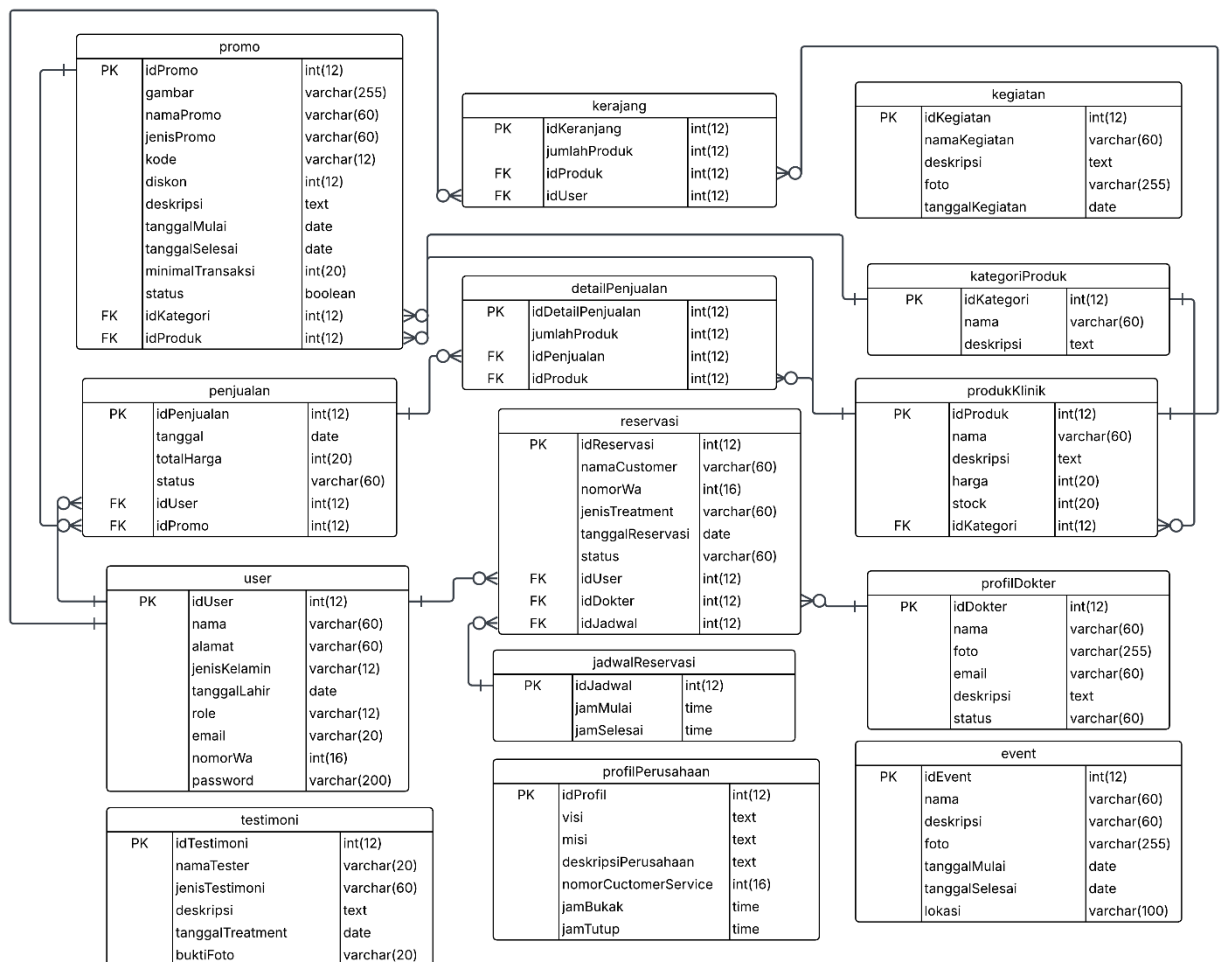
Gambar 3. 5 Class Diagram

Setelah dilakukan perancangan *class diagram*, diperoleh 14 *class model*, 14 *class controller*, dan 33 *class interface*. Penelitian ini difokuskan pada pengembangan *class model* dan *controller* yang berfungsi untuk mengelola data serta menghubungkan sistem dengan *database*. Tahap selanjutnya adalah perancangan *ERD*, struktur tabel, dan perancangan *endpoint API*.

3.2.5 Perancangan Database

a. Perancangan Entity Relation Diagram (ERD)

Dalam perancangan *ERD* ini menggunakan konsep model *ERD* fisik yang dimana untuk menjelaskan hubungan antar data dalam *database*. *ERD* fisik yang dirancang ini akan menampilkan *entitas*, *relasi*, kolom, tipe data, *primary key*, dan *foreign key*. Sehingga di harapkan *ERD* ini memastikan sesuai dengan dengan kebutuhan sistem. Dan untuk perancangan *ERD* pada sistem ini dapat dilihat pada gambar 3.6.



Gambar 3. 6 Rancangan *ERD*

b. Perancangan Struktur Tabel

Setelah melakukan perancangan terhadap *ERD* pada gambar 3.6 terdapat 13 tabel didalamnya yaitu tabel *user*, *keranjang*, *produkKlinik*, *kategoriProduk*, *penjualan*, *detailPenjualan*, *reservasi*, *jadwalReservasi*, *profileDokter*, *keranjang*, *testimoni*, *profilPerusahaan*, dan *event*. Kemudian untuk struktur tabel dari perancangan *ERD* pada tabel 3.4.

Tabel 3.3 Struktur Tabel *user*

No	Kolom	Type Data	Keterangan	Null
1	<i>idUser</i>	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	<i>nama</i>	<i>Varchar(60)</i>	Nama <i>user</i>	<i>Not null</i>
3	<i>alamat</i>	<i>Varchar(60)</i>	Alamat <i>user</i>	<i>Can null</i>
4	<i>jenisKelamin</i>	<i>Varchar(12)</i>	Jenis kelamin <i>user</i>	<i>Can null</i>
5	<i>tanggalLahir</i>	<i>Date</i>	Tanggal lahir <i>user</i>	<i>Can null</i>
6	<i>role</i>	<i>Varchar(12)</i>	Peran dari <i>user</i>	<i>Not null</i>
7	<i>email</i>	<i>Varchar(20)</i>	Alamat <i>email user</i>	<i>Not null</i>
8	<i>nomorWa</i>	<i>Int(16)</i>	Nomor wa dari <i>user</i>	<i>Not null</i>
9	<i>password</i>	<i>Varchar(200)</i>	Password <i>user</i> untuk login	<i>Not null</i>

Penjelasan lebih rinci mengenai struktur tabel untuk penjualan, produkKlinik, kategoriProduk, detailPenjualan, detailPenjualan, *reservasi*, jadwalReservasi, profileDokter, keranjang, testimoni, profilPerusahaan, jadwal dan *event* dapat dilihat pada **Lampiran F**.

3.2.6 Perancangan *API*

Pada perancangan sistem *backend* didalamnya memerlukan sebuah *API* untuk menghubungkan antara *backend* dengan *frontend*. Oleh karena itu, pada bagian ini akan dilakukan perancang *API* dengan merincikan *endpoint* yang dibutuhkan oleh sistem. Rincian perancangan *API* tersebut dapat dilihat pada Tabel 3.5.

Tabel 3.4 Perancangan *API*

No	Endpoint	Metode	Deskripsi
1.	/api/auth/register	POST	Mendaftarkan pengguna
2.	/api/auth/login	POST	Masuk kedalam sistem
3.	/api/auth/logout	POST	Keluar dari sistem
4.	/api/auth/reset-password	POST	Mengatur ulang <i>password</i>
5.	/api/admin/users	GET	Mengambil daftar <i>user</i> oleh <i>admin</i>
6.	/api/admin/users/{idUser}	GET	Mengambil data <i>user</i> berdasarkan id
7.	/api/admin/users	POST	Menambahkan <i>user</i> baru
8.	/api/admin/users/{idUser}	PUT	Memperbarui data <i>user</i> berdasarkan id
9.	/api/admin/users/{idUser}	DELETE	Menghapus data <i>user</i>

Penjelasan lebih rinci mengenai perancangan *API* untuk sistem klinik kecantikan Mische dapat dilihat pada **Lampiran G**.

3.3 Testing

Tahapan testing merupakan tahapan untuk melakukan validasi terhadap sistem yang sudah dikembangkan sehingga dibutuhkan sebuah *tools* yang membantu dalam melakukan testing, pada penelitian ini dilakukan beberapa testing yang akan dilakukan sebagai berikut ini:

3.3.1 Rancangan *Unit Testing*

Pada tahapan ini dilakukan perancangan pengujian dengan teknik *unit testing*. Pada tahapan pengujian *unit testing* menggunakan *framework unit testing* bawaan *laravel*, yaitu *PHP Unit*. Salah satu rancangan pengujian dengan unit testing dapat dilihat pada tabel 3.6.

Tabel 3. 5 Rancangan *Unit Testing* Authentikasi

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	registerUser(\$email,\$password)	Register user baru dengan data valid	Email dan password valid	Akun berhasil dibuat dengan password terhash
2	registerUser(\$email,\$password)	Register user dengan email sudah terdaftar	Email yang sama sudah ada	Error: "Email sudah digunakan"
3	loginUser(\$email,\$password)	Login dengan kredensial benar	Email dan password benar	Login sukses, token/session diberikan
4	loginUser(\$email,\$password)	Login dengan password salah	Email benar, password salah	Error: "Password salah"
5	logoutUser(\$token)	Logout dengan token valid	Token valid	Logout berhasil
6	resetPassword(\$email)	Reset password user yang valid	Email valid	Password direset, pesan "Reset berhasil"

Penjelasan lebih rinci mengenai rancangan *unit testing* untuk bagian user, dokter, profil klinik, testimoni, *event*, *reservasi*, jadwal, kategori, produk, keranjang, kegiatan, promo, penjualan, laporan dan profil dapat dilihat pada **Lampiran H**.

3.3.2 Rancangan *Integration Testing*

Pada tahapan ini dilakukan perancangan pengujian dengan teknik *integration testing*. Pengujian dilakukan menggunakan Postman untuk mengirim *request* dan memverifikasi *response*. Salah satu rancangan pengujian dengan *Integration Testing* dapat dilihat pada Tabel 3.7.

Tabel 3. 6 Rancangan *Integration Testing* Authentikasi

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/auth/register	POST	Pengguna berhasil terdaftar dan mendapatkan token autentikasi	True/False
2	/api/auth/login	POST	Pengguna berhasil masuk dan menerima token autentikasi	True/False
3	/api/auth/logout	POST	Pengguna berhasil keluar dari sistem	True/False
4	/api/auth/reset-password	POST	Password berhasil direset dan pengguna bisa login dengan password baru	True/False

Penjelasan lebih rinci mengenai rancangan *integration testing* untuk bagian user, dokter, profil klinik, testimoni, *event*, *reservasi*, jadwal, kategori, produk, keranjang, kegiatan, promo, penjualan, laporan dan profil dapat dilihat pada **Lampiran I**.

3.3.3 Rancangan *Performance Testing*

Pada tahapan ini dilakukan perancangan pengujian dengan teknik *performance testing*. Pengujian dilakukan menggunakan *JMeter* untuk mengirim request dalam jumlah tertentu (simulasi beban) dan memverifikasi waktu respon serta kestabilan sistem. Salah satu rancangan pengujian dengan *Performance Testing* dapat dilihat pada tabel 3.8.

Tabel 3. 7 Rancangan *Performance Testing* Authentikasi

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/auth/register	POST	100 user registrasi bersamaan	100	≤ 2.0
2	/api/auth/login	POST	100 user login bersamaan	100	≤ 2.0
3	/api/auth/logout	POST	100 user logout bersamaan	100	≤ 1.5
4	/api/auth/reset-password	POST	50 user reset password bersamaan	50	≤ 2.5

Penjelasan lebih rinci mengenai rancangan *performance testing* untuk bagian user, dokter, profil klinik, testimoni, *event*, *reservasi*, jadwal, kategori, produk, keranjang, kegiatan, promo, penjualan, laporan dan profil dapat dilihat pada **Lampiran J**.

3.3.4 Rancangan *Security Testing*

Pada tahapan ini dilakukan perancangan pengujian dengan teknik *security testing*. Pengujian dilakukan secara manual menggunakan *Postman* dengan fokus pada pengujian autentikasi, otorisasi, dan validasi *input*. Tujuannya adalah memastikan bahwa sistem tidak rentan terhadap akses tidak sah dan dapat menangani *input* yang tidak valid dengan benar. Salah satu rancangan pengujian dengan *security testing* dapat dilihat pada tabel 3.9.

Tabel 3. 8 Rancangan *Security Testing* Autentikasi

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/auth/register (POST)	SQL Injection di field email	email=" OR 1=1 --"	400 Bad Request, validasi input
2	/api/auth/register (POST)	XSS di field nama	nama="<script>alert(1)</script>"	400 Bad Request
3	/api/auth/login (POST)	Brute force login (5x salah cepat)	password salah	Sistem membatasi / delay
4	/api/auth/logout (POST)	Akses tanpa token	-	401 Unauthorized
5	/api/auth/reset-password (POST)	Reset dengan email tidak terdaftar	email=fake@mail.com	404 Not Found

Penjelasan lebih rinci mengenai rancangan *security testing* untuk bagian user, dokter, profil klinik, testimoni, *event*, *reservasi*, kategori, produk, keranjang, kegiatan, promo, penjualan, laporan dan profil dapat dilihat pada **Lampiran K**.

Penelitian ini akan dijalankan dengan menggunakan metode *incremental* dengan jumlah *increment*-nya sebanyak 10 tahapan yang telah dirancang pada bagian *project plan*. Setiap *increment* akan dilakukan pengujian terlebih dahulu sebelum berlanjut ke *increment* selanjutnya, dimulai dari *unit testing*, kemudian dilanjutkan dengan *integration testing*, *performance testing*, dan terakhir *security testing*. Proses pengujian ini dilakukan untuk memastikan bahwa setiap fungsi yang dikembangkan bekerja sesuai harapan serta stabil dalam berbagai kondisi. Apabila hasil pengujian sesuai dengan kriteria yang telah ditetapkan dan seluruh kebutuhan yang terdapat dalam *user stories* pada *project plan* telah terpenuhi, maka pengembangan akan dilanjutkan ke *increment* berikutnya. Tahapan ini akan terus dilakukan secara konsisten hingga seluruh 10 *increment* yang telah dirancang dapat terselesaikan dengan baik, sehingga menghasilkan sistem *backend* yang kebutuhan pengguna.

IV. JADWAL DAN ANGGARAN BIAYA

4.1 Jadwal

Pada jadwal pelaksanaan ini, disusunlah rencana pengerjaan tugas akhir untuk memastikan penggunaan waktu yang lebih baik dalam menyelesaikan proyek akhir, sesuai dengan metodologi yang diterapkan. Jadwal pengerjaan tersebut dapat dilihat pada tabel 4.1 di bawah ini:

Tabel 4. 1 Jadwal Pengerjaan Proyek Akhir

No	Tahapan	2025									
		Mar	Apr	May	Jun	Jul	Aug	Sep	Okt	Nov	Des
1	Wawancara dan pengumpulan data										
2	Bimbingan										
3	Pembuatan proposal										
4	Seminar proposal										
5	Pembuatan sistem										
6	Pengujian dan analisa										
7	Penulisan laporan										
8	Sidang proyek akhir										

4.2 Anggaran Biaya

Berikut adalah daftar peralatan dan biaya yang diperlukan untuk mendukung kelancaran pengerjaan proyek akhir:

Tabel 4. 2 Perkiraan Biaya

No	Nama Barang	Satuan	Jumlah	Harga
1	Laptop	Unit	1	Rp 10.0000.000
2	Wifi	Paket	1	Rp 200.000
3	Hosting	Unit/tahun	1	Rp 1.000.000
4	Domain	Unit/tahun	1	Rp 280.000
5	Backend Developer	Orang/proyek	1	Rp 10.000.000
6	Kertas HVS	Rim	4	Rp 260.000
7	Transportasi Observasi dan wawancara	Trip	3	Rp 500.000
Harga Total				Rp 22.240.000

V. DAFTAR PUSTAKA

- Abraham, J., Ismail, I. E., Kom, S., Kom, M., Informatika, P. S., Teknik, J. & Jakarta, P. N. (2021). Unit Testing dan User Acceptance Testing pada Sistem Informasi Pelayan Kategorial Pelayanan Anak. *Repos. PNJ*.
- Adelia, N. W. P. (2023). *Perancangan UI/UX Website Laksmi Muslimah Menggunakan Metode Design Thinking untuk Memudahkan Transaksi Klien Memilih Paket Pernikahan*.
- Agustin, F. (2025). Sistem Informasi Pengingat Treatment Pada Pasien Di Klinik Kecantikan Ramdani Skincare Berbasis SMS Gateway. *Universitas Islam Raden Rahmat*.
- Akbar, I. S. & Haryanti, T. (2023). Pengembangan Entity Relationship Diagram Database Toko Online Ira Surabaya. *Computing Insight : Journal of Computer Science*, 3(2), 28–35. https://doi.org/10.30651/comp_insight.v3i2.12002
- Alief, N. (2024). Perancangan Dan Analisa Timeline: Pengelompokkan Aktivitas Pengembangan Sistem Informasi Inventori Menggunakan Metode Pearson Dan K-Means. *UNIVERSITAS MUHAMMADIYAH GRESIK*.
- Amrina, A. F. N., Santoso, N. & Widodo, A. W. (2021). Pengembangan Sistem Informasi Manajemen Klinik Kecantikan Dan Umum Berbasis Web (Studi Kasus: Praktik Mandiri Dokter Lina Malang). *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, 5(6), 2495–2503.
- Andini, N., Taufiq, R., Priyanggodo, D. Y. & Sugiyani, Y. (2023). Penggunaan Metode Prototype Pada Pengembangan Sistem Informasi Imunisasi Posyandu. *JIKA (Jurnal Informatika)*, 7(4), 431–439.
- Apandi, A. & Istini, S. I. M. (2023). Pembuatan Website Penjualan Toko Baju Biazra-Store Menggunakan PHP dan MySQL. *Jurnal Teknik dan Science*, 2(3), 80–91. <https://doi.org/10.56127/jts.v2i3.998>
- Aprila, M. A. & Victor, R. (2025). Penggunaan Black-Box Testing dalam Integration Testing Sistem Pembayaran PaDi UMKM. *Jurnal STRATEGI-Jurnal Maranatha*, 7(1), 86–93.

- Bachtiar, D. H., Paniran, P. & Suksmadana, I. M. B. (2024). Perancangan Back-end Api pada Aplikasi Mobile Fruityfit Menggunakan Framework Express JS. *Mars*, 2(3), 107–117.
- Badri, F. & Sari, S. U. R. (2023). Sistem Informasi Manajemen Penjualan Obat Pada Apotek Berbasis Web Dengan Menggunakan Metode Incremental Model. *Jurnal Sistem Informasi Dan Informatika*, 1(2), 70–79.
- BPMPP UMA. (2022, 24. Maret). *Security Testing: Tipe serta Metodologi yang digunakan*. <https://bpmpp.uma.ac.id/2022/03/24/security-testing-tipe-serta-metodologi-yang-digunakan/>
- Daud Pujas, M. S., Amar, Muh. I., Arif, M. E. & Mustamin, M. M. (2024). Pengukuran Kinerja Database SQL dan NoSQL Pada Aplikasi E-Commerce. *Jurnal Fokus Elektroda : Energi Listrik, Telekomunikasi, Komputer, Elektronika dan Kendali*, 9(1), 12–15. <https://doi.org/10.33772/jfe.v9i1.165>
- Fahzirah, I. & Nasution, M. I. P. (2024). Pengenalan Sistem Database: Konsep Dasar Dan Manfaatnya Dalam Perusahaan. *JURNAL ILMIAH NUSANTARA*, 1(4), 673–678.
- Fanani, D. Y., Febrianto, M. F., Firmansyah, R. & Nugroho, H. (2023). Pembuatan Company Profile Perusahaan P3I Jatim (Persatuan Perusahaan Periklanan Indonesia) di Surabaya. *Prosiding Seminar Implementasi Teknologi Informasi dan Komunikasi*, 2(2), 243–248.
- Filiana, A., Rini, M. N. A., Prabawati, A. G. & Samat, R. A. (2022). Pengembangan rest API untuk informasi pasar tradisional di Kota Yogyakarta dengan metode incremental. *SINTECH (Science and Information Technology) Journal*, 5(1), 10–23.
- Habib, A. & Kartika, A. W. H. (2020). Development of an Online Sales Information System for SMEs Using Incremental Methods. *INTENSIF: Jurnal Ilmiah Penelitian dan Penerapan Teknologi Sistem Informasi*, 4(1), 51–62.
- Hasibuan, A. N. & Dirgahayu, T. (2021). Pengujian dengan Unit Testing dan Test case pada Proyek Pengembangan Modul Manajemen Pengguna. *AUTOMATA*, 2(1).
- Herdiputra, M. A., Pramiyati, T. & Purabaya, R. H. (2021). Aplikasi Penyebaran Informasi FIKNET (FIK News Update) Berbasis Android. *Prosiding Seminar Nasional Mahasiswa Bidang Ilmu Komputer dan Aplikasinya*, 2(1), 274–285.

- Herdiyatomoko, H. F. (2022). Back-End System Design Based On Rest API. *Jurnal Tekinkom (Teknik Informasi dan Komputer)*, 5(1), 123–129.
- Hermiati, R., Asnawati, A. & Kanedi, I. (2021). Pembuatan E-Commerce Pada Raja Komputer Menggunakan Bahasa Pemrograman Php Dan Database Mysql. *jurnal media infotama*, 17(1).
- Kansha, W. M. (2023). Analisis Perbandingan Framework Codeigniter Dan Laravel Dalam Pengembangan Web Application. *Jurnal Teknik Informatika*, 9(1), 27–33.
- Maulana, A., Riady, N., Paniran, P. & Suksmadana, I. M. B. (2024). *Perancangan Backend Api Berbasis Rest-API pada Aplikasi Rekomendasi Resep Makanan (Rest-API Based Backend Api Design In Food Recipe Recommendation Application)*. vol.
- Melyani, R. I., Rosita, R. & Aji, S. (2023). Pengembangan sistem informasi penggajian berbasis web menggunakan framework laravel dengan metode agile software development. *Jurnal Sistem Informasi Akuntansi (JASIKA)*, 3(1), 31–36.
- Mohidin, I. K. H. (2022). Penerapan Teknologi Rest Api pada Aplikasi Perpustakaan Digital Politeknik Gorontalo. *Jurnal Technopreneur (JTech)*, 10(1), 34–39.
- Mulana, L., Prihandani, K. & Rizal, A. (2022a). Analisis Perbandingan Kinerja Framework Codeigniter Dengan Express. Js Pada Server RESTful Api. *Jurnal Ilmiah Wahana Pendidikan*, 8(16), 316–326.
- Mulana, L., Prihandani, K. & Rizal, A. (2022b). Analisis Perbandingan Kinerja Framework Codeigniter Dengan Express. Js Pada Server RESTful Api. *Jurnal Ilmiah Wahana Pendidikan*, 8(16), 316–326.
- Murdani, M. H., Widhiyanta, N., Priyambudi, S. & Asrori, M. (2022). Rancang Bangun Sistem Informasi Akuntansi menggunakan Metode Incremental–Studi Kasus di Koperasi Karyawan Coca Cola SIER. *SMATIKA JURNAL: STIKI Informatika Jurnal*, 12(01), 67–74.
- Musliyana, Z. & Helinda, A. (2022). Analisis Performansi Query Mysql Menggunakan Query Builder Pada Framework Codeigniter 4. *Journal of Informatics and Computer Science*, 8(1), 36–40.
- Noviana, R. (2022). Pembuatan Aplikasi Penjualan Berbasis Web Monja Store Menggunakan PHP dan MySQL. *Jurnal Teknik dan Science*, 1(2), 112–124. <https://doi.org/10.56127/jts.v1i2.128>

- Nurjaman, I., Utomo, F. S. & Hermanto, N. (2024). Penerapan REST API Laravel sebagai Fondasi Back-end Aplikasi G-MOOC 4D. *Journal of Informatics and Interactive Technology*, 1(1), 9–18.
- Oktaviyana, A. (2023). Analisis Dan Pengembangan Sistem Informasi Manajemen. *Circle Archive*, 1(1).
- Prasetya, A. B. & Suharjo, I. (2025). Backend Api Data Protection Menggunakan Jwt Token Dan Algoritma Aes 256-Bit Dengan Bahasa Pemrograman Golang. *Jurnal Informatika dan Teknik Elektro Terapan*, 13(1).
<https://doi.org/10.23960/jitet.v13i1.5699>
- Raharja, M. P. & Wijayanto, H. (2023). Pengembangan Prototype Back-End Website Pelaporan Barang Hilang di PT. Presentologics Dicoding Academy Indonesia. *Jurnal Begawe Teknologi Informasi (JBegaTI)*, 4(2).
<https://doi.org/10.29303/jbegati.v4i2.1078>
- Rahmawati, L. & Sumarsono, S. (2024). Desain Pengembangan Website dengan Arsitektur Model View Controller pada Framework Laravel. *Jurnal Teknologi Dan Sistem Informasi Bisnis*, 6(4), 785–790.
- Rizkyana, M. A., Yunanto, Y., Yoga, Y. & Widiyanto, S. R. (2021). Implementasi Unit Testing Menggunakan Metode Test-First Development. *MULTINETICS*, 7(1), 37–47.
- Salam, I. A., Prihandani, K. & Purnamasari, I. (2023). Rancang Bangun Aplikasi Profit Penjualan Motor Berbasis Desktop Konsep Arsitektur Model View Controller (Mvc). *Jurnal Informatika Dan Teknik Elektro Terapan*, 11(3s1).
- Setiawan, R. & Risal, L. (2024). Pengembangan Unit Testing dan Integration Testing REST API Pengelola Data Bootcamp PT Mitra Integrasi Informatika. *Jurnal STRATEGI-Jurnal Maranatha*, 6(2), 357–368.
- Shofyan & Shita. (2024). Implementasi Web Service Restful API dengan Autentikasi Personal Access Tokens dan Algoritma AES 256. *Jurnal Ticom: Technology of Information and Communication*, 12(3), 108–114.
<https://doi.org/10.70309/ticom.v12i3.130>
- Sianturi, R. A. (2021). Penerapan user experience design pada pengembangan aplikasi mobile Markopi. *Jurnal Teknologi Informasi dan Ilmu Komputer*, 8(4), 703–712.

- Sianturi, R. A., Sinaga, A. M., Pratama, Y., Simatupang, H., Panjaitan, J. & Sihotang, S. (2021). Perancangan Pengujian Fungsional dan Non Fungsional Aplikasi SIAPPARA di Kabupaten Humbang Hasundutan. *Jurnal Komputer dan Informatika*, 9(2), 133–141. <https://doi.org/10.35508/jicon.v9i2.4706>
- Silalahi, F. D. (2022). Manajemen Database MySQL (Structured Query Language). *Penerbit Yayasan Prima Agus Teknik*, 1–158.
- Siregar, L. (2021). Review Pengujian Keamanan Perangkat Lunak dalam Software Development Life Cycle (SDLC). *Jurnal ASEECT*, 1(3), 1–11.
- Sitanggang, R., Dachi, T. U. & Manurung, I. H. G. (2022). Rancang Bangun Sistem Penjualan Tanaman Hias Berbasis Web Menggunakan PHP dan MySql. *Jurnal Teknologi Kesehatan Dan Ilmu Sosial (TEKESNOS)*, 4(1), 84–90.
- Sitepu, V. J., Tarigan, S. N., Panjaitan, C., Hutapea, F., Sinaga, N. H., Zendrato, D. C. & Lahagu, M. (2023). Perancangan Aplikasi Layanan Bimbingan Konseling Berbasis Website Menggunakan Metode Incremental. *Seminar Nasional Inovasi Sains Teknologi Informasi Komputer*, 1–6.
- Sutjiadi, R., Wirapraja, A., Trianto, E. M., Rahmawati, T., Basatha, R., Adiwena, B. & Krisopras, A. (2022). Perancangan Sistem Informasi Manajemen Tugas Akhir Pada Institut Informatika Indonesia Menggunakan Metode Incremental. *Jurnal Ilmiah Telsinas Elektro, Sipil dan Teknik Informasi*, 5(2), 152–164.
- Togatorop, P. R., Simanjuntak, R. P., Manurung, S. B. & Silalahi, M. C. (2021). Pembangkit Entity Relationship Diagram dari Spesifikasi Kebutuhan Menggunakan Natural Language Processing untuk Bahasa Indonesia. *Jurnal Komputer dan Informatika*, 9(2), 196–206. <https://doi.org/10.35508/jicon.v9i2.5051>
- Zulkifli, A. & Rahmalisa, U. (2022). Implementasi Framework Laravel pada Aplikasi Klinik Kecantikan Qatrunnada SkinCare. *Jurnal Sistem Informasi dan E-Bisnis*, 4(1), 49–59.

LAMPIRAN A – SURAT PERNYATAAN

SURAT PERNYATAAN KERJA SAMA PENGEMBANGAN SISTEM INFORMASI KLINIK KECANTIKAN MISCHE BERBASIS WEBSITE

Kami yang bertanda tangan di bawah ini:

Nama	: Fauzi Ramadhan, S.Ak.
Jabatan	: Supervisor
Nama Instansi	: Klinik Kecantikan Mische
Alamat	: JL. KH. Ahmad Dahlan No.80, Sukajadi - Pekanbaru

Dengan ini, Klinik Kecantikan Mische menyatakan bersedia bekerja sama dengan mahasiswa Politeknik Caltex Riau, yaitu Muhammad As Shaff dan Bintang Puspita Dewi, dalam pengembangan Sistem Informasi Klinik Kecantikan Mische Berbasis Website. Sistem ini bertujuan untuk meningkatkan efisiensi operasional klinik serta membantu menyelesaikan permasalahan yang ada di Klinik Kecantikan Mische.

Kami juga memberikan izin kepada Muhammad As Shaff dan Bintang Puspita Dewi untuk menggunakan proyek ini sebagai judul Proyek Akhir (PA) pada Program Studi Sistem Informasi, Politeknik Caltex Riau.

Demikian surat pernyataan ini dibuat dengan sebenar-benarnya untuk dapat digunakan sebagaimana mestinya.

Pekanbaru, 15 Februari 2025

**Pihak Klinik
Kecantikan Mische**


Fauzi Ramadhan, S.Ak.
Supervisor

LAMPIRAN B – FOTO WAWANCARA



LAMPIRAN C – HASIL WAWANCARA

TRANSKRIP HASIL WAWANCARA KLINIK KECANTIKAN MISCHE

Hari/Tanggal : Rabu, 26 Maret 2025
Tempat : Klinik Kecantikan Mische
Pewawancara : Bintang Puspita Dewi dan Muhammad As Shaff
Narasumber : Bapak Fauzi Ramadhan, S.Ak.

HASIL WAWANCARA

1. Target pasar dari klinik mische ini siapa saja?

Jawaban:

Untuk target pasar di klinik mische ini itu semua gender mulai dari usia 17 tahun hingga 40 tahun lebih untuk target pasarnya itu sendiri orang yang datang dengan permasalahan kulit seperti acne, flek dan juga bopeng dan juga untuk pria dan wanita kemudian yang paling banyak cewe dan permasalahan di acne.

2. Sejauh ini, bagaimana cara klinik menjangkau atau menarik pelanggan baru?

Jawaban:

Sejauh ini cara yang dilakukan untuk menjangkau atau menarik pelanggan itu dengan menggunakan media sosial yang berupa instagram, facebook, dan juga tiktok yang didukung dengan rencana merekrut tim kreatif untuk mengelola konten.

3. Bagaimana biasanya pelanggan mengetahui informasi tentang layanan atau produk terbaru dari klinik?

Jawaban:

Biasanya pelanggan mengetahui informasi klinik itu melalui facebook ads (disambungkan dengan nomor telepon) kemudian saat pelanggan tertarik mereka akan mencari tahu lebih lanjut melalui instagram dan setelah itu pelanggan menghubungi nomor customer service yang tertera pada bio instagram klinik.

4. Dalam setahun terakhir, apakah klinik rutin membuat program promosi atau penawaran spesial?

Jawaban:

Dalam setahun terakhir ini klinik ada melakukan program promosi dengan cara mensupport acara atau event yang dilakukan di universitas islam riau dengan memfasilitasi tempat untuk mempromosikan klinik dan juga memberikan edukasi tentang kesehatan kulit selain itu event yang dilakukan bukan hanya di universitas saja tetapi juga dilakukan di sebuah perusahaan yang telah memberikan izin kerja sama untuk memberikan edukasi dan juga wawasan.

5. Saat ada promo, biasanya disampaikan lewat media apa saja ke pelanggan?

Jawaban:

Promo yang tersedia biasanya disampaikan melalui platform instagram, dan juga whatsapp.

6. Apa tantangan yang dihadapi saat menyebarkan promo?

Jawaban:

Tantangan yang biasanya dihadapi itu ketika promo sudah tersampaikan ke pelanggan lalu pelanggan sekedar bertanya tapi tidak melanjutkan untuk mengambil promo tersebut.

7. Klinik pernah mengadakan event khusus atau campaign? (misalnya beauty class, seminar, soft launching produk baru)

Jawaban:

klinik mische pernah melakukan event atau campaign yang dimana event yang dilakukan ini cukup aktif mulai dari event yang dilakukan secara mandiri maupun event yang dilakukan dengan kerja sama melalui perusahaan maupun universitas

8. Bagaimana proses pemberitahuan ke pelanggan kalau ada event seperti itu?

Jawaban:

Event yang akan diselenggarakan biasanya diberitahu melalui instastory instagram dan juga whatsapp serta dibantu promosi oleh pihak terkait seperti event yang dilakukan di universitas islam riauu yang dipromosikan melalui media sosial dari pihak terkait, namun dengan cara tersebut masih belum banyak juga yang tau tentang event yang akan diselenggarakan.

9. Menurut pihak klinik, seberapa penting mengatur dan menginfokan event secara rapi ke pelanggan?

Jawaban:

Klinik mische mengatakan menginfokan event secara rapi itu sangat penting selain agar event kita sampai ke orang lain, dengan penyampaian event yg rapi bisa juga menandakan bahwa perusahaan terlihat profesional

10. Ceritakan alur reservasi atau booking treatment di klinik selama ini, baik yang dilakukan pelanggan langsung atau via chat?

Jawaban:

Pelanggan menghubungi customer service untuk reservasi treatment, kemudian pihak klinik melakukan follow up dengan membantu menyesuaikan jadwal yang tersedia di klinik dengan jadwal customer, semisal jadwal yang di booking oleh customer di jam 10 namun customer berhalangan maka pihak klinik akan memindahkan ke jam atau ke hari lain yang sesuai dengan jadwal customer. Kemudian untuk penjadwalan reservasi untuk saat ini masih menggunakan form berupa tulis tangan.

11. Pernahkah ada kendala dalam pengaturan jadwal treatment?

Jawaban:

Pastinya ada ketika customer ingin menetapkan jadwal sendiri tanpa bertanya kepada pihak klinik semisal ketika customer ingin menjadwalkan treatment di jam 10 pagi, ternyata di jam itu sudah ada yang booking untuk jadwal treatment maka pihak klinik akan menyesuaikan dengan jadwal yang tersedia dan melakukan follow up ke customer terkait jadwal treatment yang akan dilakukan.

12. Bagaimana cara klinik memastikan pelanggan dapat reminder jadwal mereka?

Jawaban:

Klinik akan melakukan follow up kepada customer terkait jadwal treatment yang akan dilakukan dengan cara mengirimkan customer pesan terkait reminder jadwal melalui whatsapp dan yang dilakukan oleh customer service.

13. Selain treatment, bagaimana proses penjualan produk skincare di klinik?

Jawaban:

Produk skincare yang tersedia di klinik penjualannya bukan hanya secara offline dengan cara datang ke klinik langsung tetapi juga melalui platform shoppe dan juga untuk pelanggan yang ingin membeli produk skincare tidak harus melakukan treatment dulu.

14. Produk skincare biasanya dijual langsung di tempat atau ada sistem pre-order?

Jawaban:

Produk skincare di klinik sebagian besar itu ready di tempat tanpa harus ikut sistem pre-order, namun ada beberapa produk seperti pelembab dan sunscreen yang harus pre-order karena jumlah pesanan yang banyak tetapi pre-order yang dilakukan hanya hitungan hari saja.

15. Bagaimana pencatatan penjualannya? Apakah sudah terintegrasi atau masih manual?

Jawaban:

Pencatatan penjualan di klinik sudah menggunakan sistem yang bernama Pawoon. Dan sudah terintegrasi dengan baik.

16. Seberapa penting menurut Anda, klinik punya platform yang bisa memperkenalkan profil perusahaan secara profesional ke pelanggan?

Jawaban:

Menurut pihak mische platform ini penting karena sekarang serba canggih sebagian besar orang pasti akan mencari melalui google dan juga instagram, namun untuk instagram pihak klinik sudah ada kalau untuk di google belum ada karena tujuan dari pihak mische untuk memperkenalkan lebih luas terkait klinik mische ini.

17. Apa yang ingin Anda tampilkan jika klinik punya halaman profil di sistem atau aplikasi?

Jawaban:

Pihak klinik ingin yang ditampilkan pada website itu adalah tentang profile company yang terdiri dari visi misi, dokter yang tersedia, jadwal treatment dan juga penjelasan terkait treatment lalu tentang galeri event yang akan dilaksanakan dan sudah dilaksanakan kemudian terdapat informasi lokasi dari perusahaan dan promo yang tersedia pada saat ini.

18. Data atau laporan apa yang paling sering dibahas setiap bulan untuk evaluasi kerja klinik?

Jawaban:

Laporan yang sering dibahas untuk menjadi evaluasi setiap bulannya itu yang pertama terkait complain customer, penjualan produk, dan yang terakhir terkait pendapatan.

19. Jika ada sistem yang bisa otomatis kasih laporan promosi yang efektif atau produk yang paling laku, apakah itu membantu?

Jawaban:

Pihak klinik mengatakan sistem yang dapat memberikan promo dan produk yang paling laku itu sangat membantu, karena kecenderungan pasien akan memilih produk yg best seller

20. Untuk perangkat atau device apa saja yang biasa membantu dalam pelaksanaan operasional klinik? (semisal menggunakan laptop untuk pembukuan atau penggunaan tablet untuk pengisian form)

Jawaban:

Untuk perangkat atau device yang biasa digunakan untuk mempermudah pelaksanaan operasional klinik yaitu 1 buah laptop yang digunakan untuk pencatatan pembukuan, 2 tablet yang digunakan untuk transaksi pasien (untuk aplikasi POS/kasir) kemudian terdapat 3 buah handphone yang digunakan untuk operasional dan juga konten.

Tambahan:

1. Di website yang lama selain dari profil perusahaan terdapat juga pemberitahuan tentang jadwal dari dokter yang akan masuk di hari apa saja dan untuk di klinik ini terdapat 2 dokter jadi pasti nya nanti akan di beritahukan jadwalnya di hari apa saja.
2. Untuk operasional dokter dari jam 9 - jam 5.
3. Keinginan untuk penerapan pada website nya yaitu profile perusahaan, kemudian jadwal reservasi yang dibuat seperti tix.id jadi yang mengatur itu admin namun nanti akan muncul di tampilan website dan bisa dilihat oleh calon customer yang ingin reservasi dan menyesuaikan di jam berapa kemudian bisa menghubungi lebih lanjut.
4. kemudian juga bisa ditampilkan product kemudian bisa dilakukan pemesanan langsung di website nya.
5. kemudian reminder event langsung ke email atau wa ke customer yang sudah kita milikin data nya.
6. Yang akan menggunakan adalah abangnya sendiri dan juga dibantu dengan cs untuk melakukan reservasi.

7. Kalo untuk keuangan masih menggunakan pembukuan dengan bantuan excel.
8. Kemudian bisa memberikan informasi berupa lokasi.
9. Menghitung fee dari perawat yang sudah merawat pasien yang lebih dari 5 pengurusan untuk pembukuan dan pembiayaan karyawan.

Dengan ini saya menyatakan bahwa informasi dalam transkrip wawancara ini benar dan sesuai dengan keterangan yang saya sampaikan pada tanggal 26 Maret 2025 di Klinik Kecantikan Mische. Pernyataan ini saya buat dengan penuh kesadaran dan tanpa paksaan, serta dapat dipertanggungjawabkan kebenarannya.

Pekanbaru, 26 Maret
2025

Narasumber


Fauzi Ramadhan, S.Ak.

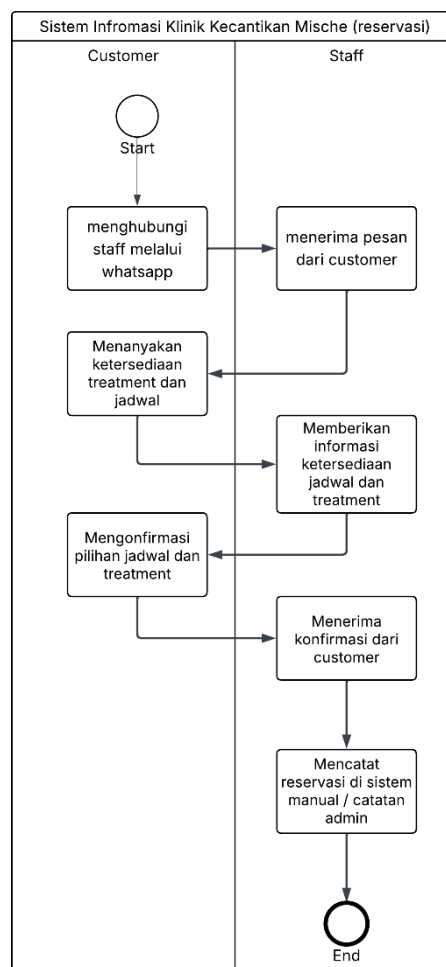
mische
AESTHETIC CLINIC

LAMPIRAN D – PROSES BISNIS *AS-IS*

Pada lampiran ini berisi rancangan proses bisnis dari sistem yang akan dibangun, dan untuk penjelasan nya sebagai berikut ini:

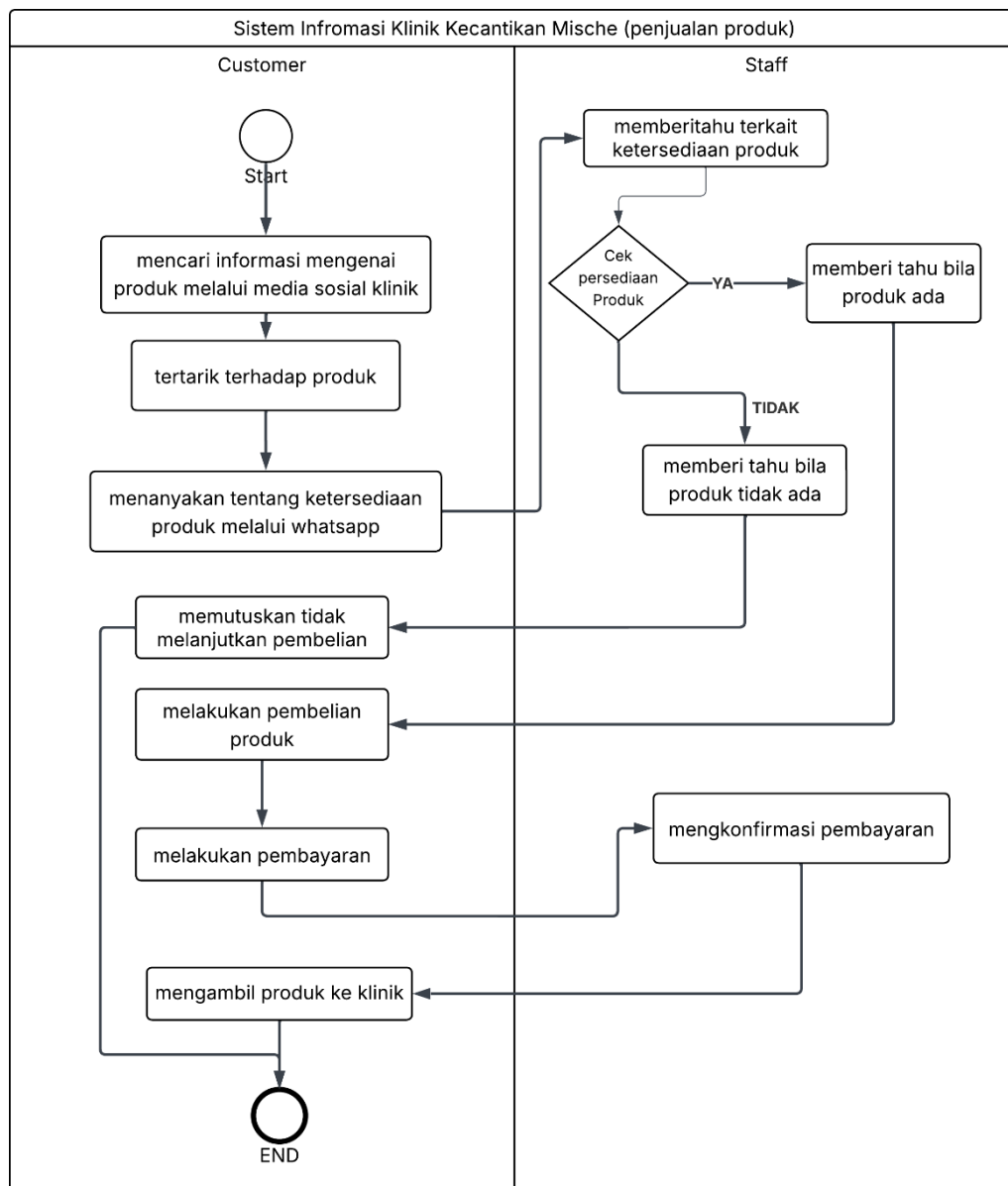
a) Reservasi

Klinik kecantikan Mische memberikan layanan reservasi *treatment* yang membantu *customer* dalam menyesuaikan jadwal dengan dokter di klinik. Untuk proses bisnis nya dimulai dari *customer* yang menghubungi staff melalui *Whatsapp* untuk memberitahu ingin melakukan reservasi, kemdian staff menerima pesan dari *customer*, kemudian *customer* menanyakan jadwal yang tersedia, kemudian staff memberikan informasi jadwal yang tersedia, dan nanti nya *cutomer* akan konfirmasi bila setuju dengan jadwalnya, kemudian staff mencatat jadwal reservasi dari *customer*. Untuk lebih jelasnya dapat dilihat pada gambar dibawah ini.



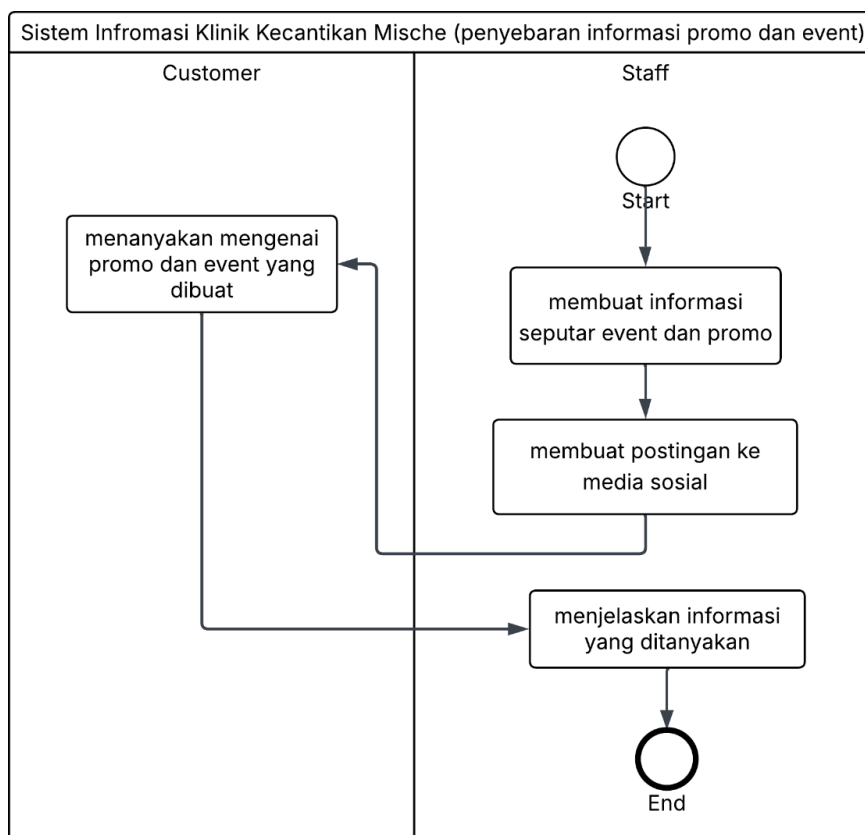
b) Penjualan Produk

Klinik kecantikan Mische juga memiliki produk berupa *skincare*, untuk melakukan pemesanan produk *skincare* dapat dilakukan dengan cara memesan secara langsung melalui kontak *WhatsApp* dari Klinik kecantikan Mische, kemudian staff akan merespon dan memberitahu ketersediaan stok produk, kemudian customer melakukan pembelian produk bila produk tersedia, kemudian melakukan pembayaran dan staff melakukan konfirmasi pembayaran nya, dan bila sudah customer dapat menjemput produknya di Klinik. Untuk pemaparan lebih jelasnya dapat dilihat pada gambar diatas.



c) Penyebaran Informasi *Event* dan Promo

Dalam menjalankan operasionalnya klinik kecantikan Mische sering memuat program promo serta *event* untuk menarik dan mempertahankan *customer*. Saat klinik membuat promo dan *event* pastinya diharuskan untuk menyebarkan informasi nya secara meluas agar tercapai dari target *customer* untuk program promo dan *event* yang dibuat. Tahapan dalam pembuatan promo dan *event* dimulai dari perencanaan oleh staff, kemudian menyebarkan informasi melalui media sosial, dan nantinya terdapat *customer* yang akan menghubungi staff untuk menanyakan informasi lebih lanjut dari promo dan *event* yang di buat. Untuk lebih jelasnya dapat dilihat pada gambar diatas

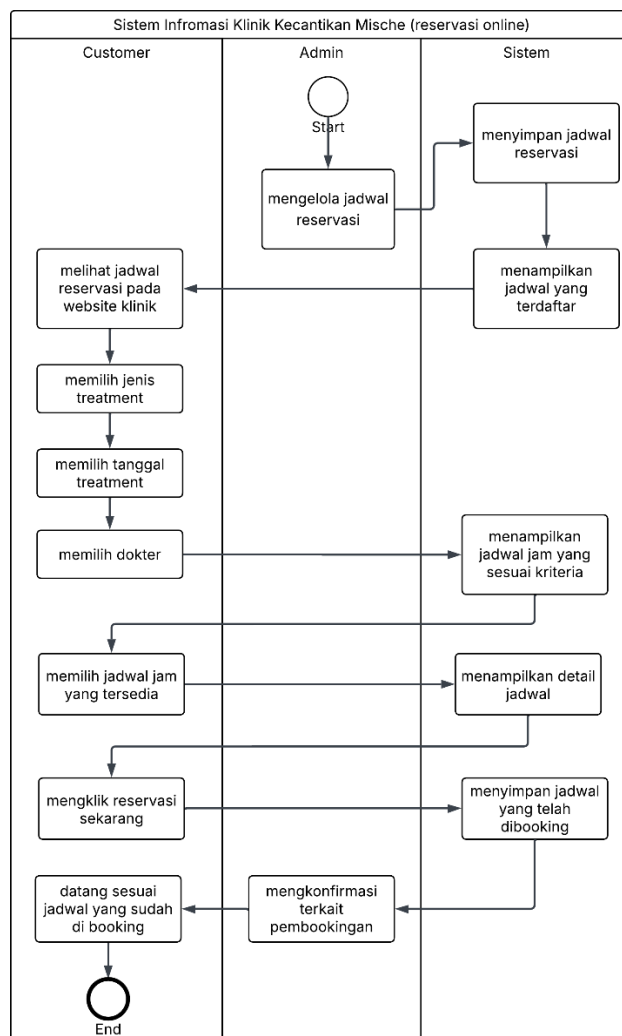


LAMPIRAN E – PROSES BISNIS *TO-BE*

Pada lampiran ini berisi proses bisnis *to-be* dari sistem yang akan dibangun, dan untuk penjelasan nya sebagai berikut ini:

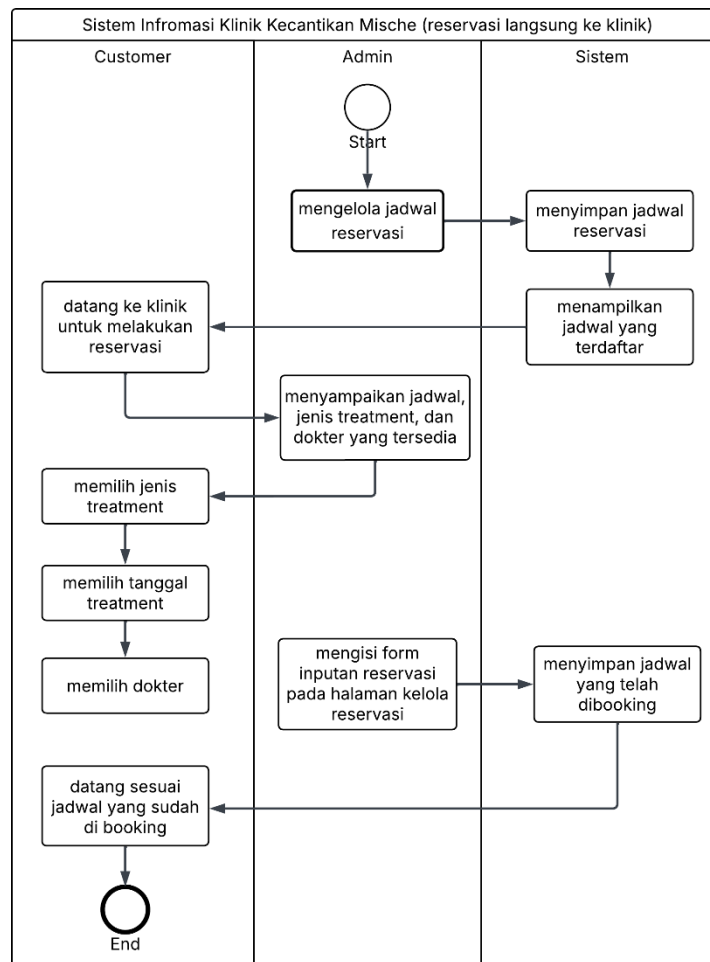
a) *Reservasi Online* (Melalui sistem)

Setelah dilakukan pembaruan pada proses bisnis reservasi yang sudah terintegrasi teknologi, maka menciptakan alur baru yang dimulai dari *admin* untuk melakukan pengelolaan jadwal dengan menambah pada sistem, kemudian secara otomatis sistem akan menampilkan jadwal yang sudah dibuat. Saat sudah di tampilkan oleh sistem maka customer dapat melihat jadwal dan dapat membooking jadwal yang diinginkan, setelah itu sistem akan menyimpan jadwal yang telah diboeking oleh customer dan nanti nya akan di konfirmasi kembali oleh *admin* mengenai jadwal tersebut. Untuk melihat alur lebih jelasnya dapat dilihat pada gambar dibawah ini.



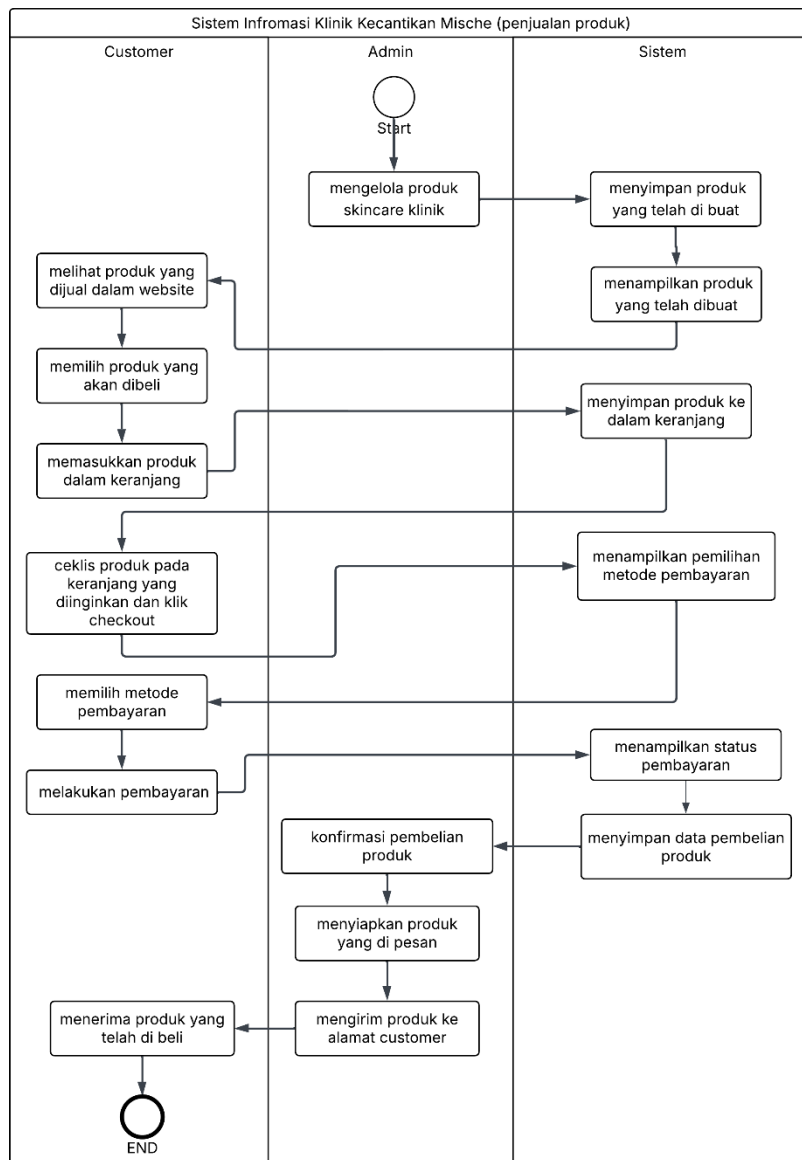
b) *Reservasi Offline* (Datang ke klinik)

Setelah dilakukan pembaruan pada proses bisnis reservasi secara langsung ke klinik dengan penerapan sistem informasi, maka tercipta alur baru yang lebih terintegrasi. Proses dimulai dari admin yang mengelola jadwal reservasi dengan menambahkan data ke dalam sistem, kemudian sistem akan menampilkan jadwal yang telah terdaftar. Ketika customer datang ke klinik untuk melakukan reservasi, admin akan menyampaikan informasi terkait jadwal, jenis treatment, dan dokter yang tersedia. Selanjutnya, customer dapat memilih jadwal, treatment, dan dokter yang diinginkan. Setelah itu admin akan mengisi form inputan reservasi pada halaman sistem untuk mencatat pilihan customer, dan sistem akan menyimpan jadwal yang telah dibooking tersebut. Tahapan akhir dari proses ini adalah customer datang sesuai dengan jadwal yang sudah dibooking. Untuk melihat alur lebih jelasnya dapat dilihat pada gambar bawah ini.



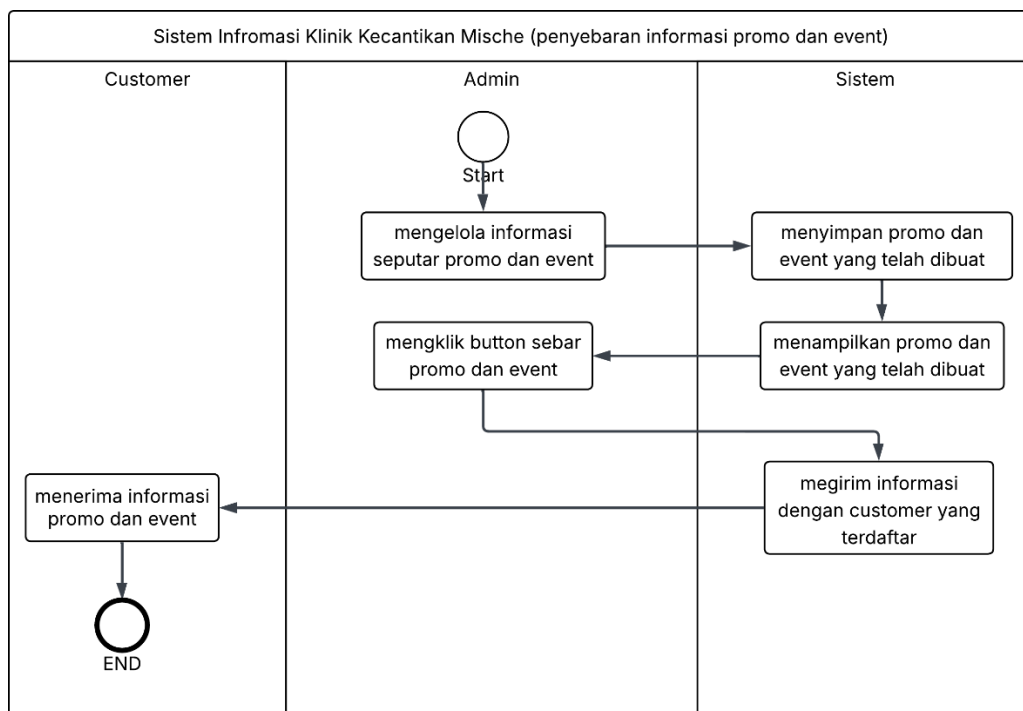
c) Penjualan Produk

Ketika proses bisnis sudah terintegrasi dengan teknologi maka menghasilkan alur baru pada penjualan produk yang dimulai dari *admin* melakukan pengelolaan dengan cara menambahkan produk yang akan dijual, kemudian secara otomatis sistem akan menyimpan daftar produknya. Setelah produk tersimpan dan dapat di tampilkan oleh sistem, maka customer dapat melihat dan melakukan pembelian produk dengan cara memasukkan produk yang diinginkan ke dalam keranjang. Kemudian *customer* dapat memilih produk mana saja yang akan di checkout, dilanjutkan dengan tahapan pembayaran yang metodenya dapat disesuaikan oleh *customer*. Bila sudah melakukan pembayaran maka produk akan langsung di proses oleh pihak klinik dan akan dikirimkan ke alamat customer. Untuk melihat alur lebih jelasnya dapat dilihat pada gambar dibawah ini.



d) Penyebaran Informasi Event dan Promo

Setelah dilakukan pembaruan pada proses bisnis penyebaran informasi event dan promo yang sudah terintegrasi teknologi, maka menciptakan alur baru yang dimulai dari *admin* mengelola event dan promo. Kemudian akan tersimpan dan di tampilkan pada sistem, *admin* dapat melakukan penyebaran promo dan event melalui whatsapp dengan mengklik button share. Untuk nomor tujuan nya adalah customer yang terdaftar pada sistem. Setelah itu *admin* akan mengirimkan yang telah ditentukan dan customer akan menerima informasi promo dan event dengan cepat. Untuk melihat alur lebih jelasnya dapat dilihat pada gambar dibawah ini.



LAMPIRAN F – STRUKTUR TABEL

Pada lampiran ini berisi struktur tabel dari sistem yang akan dibangun, dan untuk penjelasan nya sebagai berikut ini:

1. Struktur Tabel produkKlinik

Struktur dari tabel produkKlinik pada *ERD* yang sudah dibuat dapat dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idProduk	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	nama	<i>Varchar(60)</i>	Nama dari produk	<i>Not null</i>
3	deskripsi	<i>Text</i>	Deskripsi singkat produk	<i>Not null</i>
4	harga	<i>Int(20)</i>	Harga dari produk	<i>Not null</i>
5	stock	<i>Int(20)</i>	<i>Stock</i> produk	<i>Not null</i>
6	idKategori	<i>Int(12)</i>	<i>Foreign Key</i>	<i>Not null</i>

2. Struktur Tabel kategoriProduk

Struktur dari tabel kategoriProduk pada *ERD* yang sudah dibuat dapat dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idKategori	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	nama	<i>Varchar(60)</i>	Nama kategori	<i>Not null</i>
3	deskripsi	<i>Text</i>	Deskripsi Kategori	<i>Not Null</i>

3. Struktur Tabel penjualan

Struktur dari tabel penjualan pada *ERD* yang sudah dibuat dapat dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idPenjualan	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	tanggal	<i>date</i>	Tanggal penjualan	<i>Not null</i>
3	totalHarga	<i>Int(20)</i>	Total harga produk	<i>Not null</i>
4	status	<i>Varchar(60)</i>	Status dari transaksi	<i>Not null</i>
5	<i>idUser</i>	<i>Int(12)</i>	<i>Froreign Key</i>	<i>Not null</i>
6	idPromo	<i>Int(12)</i>	<i>Foreign Key</i>	<i>Not null</i>

4. Struktur Tabel detailPenjualan

Struktur dari tabel detailPenjualan pada *ERD* yang sudah dibaut dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idDetailPenjualan	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	jumlahProduk	<i>Int(12)</i>	Jumlah produk terjual	<i>Not null</i>
3	idPenjualan	<i>Int(12)</i>	<i>Foreign Key</i>	<i>Not null</i>
4	idProduk	<i>Int(12)</i>	<i>Foreign Key</i>	<i>Not null</i>

5. Struktur Tabel keranjang

Struktur dari tabel keranjang pada *ERD* yang sudah dibaut dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idKeranjang	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	jumlahProduk	<i>Int(12)</i>	Jumlah produk di keranjang	<i>Can null</i>
3	idProduk	<i>Int(12)</i>	<i>Foreign Key</i>	<i>Not null</i>
4	idUser	<i>Int(12)</i>	<i>Foreign Key</i>	<i>Not null</i>

6. Struktur Tabel promo

Struktur dari tabel promo pada *ERD* yang sudah dibaut dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idPromo	<i>Int(12)</i>	<i>Primary key</i>	<i>Not null</i>
2	gambar	<i>Varchar(255)</i>	Gambar promo	<i>Not null</i>
3	nama	<i>Varchar(60)</i>	Nama Promo	<i>Not null</i>
4	jenisPromo	<i>Varchar(60)</i>	Jenis Promo	<i>Not null</i>
5	kode	<i>Varchar(12)</i>	Kode promo produk	<i>Can null</i>
6	diskon	<i>Int(12)</i>	Diskon produk	<i>Not null</i>
7	dekripsi	<i>Text</i>	Deskripsi singkat promo	<i>Not null</i>
8	tanggalMulai	<i>date</i>	Tanggal dimulai promo	<i>Not null</i>
9	tanggalSelesai	<i>Date</i>	Tanggal berakhir promo	<i>Not null</i>
10	minimalTransaksi	<i>Int(20)</i>	Harga minimal pembelian produk	<i>Can null</i>
11	status	<i>boolean</i>	Satatus promo	<i>Not null</i>
12	idKategori	<i>Int(12)</i>	<i>Foreign Key</i>	<i>Can null</i>
13	idProduk	<i>Int(12)</i>	<i>Foreign Key</i>	<i>Can null</i>

7. Struktur Tabel testimoni

Struktur dari tabel testimoni pada *ERD* yang sudah dibaut dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idTestimoni	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	namaTester	<i>Varchar(20)</i>	Nama dari tester	<i>Not null</i>
3	jenisTestimoni	<i>Varchar(60)</i>	Jenis dari testimoni	<i>Not null</i>
4	tanggalTreatment	<i>Date</i>	Tanggal melakukan <i>treatment</i>	<i>Not null</i>
5	buktiFoto	<i>Varchar(20)</i>	Bukti testimoni	<i>Not null</i>

8. Struktur Tabel profilPerusahaan

Struktur dari tabel profilPerusahaan pada *ERD* yang sudah dibaut dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idProfil	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	visi	<i>Text</i>	Visi klinik	<i>Not null</i>
3	misi	<i>Text</i>	Misi klinik	<i>Not null</i>
4	deskripsiPerusahaan	<i>Text</i>	Deskripsi klinik	<i>Not null</i>
5	nomorCustomerService	<i>Int(16)</i>	Nomor <i>customer service</i>	<i>Not null</i>

9. Struktur Tabel event

Struktur dari tabel event pada *ERD* yang sudah dibaut dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idEvent	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	deskripsi	<i>Text</i>	Deskripsi dari <i>event</i>	<i>Not null</i>
3	foto	<i>Varchar(255)</i>	Foto Event	<i>Not null</i>
4	tanggalMulai	<i>Date</i>	Tanggal dimulai <i>event</i>	<i>Not null</i>
5	tanggalSelesai	<i>Date</i>	Tanggal selesai <i>event</i>	<i>Not null</i>
6	lokasi	<i>Varchar(100)</i>	Lokasi <i>event</i> berlangsung	<i>Not null</i>

10. Struktur Tabel profilDokter

Struktur dari tabel profilDokter pada *ERD* yang sudah dibaut dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idDokter	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	nama	<i>Varchar(60)</i>	Nama dari dokter	<i>Not null</i>
3	foto	<i>Varchar(255)</i>	Foto dari dokter	<i>Can null</i>
4	email	<i>Varchar(60)</i>	Email dari dokter	<i>Not null</i>
5	deskripsi	<i>Text</i>	Biografi	<i>Not null</i>

11. Struktur Tabel reservasi

Struktur dari tabel reservasi pada *ERD* yang sudah dibuat dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idReservasi	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	namaCustomer	<i>Varchar(60)</i>	Nama Customer	<i>Not null</i>
3	nomorWa	<i>Int(16)</i>	Nomor Customer	<i>Not null</i>
4	jenisTreatment	<i>Varchar(60)</i>	Jenis Treatment	<i>Not null</i>
5	tanggal	<i>date</i>	Tanggal reservasi	<i>Not null</i>
6	status	<i>Varchar(60)</i>	Status reservasi	<i>Not null</i>
7	idUser	<i>Int(12)</i>	Foreign Key	<i>Can null</i>
8	idDokter	<i>Int(12)</i>	Foreign Key	<i>Not null</i>
9	idJadwal	<i>Int(12)</i>	Foreign Key	<i>Not null</i>

12. Struktur Tabel jadwalReservasi

Struktur dari tabel jadwalReservasi pada *ERD* yang sudah dibuat dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idJadwal	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	jamMulai	<i>Time</i>	Waktu mulai	<i>Not null</i>
3	jamSelesai	<i>Time</i>	Waktu selesai	<i>Not null</i>

13. Struktur Tabel kegiatan

Struktur dari tabel kegiatan pada *ERD* yang sudah dibuat dilihat sebagai berikut ini:

No	Kolom	Type Data	Keterangan	Null
1	idKegiatan	<i>Int(12)</i>	<i>Primary Key</i>	<i>Not null</i>
2	namaKegiatan	<i>Varchar(60)</i>	Nama kegiatan klinik	<i>Not null</i>
3	deskripsi	<i>Text</i>	Foto kegiatan	<i>Not null</i>
4	foto	<i>Varchar(255)</i>	Foto kegiatan	<i>Not null</i>
5	tanggalKegiatan	<i>Date</i>	Tanggal kegiatan	<i>Not null</i>

LAMPIRAN G – PERANCANGAN *API*

Pada lampiran ini berisi perancangan API dari sistem yang akan dibangun, dan untuk penjelasannya sebagai berikut ini:

No	Endpoint	Metode	Deskripsi
1.	/api/auth/register	POST	Mendaftarkan pengguna
2.	/api/auth/login	POST	Masuk kedalam sistem
3.	/api/auth/logout	POST	Keluar dari sistem
4.	/api/auth/reset-password	POST	Mengatur ulang password
5.	/api/admin/users	GET	Mengambil daftar user oleh <i>admin</i>
6.	/api/admin/users/{idUser}	GET	Mengambil data user berdasarkan id
7.	/api/admin/users	POST	Menambahkan user baru
8.	/api/admin/users/{idUser}	PUT	Memperbarui data user berdasarkan id
9.	/api/admin/users/{idUser}	DELETE	Menghapus data user
10.	/api/admin/doctors	GET	Mengambil daftar dokter
11.	/api/customer/doctors	GET	Menampilkan data dokter pada halaman customer
12.	/api/admin/doctors/{idDokter}	GET	Mengambil data dokter berdasarkan id
13.	/api/admin/doctors	POST	Menambahkan data dokter
14.	/api/admin/doctors/{idDokter}	PUT	Memperbarui data dokter berdasarkan id
15.	/api/admin/doctors/{idDokter}	DELETE	Menghapus data dokter berdasarkan id
16.	/api/admin/clinic/profile	GET	Menampilkan data profil klinik
17.	/api/customer/clinic/	GET	Menampilkan data profil klinik pada halaman customer
18.	/api/admin/clinic/profile	POST	Menambahkan data profil klinik
19.	/api/admin/clinic/profile/{idProfil}	PUT	Memperbarui profil klinik
20.	/api/admin/clinic/profile/{idProfil}	DELETE	Menghapus profil klinik

No	Endpoint	Metode	Deskripsi
21.	/api/admin/testimoni	GET	Menampilkan daftar testimoni
22.	/api/customer/testimoni	GET	Menampilkan data testimoni pada halaman customer
23.	/api/customer/testimoni/{idTestimoni}	GET	Menampilkan detail testimoni
24.	/api/admin/testimoni	POST	Menambahkan data testimoni
25.	/api/admin/testimoni/{idTestimoni}	PUT	Memperbarui data testimoni
26.	/api/admin/testimoni/{idTestimoni}	DELETE	Menghapus data testimoni
27.	/api/admin/event	GET	Menampilkan daftar event
28.	/api/customer/event	GET	Menampilkan data event pada halaman customer
29.	/api/customer/event/{idEvent}	GET	Menampilkan detail event
30.	/api/admin/event	POST	Menambahkan data event
31.	/api/admin/event/{idEvent}	PUT	Memperbarui data event
32.	/api/admin/event/{idEvent}	DELETE	Menghapus data event
33.	/api/admin/reservations	GET	Menampilkan daftar reservasi
34.	/api/customer/reservations	GET	Menampilkan daftar reservasi pada halaman customer
35.	/api/customer/reservations/{idReservasi}	GET	Menampilkan detail reservasi
36.	/api/admin/reservations/{idReservasi}	PATCH	Mengubah status reservasi
37.	/api/customer/reservations/{idReservasi}	POST	Melakukan reservasi
38.	/api/admin/reservations/{idReservasi}	DELETE	Menghapus data reservasi
39.	/api/admin/schedule	GET	Menampilkan data jadwal reservasi
40.	/api/admin/schedule	POST	Menambah data jadwal reservasi
41.	/api/admin/schedule/{idJadwal}	PUT	Memperbarui data jadwal reservasi
42.	/api/admin/schedule/{idJadwal}	DELETE	Menghapus data jadwal reservasi
43.	/api/admin/kategori	GET	Menampilkan daftar kategori
44.	/api/admin/kategori	POST	Menambahkan data kategori

No	Endpoint	Metode	Deskripsi
45.	/api/admin/kategori/{idKategori}	PUT	Memperbarui kategori
46.	/api/admin/kategori/{idKategori}	DELETE	Menghapus data kategori
47.	/api/admin/product	GET	Menampilkan daftar produk
48.	/api/admin/product	POST	Menambahkan data produk
49.	/api/admin/product/{idProduk}	PUT	Memperbarui data produk
50.	/api/admin/product/{idProduk}	PATCH	Memperbarui stok produk
51.	/api/customer/product	GET	Menampilkan daftar produk pada halaman customer
52.	/api/customer/product/{idProduk}	GET	Menampilkan detail produk
53.	/api/customer/product/{idProduk}/order	POST	Memesan produk
54.	/api/admin/product/{idProduk}	DELETE	Menghapus data produk
55.	/api/customer/card	GET	Menampilkan data produk di keranjang
56.	/api/customer/card	POST	Menambahkan ke keranjang
57.	/api/customer/card/{idKeranjang}	PATCH	Memperbarui jumlah produk
58.	/api/customer/card/{idKeranjang}	DELETE	Menghapus data dari keranjang
59.	/api/admin/kegiatan	GET	Menampilkan daftar kegiatan
60.	/api/customer/kegiatan	GET	Menampilkan data kegiatan pada halaman customer
61.	/api/admin/kegiatan	POST	Menambah data kegiatan
62.	/api/admin/kegiatan/{idKegiatan}	PUT	Memperbarui data kegiatan
63.	/api/admin/kegiatan	DELETE	Menghapus data kegiatan
64.	/api/admin/promo	GET	Menampilkan daftar promo
65.	/api/customer/promo	GET	Menampilkan promo pada halaman customer
66.	/api/admin/promo	POST	Menambah promo
67.	/api/admin/promo/{idPromo}	PUT	Memperbarui promo
68.	/api/admin/promo/{idPromo}	DELETE	Menghapus promo
69.	/api/admin/report-sales	GET	Mengexport data penjualan ke excel

No	Endpoint	Metode	Deskripsi
70.	/api/admin/report-reservations	GET	Mengexport data reservasi ke excel
71.	/api/customer/profil/{idUser}	GET	Menampilkan profile customer
72.	/api/customer/profil/{idUser}	PUT	Memperbarui profile user
73.	/api/admin/promo/{idPromo}/send-wa	POST	Mengirim informasi promo ke WhatsApp
74.	/api/admin/event/{idEvent}/send-wa	POST	Mengirim informasi promo ke WhatsApp
75.	/api/admin/penjualan	GET	Menampilkan data penjualan
76.	/api/admin/penjualan/{idPenjualan}	DELETE	Menghapus data penjualan
77.	/api/admin/penjualan/{idPenjualan}	PATCH	Mengubah status data penjualan
78.	/api/customer/penjualan/{idPenjualan}	PATCH	Menerima barang

LAMPIRAN H – RANCANGAN UNIT TESTING

Pada lampiran ini berisi rancangan *unit testing* dari sistem yang akan dibangun, dan untuk penjelasannya sebagai berikut ini:

a) *User (admin)*

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllUsers()	Ambil semua daftar user	-	Mengembalikan list semua user sesuai database
2	getUserById(\$idUser)	Ambil user dengan ID valid	ID = 1 (ada di database)	Object user sesuai ID
3	getUserById(\$idUser)	Ambil user dengan ID tidak ada	ID = 999 (tidak ada di database)	Error: "User tidak ditemukan"
4	createUser(\$dataUser)	Tambah user baru dengan data valid	Data user yang belum terdaftar dan valid	Akun berhasil dibuat
5	createUser(\$dataUser)	Tambah user dengan email sudah ada	Email yang sama dengan yang ada di database	Error: "Email sudah digunakan"
6	updateUser(\$idUser, \$dataUser)	Update user dengan ID valid	ID = 1, data valid	User berhasil diperbarui
7	updateUser(\$idUser, \$dataUser)	Update user dengan ID tidak ada	ID = 999, data valid	Error: "User tidak ditemukan"
8	deleteUser(\$idUser)	Hapus user dengan ID valid	ID = 1 (ada di database)	User berhasil dihapus
9	deleteUser(\$idUser)	Hapus user dengan ID tidak ada	ID = 999 (tidak ada di database)	Error: "User tidak ditemukan"

b) *Dokter*

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllDoctors()	Ambil semua data dokter	-	Mengembalikan list semua dokter
2	getDoctorById(\$idDokter)	Ambil dokter dengan ID valid	ID = 1 (ada di database)	Object dokter sesuai ID

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
3	getDoctorById(\$idDokter)	Ambil dokter dengan ID tidak ada	ID = 999 (tidak ada di database)	Error: "Dokter tidak ditemukan"
4	createDoctor(\$dataDokter)	Tambah dokter baru dengan data valid	Data dokter yang valid dan belum terdaftar	Dokter baru berhasil dibuat
5	createDoctor(\$dataDokter)	Tambah dokter tanpa nama	Nama kosong	Error: "Field name wajib diisi"
6	updateDoctor(\$idDokter, \$dataDokter)	Update dokter dengan ID valid	ID = 1, data valid	Data dokter berhasil diperbarui
7	updateDoctor(\$idDokter, \$dataDokter)	Update dokter dengan ID tidak ada	ID = 999, data valid	Error: "Dokter tidak ditemukan"
8	deleteDoctor(\$idDokter)	Hapus dokter dengan ID valid	ID = 1 (ada di database)	Data dokter berhasil dihapus
9	deleteDoctor(\$idDokter)	Hapus dokter dengan ID tidak ada	ID = 999 (tidak ada di database)	Error: "Dokter tidak ditemukan"
10	getPublicDoctors()	Ambil daftar dokter untuk customer (public)	-	Mengembalikan list dokter publik (field terbatas)

c) Profil Klinik

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getProfile()	Ambil profil klinik (<i>admin</i>)	-	Mengembalikan detail lengkap profil klinik
2	getPublicProfile()	Ambil profil klinik untuk customer	-	Mengembalikan data profil publik (field tertentu)
3	createProfile(\$data Profil)	Tambah profil klinik dengan data valid	Data profil klinik yang valid	Profil klinik baru berhasil dibuat
4	updateProfile(\$idProfil, \$dataProfil)	Update profil dengan ID valid	ID = 1, data valid	Profil berhasil diperbarui
5	updateProfile(\$idProfil, \$dataProfil)	Update profil dengan ID tidak ada	ID = 999, data valid	Error: "Profil tidak ditemukan"
6	deleteProfile(\$idProfil)	Hapus profil dengan ID valid	ID = 1 (ada di DB)	Profil berhasil dihapus
7	deleteProfile(\$idProfil)	Hapus profil dengan ID tidak ada	ID = 999 (tidak ada di DB)	Error: "Profil tidak ditemukan"

d) Testimoni

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllTestimonials()	Ambil semua testimoni (<i>admin</i>)	-	Mengembalikan list semua testimoni lengkap
2	getPublicTestimonials()	Ambil semua testimoni untuk customer	-	Mengembalikan list testimoni publik (field tertentu)
3	getTestimoniById(\$idTestimoni)	Ambil testimoni dengan ID valid	ID = 1 (ada di database)	Object testimoni sesuai ID
4	getTestimoniById(\$idTestimoni)	Ambil testimoni dengan ID tidak ada	ID = 999 (tidak ada di database)	Error: "Testimoni tidak ditemukan"
5	createTestimoni(\$dataTestimoni)	Tambah testimoni dengan data valid	Data testimoni yang valid	Testimoni baru berhasil dibuat
6	updateTestimoni(\$idTestimoni, \$dataTestimoni)	Update testimoni dengan ID valid	ID = 1, data valid	Testimoni berhasil diperbarui
7	deleteTestimoni(\$idTestimoni)	Hapus testimoni dengan ID valid	ID = 1 (ada di database)	Testimoni berhasil dihapus
8	deleteTestimoni(\$idTestimoni)	Hapus testimoni dengan ID tidak ada	ID = 999 (tidak ada di database)	Error: "Testimoni tidak ditemukan"

e) Event

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllEvents()	Ambil semua event (<i>admin</i>)	-	Mengembalikan list semua event
2	getPublicEvents()	Ambil semua event untuk customer	-	Mengembalikan list event publik (field tertentu)
3	getEventById(\$idEvent)	Ambil event dengan ID valid	ID = 1 (ada di database)	Object event sesuai ID
4	getEventById(\$idEvent)	Ambil event dengan ID tidak ada	ID = 999 (tidak ada)	Error: "Event tidak ditemukan"

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
			di database)	
5	createEvent(\$dataEvent)	Tambah event dengan data valid	Data event yang valid	Event baru berhasil dibuat
6	updateEvent(\$idEvent, \$dataEvent)	Update event dengan ID valid	ID = 1, data valid	Event berhasil diperbarui
7	deleteEvent(\$idEvent)	Hapus event dengan ID valid	ID = 1 (ada di database)	Event berhasil dihapus
8	deleteEvent(\$idEvent)	Hapus event dengan ID tidak ada	ID = 999 (tidak ada di database)	Error: "Event tidak ditemukan"
9	sendEventWA(\$idEvent)	Kirim event valid via WA	ID = 1	Event berhasil dikirim ke WhatsApp
10	sendEventWA(\$idEvent)	Kirim event yang tidak ada	ID = 999	Error: "Event tidak ditemukan"

f) Reservasi

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllReservations()	Ambil semua reservasi (<i>admin</i>)	-	Mengembalikan list semua reservasi
2	getCustomerReservations(\$id User)	Ambil semua reservasi untuk customer	customerId = 5	Mengembalikan list reservasi milik customer tersebut
3	getReservationById(\$idReservasi)	Ambil reservasi dengan ID valid	ID = 1 (ada di database)	Object reservasi sesuai ID
4	getReservationById(\$idReservasi)	Ambil reservasi dengan ID tidak ada	ID = 999 (tidak ada di database)	Error: "Reservasi tidak ditemukan"
5	bookReservation(\$id, \$data)	Customer melakukan reservasi (patch) valid	ID = 1, data patch valid	Reservasi berhasil dipesan
6	deleteReservation(\$idReservasi)	Hapus reservasi dengan ID valid	ID = 1 (ada di database)	Reservasi berhasil dihapus

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
7	deleteReservation(\$idReservasi)	Hapus reservasi dengan ID tidak ada	ID = 999 (tidak ada di database)	Error: "Reservasi tidak ditemukan"
8	updateReservationStatus(\$idReservasi, \$status)	Ubah status reservasi valid	ID = 1, status = "approved"	Status reservasi berhasil diubah
9	updateReservationStatus(\$idReservasi, \$status)	Ubah status reservasi yang tidak ada	ID = 999, status = "approved"	Error: "Reservasi tidak ditemukan"
10	updateReservationStatus(\$idReservasi, \$status)	Ubah status reservasi dengan status tidak valid	ID = 1, status = "invalid_status"	Error: "Status tidak valid"

g) Jadwal

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllSchedules()	Ambil semua jadwal reservasi	-	Mengembalikan list semua jadwal reservasi
2	getScheduleById(\$idJadwal)	Ambil jadwal dengan ID valid	idJadwal = 1 (ada di DB)	Object jadwal sesuai ID
3	getScheduleById(\$idJadwal)	Ambil jadwal dengan ID tidak ada	idJadwal = 999	Error: "Jadwal tidak ditemukan"
4	createSchedule(\$dataJadwal)	Tambah jadwal baru dengan data valid	jamMulai=10:00, jamSelesai=11:00	Jadwal berhasil ditambahkan, response 201 dengan data jadwal
5	createSchedule(\$dataJadwal)	Tambah jadwal baru dengan	jamMulai kosong	Error: "Data tidak valid"

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
		data tidak valid		
6	updateSchedule(\$idJadwal, \$dataJadwal)	Update jadwal dengan ID valid	idJadwal = 1, jamMulai=12:00	Jadwal berhasil diperbarui, response 200 dengan data baru
7	updateSchedule(\$idJadwal, \$dataJadwal)	Update jadwal dengan ID tidak ada	idJadwal = 999	Error: "Jadwal tidak ditemukan"
8	updateSchedule(\$idJadwal, \$dataJadwal)	Update jadwal dengan data tidak valid	idJadwal = 1, jamMulai kosong	Error: "Data tidak valid"
9	deleteSchedule(\$idJadwal)	Hapus jadwal dengan ID valid	idJadwal = 1	Jadwal berhasil dihapus, response 200
10	deleteSchedule(\$idJadwal)	Hapus jadwal dengan ID tidak ada	idJadwal = 999	Error: "Jadwal tidak ditemukan"

h) Kategori

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllCategories()	Ambil semua kategori	-	Mengembalikan list semua kategori
2	createCategory(\$dataKategori)	Tambah kategori valid	Nama = "Skincare"	Kategori baru berhasil dibuat
3	createCategory(\$dataKategori)	Tambah kategori tanpa nama	Nama kosong	Error: "Field nama wajib diisi"
4	updateCategory(\$idKategori, \$dataKategori)	Update kategori dengan ID valid	ID = 1, Nama = "Bodycare"	Kategori berhasil diperbarui
5	updateCategory(\$idKategori, \$dataKategori)	Update kategori dengan ID tidak ada	ID = 999, Nama valid	Error: "Kategori tidak ditemukan"
6	deleteCategory(\$idKategori)	Hapus kategori dengan ID valid	ID = 1	Kategori berhasil dihapus

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
7	deleteCategory(\$idKategori)	Hapus kategori dengan ID tidak ada	ID = 999	Error: "Kategori tidak ditemukan"

i) Produk

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllProducts()	Ambil semua produk (<i>admin</i>)	-	Mengembalikan list semua produk
2	getPublicProducts()	Ambil semua produk untuk customer	-	Mengembalikan list produk publik
3	getProductById(\$idProduk)	Ambil detail produk valid	ID = 1 (ada di database)	Detail produk sesuai ID
4	getProductById(\$idProduk)	Ambil detail produk yang tidak ada	ID = 999 (tidak ada di database)	Error: "Produk tidak ditemukan"
5	createProduct(\$dataProduk)	Tambah produk dengan data valid	Nama = "Facial Wash", harga, stok valid	Produk baru berhasil dibuat
6	createProduct(\$dataProduk)	Tambah produk tanpa nama	Nama kosong	Error: "Field nama wajib diisi"
7	updateProduct(\$idProduk, \$dataProduk)	Update produk dengan ID valid	ID = 1, data baru valid	Produk berhasil diperbarui
8	updateProduct(\$idProduk, \$dataProduk)	Update produk dengan ID tidak ada	ID = 999, data baru valid	Error: "Produk tidak ditemukan"
9	updateStock(\$idProduk, \$stokProduk)	Update stok produk valid	ID = 1, stok = 50	Stok produk berhasil diperbarui
10	updateStock(\$idProduk, \$stokProduk)	Update stok produk dengan nilai negatif	ID = 1, stok = -5	Error: "Stok tidak boleh negatif"
11	updateStock(\$id, \$stok)	Update stok pada produk yang tidak ada	ID = 999, stok = 20	Error: "Produk tidak ditemukan"

j) Keranjang

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	addToCart(\$idProduk, \$jumlah)	Tambah item ke keranjang valid	productId = 1, jumlah = 2	Item berhasil ditambahkan, keranjang diperbarui
2	addToCart(\$idProduk, \$jumlah)	Tambah produk yang tidak ada	productId = 999, jumlah = 1	Error: "Produk tidak ditemukan"
3	updateCartItem(\$idKeranjang, \$jumlah)	Update jumlah produk valid	cartId = 1, jumlah = 3	Item keranjang berhasil diperbarui
4	updateCartItem(\$idKeranjang, \$jumlah)	Update item keranjang yang tidak ada	cartId = 999, jumlah = 3	Error: "Item keranjang tidak ditemukan"
5	deleteCartItem(\$idKeranjang)	Hapus item keranjang valid	cartId = 1	Item keranjang berhasil dihapus
6	deleteCartItem(\$idKeranjang)	Hapus item keranjang yang tidak ada	cartId = 999	Error: "Item keranjang tidak ditemukan"

k) Kegiatan

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllKegiatan()	Ambil semua kegiatan (<i>admin</i>)	-	Mengembalikan list semua kegiatan
2	getPublicKegiatan()	Ambil kegiatan customer (publik)	-	Mengembalikan list kegiatan publik
3	createKegiatan(\$dataKegiatan)	Tambah kegiatan valid	Nama = "Workshop Skincare"	Kegiatan baru berhasil dibuat
4	createKegiatan(\$dataKegiatan)	Tambah kegiatan tanpa nama	{ } (kosong)	Error: "Field nama wajib diisi"
5	updateKegiatan(\$idKegiatan, \$dataKegiatan)	Update kegiatan dengan ID valid	ID = 1, Nama = "Workshop Haircare"	Kegiatan berhasil diperbarui
6	updateKegiatan(\$idKegiatan, \$dataKegiatan)	Update kegiatan dengan ID tidak ada	ID = 999, Data valid	Error: "Kegiatan tidak ditemukan"

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
7	deleteKegiatan(\$idKegiatan)	Hapus kegiatan dengan ID valid	ID = 1	Kegiatan berhasil dihapus
8	deleteKegiatan(\$idKegiatan)	Hapus kegiatan dengan ID tidak ada	ID = 999	Error: "Kegiatan tidak ditemukan"

1) Promo

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getAllPromos()	Ambil semua promo (<i>admin</i>)	-	Return list semua promo
2	getPublicPromos()	Ambil semua promo untuk customer	-	Return list promo publik
3	createPromo(\$dataPromo)	Tambah promo dengan data valid	judul = "Diskon 10%"	Return object promo baru, tersimpan di DB
4	createPromo(\$dataPromo)	Tambah promo tanpa judul	{}	Throw exception / return error "Field judul wajib diisi"
5	updatePromo(\$idPromo, \$dataPromo)	Update promo dengan ID valid	id = 1, judul = "Diskon 20%"	Return sukses "Promo diperbarui"
6	updatePromo(\$idPromo, \$dataPromo)	Update promo dengan ID tidak ada	id = 999, data valid	Throw exception / return error "Promo tidak ditemukan"
7	deletePromo(\$idPromo)	Hapus promo dengan ID valid	id = 1	Return sukses "Promo dihapus"
8	deletePromo(\$id)	Hapus promo dengan ID tidak ada	id = 999	Throw exception / return error "Promo tidak ditemukan"
9	sendPromoWA(\$idPromo)	Kirim promo valid via WA	ID = 1	Promo berhasil dikirim ke WhatsApp
10	sendPromoWA(\$idPromo)	Kirim promo yang tidak ada	ID = 999	Error: "Promo tidak ditemukan"

m) Penjualan

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getSales()	Ambil semua data penjualan	-	Mengembalikan list semua penjualan
2	getSales()	Ambil data penjualan dengan filter	filter=status=completed	Mengembalikan list penjualan sesuai filter
3	deleteSale(\$idPenjualan)	Hapus penjualan valid	ID = 1	Penjualan berhasil dihapus
4	deleteSale(\$idPenjualan)	Hapus penjualan yang tidak ada	ID = 999	Error: "Penjualan tidak ditemukan"
5	updateSaleStatus(\$idPenjualan, \$status)	Ubah status penjualan valid	ID = 1, status = "shipped"	Status penjualan berhasil diperbarui
6	updateSaleStatus(\$idPenjualan, \$status)	Ubah status penjualan yang tidak ada	ID = 999, status = "shipped"	Error: "Penjualan tidak ditemukan"
7	updateSaleStatus(\$idPenjualan, \$status)	Ubah status penjualan dengan status tidak valid	ID = 1, status = "invalid_status"	Error: "Status tidak valid"
8	confirmReceiveOrder(\$idPenjualan)	Customer terima barang valid	ID = 1	Status penjualan berubah menjadi "delivered"
9	confirmReceiveOrder(\$idPenjualan)	Customer terima barang yang tidak ada	ID = 999	Error: "Penjualan tidak ditemukan"

n) Laporan

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	exportReportSales(\$filter)	Export penjualan dengan filter produk valid dan tanggal range berisi data	Produk = "Facial Wash", Tanggal = 2025-01-01	File Excel berisi data penjualan produk tersebut

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
			s/d 2025-01-31	dalam range tanggal
2	exportReportSales(\$filter)	Export penjualan dengan filter kategori valid dan tanggal range berisi data	Kategori = "Skincare", Tanggal = 2025-02-01 s/d 2025-02-28	File Excel berisi data penjualan kategori tersebut dalam range tanggal
3	exportReportSales(\$filter)	Export penjualan dengan filter tanggal range berisi data	Tanggal = 2025-03-01 s/d 2025-03-31	File Excel berisi semua penjualan dalam range tanggal
4	exportReportSales(\$filter)	Export penjualan dengan filter produk valid tapi tidak ada data dalam range tanggal	Produk = "Facial Wash", Tanggal = 2025-04-01 s/d 2025-04-30	File Excel kosong dengan hanya header kolom
5	exportReportSales(\$filter)	Export penjualan dengan filter kategori valid tapi tidak ada data dalam range tanggal	Kategori = "Bodycare", Tanggal = 2025-05-01 s/d 2025-05-31	File Excel kosong dengan hanya header kolom
6	exportReportReservations(\$filter)	Export reservasi dengan filter jenis treatment valid dan tanggal range berisi data	Treatment = "Facial", Tanggal = 2025-01-01 s/d 2025-01-31	File Excel berisi data reservasi treatment tersebut dalam range tanggal
7	exportReportReservations(\$filter)	Export reservasi dengan filter tanggal range berisi data	Tanggal = 2025-02-01 s/d 2025-02-28	File Excel berisi semua reservasi dalam range tanggal
8	exportReportReservations(\$filter)	Export reservasi dengan filter treatment valid tapi tidak ada data dalam range tanggal	Treatment = "Massage", Tanggal = 2025-03-01 s/d 2025-03-31	File Excel kosong dengan hanya header kolom

o) Profil

No	Fungsi yang Diuji	Skenario Pengujian	Data Uji	Expected Output
1	getProfileById(\$id)	Ambil profil customer valid	id = 1	Return data profil lengkap (nama, email, alamat, dll.)
2	getProfileById(\$id)	Ambil profil customer tidak ada	id = 999	Throw exception / return error "Profil tidak ditemukan"
3	updateProfile(\$id, \$data)	Update profil dengan data valid	id = 1, nama = "John Doe", alamat = "Jl. Baru No.2"	Return sukses "Profil diperbarui"
4	updateProfile(\$id, \$data)	Update profil dengan data tidak lengkap	id = 1, nama = ""	Throw exception / return error "Field nama wajib diisi"
5	updateProfile(\$id, \$data)	Update profil id tidak ada	id = 999, data valid	Throw exception / return error "Profil tidak ditemukan"

LAMPIRAN I – RANCANGAN *INTEGRATION TESTING*

Pada lampiran ini berisi rancangan *integration testing* dari sistem yang akan dibangun, dan untuk penjelasan nya sebagai berikut ini:

a) User (*Admin*)

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/users	GET	Daftar semua user ditampilkan	True/False
2	/api/admin/users/{idUser}	GET	Detail user dengan ID valid ditampilkan	True/False
3	/api/admin/users	POST	User baru berhasil ditambahkan	True/False
4	/api/admin/users/{idUser}	PUT	Data user berhasil diperbarui	True/False
5	/api/admin/users/{idUser}	DELETE	Data user berhasil dihapus	True/False

b) Dokter

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/dokter	GET	Daftar dokter ditampilkan	True/False
2	/api/customer/dokter	GET	Daftar dokter untuk halaman customer ditampilkan	True/False
3	/api/admin/dokter/{idDokter}	GET	Detail dokter dengan ID valid ditampilkan	True/False
4	/api/admin/dokter	POST	Data dokter baru berhasil ditambahkan	True/False
5	/api/admin/dokter/{idDokter}	PUT	Data dokter berhasil diperbarui	True/False
6	/api/admin/dokter/{idDokter}	DELETE	Data dokter berhasil dihapus	True/False

c) Profil Klinik

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/klinik/profile	GET	Data profil klinik ditampilkan	True/False
2	/api/cutomer/klinik/	GET	Data profil klinik untuk halaman	True/False

No	Endpoint	Metode	Expected Output	Hasil (True/False)
			customer ditampilkan	
3	/api/admin/klinik/profile	POST	Data profil klinik berhasil ditambahkan	True/False
4	/api/admin/klinik/profile/{idProfil}	PUT	Data profil klinik berhasil diperbarui	True/False
5	/api/admin/klinik/profile/{idProfil}	DELETE	Data profil klinik berhasil dihapus	True/False

d) Testimoni

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/testimoni	GET	Daftar semua testimoni ditampilkan	True/False
2	/api/customer/testimoni	GET	Testimoni untuk halaman customer ditampilkan	True/False
3	/api/customer/testimoni/{idTestimoni}	GET	Detail testimoni valid ditampilkan	True/False
4	/api/admin/testimoni	POST	Testimoni baru berhasil ditambahkan	True/False
5	/api/admin/testimoni/{idTestimoni}	PUT	Data testimoni berhasil diperbarui	True/False
6	/api/admin/testimoni/{idTestimoni}	DELETE	Data testimoni berhasil dihapus	True/False

e) Event

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/event	GET	Daftar semua event ditampilkan	True/False
2	/api/customer/event	GET	Event untuk halaman customer ditampilkan	True/False
3	/api/customer/event/{idEvent}	GET	Detail event dengan ID valid ditampilkan	True/False
4	/api/admin/event	POST	Event baru berhasil ditambahkan	True/False
5	/api/admin/event/{idEvent}	PUT	Data event berhasil diperbarui	True/False
6	/api/admin/event/{idEvent}	DELETE	Data event berhasil dihapus	True/False
7	/api/admin/event/{idEvent}/send-wa	POST	Event berhasil dikirim ke WhatsApp	True/False

f) Reservasi

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/reservasi	GET	Daftar reservasi (admin) ditampilkan	True/False
2	/api/customer/reservasi	GET	Daftar reservasi (customer) ditampilkan	True/False
3	/api/customer/reservasi/{idReservasi}	GET	Detail reservasi valid ditampilkan	True/False
4	/api/customer/reservasi/{idReservasi}	POST	Reservasi berhasil dilakukan	True/False
5	/api/admin/reservasi/{idReservasi}	DELETE	Reservasi berhasil dihapus	True/False
6	/api/admin/reservations/{idReservasi}	PATCH	Status reservasi berhasil diubah	True/False

g) Jadwal

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/schedule	GET	Daftar jadwal reservasi ditampilkan	True/False
2	/api/admin/schedule	POST	Jadwal reservasi baru berhasil ditambahkan	True/False
3	/api/admin/schedule/{idJadwal}	PUT	Data jadwal reservasi berhasil diperbarui	True/False
4	/api/admin/schedule/{idJadwal}	DELETE	Data jadwal reservasi berhasil dihapus	True/False

h) Kategori

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/kategori	GET	Daftar kategori ditampilkan	True/False
2	/api/admin/kategori	POST	Kategori baru berhasil ditambahkan	True/False
3	/api/admin/kategori/{idKategori}	PUT	Data kategori berhasil diperbarui	True/False
4	/api/admin/kategori/{idKategori}	DELETE	Data kategori berhasil dihapus	True/False

i) Produk

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/produk	GET	Daftar semua produk ditampilkan	True/False
2	/api/admin/produk	POST	Produk baru berhasil ditambahkan	True/False
3	/api/admin/produk/{idProduk}	PUT	Data produk berhasil diperbarui	True/False
4	/api/admin/produk/{idProduk}	PATCH	Stok produk berhasil diperbarui	True/False

No	Endpoint	Metode	Expected Output	Hasil (True/False)
5	/api/customer/produk	GET	Daftar produk untuk halaman customer ditampilkan	True/False
6	/api/customer/produk/{idProduk}	GET	Detail produk valid ditampilkan	True/False
7	/api/customer/produk/{idProduk}/order	POST	Pemesanan produk berhasil dilakukan	True/False
8	/api/admin/produk/{idProduk}	DELETE	Produk berhasil dihapus	True/False

j) Keranjang

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/customer/keranjang	POST	Item berhasil ditambahkan ke keranjang	True/False
2	/api/customer/keranjang/{idKeranjang}	PATCH	Jumlah item berhasil diperbarui	True/False
3	/api/customer/keranjang/{idKeranjang}	DELETE	Item berhasil dihapus dari keranjang	True/False

k) Kegiatan

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/kegiatan	GET	Daftar kegiatan ditampilkan	True/False
2	/api/customer/kegiatan	GET	Data kegiatan untuk halaman customer	True/False
3	/api/admin/kegiatan	POST	Kegiatan baru berhasil ditambahkan	True/False
4	/api/admin/kegiatan/{idKegiatan}	PUT	Data kegiatan berhasil diperbarui	True/False
5	/api/admin/kegiatan	DELETE	Data kegiatan berhasil dihapus	True/False

l) Promo

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/promo	GET	Daftar promo ditampilkan	True/False
2	/api/customer/promo	GET	Promo untuk halaman customer	True/False
3	/api/admin/promo	POST	Promo baru berhasil ditambahkan	True/False
4	/api/admin/promo/{idPromo}	PUT	Data promo berhasil diperbarui	True/False
5	/api/admin/promo/{idPromo}	DELETE	Promo berhasil dihapus	True/False
6	/api/admin/promo/{idPromo}/send-wa	POST	Promo berhasil dikirim ke WhatsApp	True/False

m) Penjualan

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/penjualan	GET	Menampilkan daftar semua data penjualan	True/False
2	/api/admin/penjualan/{idPenjualan}	DELETE	Data penjualan berhasil dihapus	True/False
3	/api/admin/penjualan/{idPenjualan}	PATCH	Status penjualan berhasil diperbarui	True/False
4	/api/customer/penjualan/{idPenjualan}	PATCH	Status penjualan berubah menjadi “delivered” setelah customer menerima barang	True/False

n) Laporan

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/admin/laporan-penjualan	GET	Data penjualan berhasil diexport ke file Excel	True/False
2	/api/admin/laporan-reservasi	GET	Data reservasi berhasil diexport ke file Excel	True/False

o) Profil

No	Endpoint	Metode	Expected Output	Hasil (True/False)
1	/api/customer/profil/{idUser}	GET	Profil customer valid ditampilkan	True/False
2	/api/customer/profil/{idUser}	PUT	Profil customer berhasil diperbarui	True/False

LAMPIRAN J – RANCANGAN *PEFORMANCE TESTING*

Pada lampiran ini berisi rancangan *performance testing* dari sistem yang akan dibangun, dan untuk penjelasan nya sebagai berikut ini:

a) User (Admin)

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/users	GET	Ambil daftar 10.000 user	50	≤ 3.0
2	/api/admin/users/{idUser}	GET	Ambil detail user oleh 50 user bersamaan	50	≤ 2.0
3	/api/admin/users	POST	Tambah user baru secara bersamaan	50	≤ 2.5
4	/api/admin/users/{idUser}	PUT	Update data user bersamaan	50	≤ 2.5
5	/api/admin/users/{idUser}	DELETE	Hapus user bersamaan	50	≤ 2.0

b) Dokter

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/dokter	GET	Ambil daftar dokter	50	≤ 2.0
2	/api/customer/dokter	GET	Ambil daftar dokter untuk customer	50	≤ 2.0
3	/api/admin/dokter/{idDokter}	GET	Ambil detail dokter	50	≤ 2.0
4	/api/admin/dokter	POST	Tambah dokter	50	≤ 2.5
5	/api/admin/dokter/{idDokter}	PUT	Update dokter	50	≤ 2.5
6	/api/admin/dokter/{idDokter}	DELETE	Hapus dokter	50	≤ 2.0

c) Profil Klinik

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/klinik/profile	GET	Ambil profil klinik	30	≤ 2.0
2	/api/cutomer/klinik/	GET	Ambil profil klinik (customer)	30	≤ 2.0
3	/api/admin/klinik/profile	POST	Tambah profil klinik	30	≤ 2.5
4	/api/admin/klinik/profile/{idProfil}	PUT	Update profil klinik	30	≤ 2.5
5	/api/admin/klinik/profile/{idProfil}	DELETE	Hapus profil klinik	30	≤ 2.0

d) Testimoni

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/testimoni	GET	Ambil semua testimoni	50	≤ 2.0
2	/api/customer/testimoni	GET	Ambil testimoni customer	50	≤ 2.0
3	/api/customer/testimoni/{idTestimoni}	GET	Ambil detail testimoni	50	≤ 2.0
4	/api/admin/testimoni	POST	Tambah testimoni	50	≤ 2.5
5	/api/admin/testimoni/{idTestimoni}	PUT	Update testimoni	50	≤ 2.5
6	/api/admin/testimoni/{idTestimoni}	DELETE	Hapus testimoni	50	≤ 2.0

e) Event

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/event	GET	Ambil daftar event	50	≤ 2.0
2	/api/customer/event	GET	Ambil daftar event customer	50	≤ 2.0
3	/api/customer/event/{idEvent}	GET	Ambil detail event	50	≤ 2.0
4	/api/admin/event	POST	Tambah event	50	≤ 2.5
5	/api/admin/event/{idEvent}	PUT	Update event	50	≤ 2.5
6	/api/admin/event/{idEvent}	DELETE	Hapus event	50	≤ 2.0
7	/api/admin/event/{idEvent}/send-wa	POST	Mengirim event WA ke 500 nomor secara bersamaan	50	≤ 3.0

f) Reservasi

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/reservasi	GET	Ambil daftar reservasi	50	≤ 2.0
2	/api/customer/reservasi	GET	Ambil daftar reservasi customer	50	≤ 2.0
3	/api/customer/reservasi/{idReservasi}	GET	Ambil detail reservasi	50	≤ 2.0
6	/api/customer/reservasi/{idReservasi}	POST	Melakukan reservasi	50	≤ 2.5
7	/api/admin/reservasi/{idReservasi}	DELETE	Hapus reservasi	50	≤ 2.0
8	/api/admin/reservations/{idReservasi}	PATCH	Mengubah status 500 reservasi secara bersamaan	50	≤ 2.0

g) Jadwal

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/schedule	GET	Ambil daftar jadwal reservasi	30	≤ 2.5
2	/api/admin/schedule	POST	Tambah data jadwal reservasi	30	≤ 2.5
3	/api/admin/schedule/{id Jadwal}	PUT	Update data jadwal reservasi	30	≤ 2.5
4	/api/admin/schedule/{id Jadwal}	DELETE	Hapus data jadwal reservasi	30	≤ 2.5

h) Kategori

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/kategori	GET	Ambil daftar kategori	30	≤ 2.0
2	/api/admin/kategori	POST	Tambah kategori	30	≤ 2.5
3	/api/admin/kategori/{idKategori}	PUT	Update kategori	30	≤ 2.5
4	/api/admin/kategori/{idKategori}	DELETE	Hapus kategori	30	≤ 2.0

i) Produk

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/produk	GET	Ambil daftar produk	50	≤ 2.0
2	/api/admin/produk	POST	Tambah produk	50	≤ 2.5
3	/api/admin/produk/{idProduk}	PUT	Update produk	50	≤ 2.5
4	/api/admin/produk/{idProduk}	PATCH	Update stok produk	50	≤ 2.5

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
5	/api/customer/produk	GET	Ambil daftar produk customer	50	≤ 2.0
6	/api/customer/produk/{idProduk}	GET	Ambil detail produk	50	≤ 2.0
7	/api/customer/produk/{idProduk}/order	POST	Order produk	50	≤ 2.5
8	/api/admin/produk/{idProduk}	DELETE	Hapus produk	50	≤ 2.0

j) Keranjang

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/customer/keranjang	POST	Tambah ke keranjang	50	≤ 2.5
2	/api/customer/keranjang/{idKeranjang}	PATCH	Update jumlah produk keranjang	50	≤ 2.5
3	/api/customer/keranjang/{idKeranjang}	DELETE	Hapus item keranjang	50	≤ 2.0

k) Kegiatan

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/kegiatan	GET	Ambil daftar kegiatan	30	≤ 2.0
2	/api/customer/kegiatan	GET	Ambil kegiatan customer	30	≤ 2.0
3	/api/admin/kegiatan	POST	Tambah kegiatan	30	≤ 2.5
4	/api/admin/kegiatan/{idKegiatan}	PUT	Update kegiatan	30	≤ 2.5
5	/api/admin/kegiatan	DELETE	Hapus kegiatan	30	≤ 2.0

l) Promo

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/promo	GET	Ambil daftar promo	30	≤ 2.0
2	/api/customer/promo	GET	Ambil promo customer	30	≤ 2.0
3	/api/admin/promo	POST	Tambah promo	30	≤ 2.5
4	/api/admin/promo/{idPromo}	PUT	Update promo	30	≤ 2.5
5	/api/admin/promo/{idPromo}	DELETE	Hapus promo	30	≤ 2.0
6	/api/admin/promo/{idPromo}/send-wa	POST	Mengirim promo WA ke 500 nomor secara bersamaan	50	≤ 3.0

m) Penjualan

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/penjualan	GET	Ambil daftar 50.000 data penjualan	50	≤ 3.5
2	/api/admin/penjualan/{idPenjualan}	DELETE	Hapus 500 data penjualan secara bersamaan	50	≤ 2.5
3	/api/admin/penjualan/{idPenjualan}	PATCH	Mengubah status 500 data penjualan secara bersamaan	50	≤ 2.0
4	/api/customer/penjualan/{idPenjualan}	PATCH	500 customer menerima barang bersamaan	50	≤ 2.0

n) Laporan

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/admin/laporan-penjualan	GET	Export data penjualan	20	≤ 5.0
2	/api/admin/laporan-reservasi	GET	Export data reservasi	20	≤ 5.0

o) Profil

No	Endpoint	Metode	Skenario Pengujian	Concurrent User	Target Response Time (detik)
1	/api/customer/profil/{idUser}	GET	Ambil profil customer	20	≤ 2.0
2	/api/customer/profil/{idUser}	PUT	Update profil customer	20	≤ 2.5

LAMPIRAN K – RANCANGAN *SECURITY TESTING*

Pada lampiran ini berisi rancangan *security testing* dari sistem yang akan dibangun, dan untuk penjelasannya sebagai berikut ini:

a) User (Admin)

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/users (GET)	Akses tanpa token	-	401 Unauthorized
2	/api/admin/users (GET)	Akses pakai token user biasa	-	403 Forbidden
3	/api/admin/users/{idUser} (GET)	SQL Injection di ID	idUser=1 OR 1=1	400 Bad Request
4	/api/admin/users (POST)	XSS di nama	nama=""	400 Bad Request
5	/api/admin/users/{idUser} (DELETE)	Akses tanpa token	-	401 Unauthorized

b) Dokter

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/dokter(GET)	Akses tanpa token	-	401 Unauthorized
2	/api/admin/dokter/{idDokter} (GET)	SQL Injection di ID	idDokter="1 OR 1=1"	400 Bad Request
3	/api/admin/dokter (POST)	XSS di field nama	nama="<script>alert(1)</script>"	400 Bad Request
1	/api/admin/dokter (GET)	Akses tanpa token	-	401 Unauthorized
2	/api/admin/dokter/{idDokter} (GET)	SQL Injection di ID	idDokter="1 OR 1=1"	400 Bad Request

c) Profil Klinik

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/klinik/profile (GET)	Akses tanpa token	-	401 Unauthorized
2	/api/admin/klinik/profile (POST)	Input kosong	{}	400 Bad Request
3	/api/admin/klinik/profile/{idProfil} (PUT)	SQL Injection di ID	idProfil="1 OR 1=1"	400 Bad Request

d) Testimoni

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/testimoni (GET)	Akses tanpa token	-	401 Unauthorized
2	/api/admin/testimoni (POST)	XSS di namaTester	namaTester = "<script>alert(1)</script>"	400 Bad Request

e) Event

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/event (GET)	Akses tanpa token	-	401 Unauthorized
2	/api/admin/event (POST)	XSS di nama	nama=""	400 Bad Request
3	/api/admin/event/{idEvent}/send-wa (POST)	Akses tanpa token	-	401 Unauthorized
4	/api/admin/event/{idEvent}/send-wa (POST)	Akses pakai token customer	-	403 Forbidden
5	/api/admin/penjualan (GET)	Akses tanpa token	-	401 Unauthorized

f) Reservasi

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/reservasi (GET)	Akses tanpa token	-	401 Unauthorized
2	/api/customer/reservasi/{idReservasi} (GET)	Akses reservasi	id milik orang lain	403 Forbidden

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
		milik user lain		
3	/api/customer/reservasi/{idReservasi} (PATCH)	SQL Injection di ID	idReservasi="1 OR 1=1"	400 Bad Request
4	/api/admin/reservations/{idReservasi} (PATCH)	Akses tanpa token	-	401 Unauthorized
5	/api/admin/reservations/{idReservasi} (PATCH)	Akses pakai token customer	-	403 Forbidden
6	/api/admin/reservations/{idReservasi} (PATCH)	SQL Injection di ID	idReservasi=1 OR 1=1	400 Bad Request

g) Jadwal

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/schedule (GET)	Akses tanpa token	-	401 Unauthorized
2	/api/admin/schedule (GET)	Akses pakai token customer	-	403 Forbidden
3	/api/admin/schedule (POST)	Akses tanpa token	-	401 Unauthorized
4	/api/admin/schedule (POST)	Akses pakai token customer	-	403 Forbidden
5	/api/admin/schedule/{idJadwal} (PUT)	SQL Injection di ID jadwal	idJadwal=1 OR 1=1	400 Bad Request
6	/api/admin/schedule/{idJadwal} (PUT)	Akses tanpa token	-	401 Unauthorized
7	/api/admin/schedule/{idJadwal} (PUT)	Akses pakai token customer	-	403 Forbidden
8	/api/admin/schedule/{idJadwal} (DELETE)	Akses tanpa token	-	401 Unauthorized

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
9	/api/admin/schedule/{idJadwal} (DELETE)	Akses pakai token customer	-	403 Forbidden
10	/api/admin/schedule/{idJadwal} (DELETE)	SQL Injection di ID jadwal	idJadwal=99 OR 1=1	400 Bad Request

h) Kategori

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/kategori (POST)	Input nama kosong	{}	400 Bad Request
2	/api/admin/kategori/{idKategori} (PUT)	SQL Injection di ID	idKategori="1 OR 1=1"	400 Bad Request

i) Produk

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/produk (POST)	Input harga negatif	harga=-1000	400 Bad Request
2	/api/admin/produk/{idProduk} (PATCH)	SQL Injection di ID	idProduk="1 OR 1=1"	400 Bad Request
3	/api/customer/produk/{idProduk} (GET)	Akses produk tidak ada	idProduk=999	404 Not Found

j) Keranjang

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/customer/keranjang (POST)	Tambah produk tidak ada	product_id=999	404 Not Found
2	/api/customer/keranjang/{idKeranjang} (PATCH)	Update milik user lain	idKeranjang orang lain	403 Forbidden

k) Kegiatan

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/kegiatan (POST)	Nama kosong	{}	400 Bad Request
2	/api/admin/kegiatan (DELETE)	Akses tanpa token	-	401 Unauthorized

l) Promo

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/promo (POST)	Judul kosong	{}	400 Bad Request
2	/api/admin/promo/{idPromo} (PUT)	SQL Injection di ID	idPromo="1 OR 1=1"	400 Bad Request
3	/api/admin/promo/{idPromo}/send-wa (POST)	Akses tanpa token	-	401 Unauthorized
4	/api/admin/promo/{idPromo}/send-wa (POST)	Akses pakai token customer	-	403 Forbidden
5	/api/admin/promo/{idPromo}/send-wa (POST)	XSS di pesan	pesan=<script>alert('x')</script>	400 Bad Request

m) Penjualan

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/penjualan/{idPenjualan} (DELETE)	Akses tanpa token	-	401 Unauthorized
2	/api/admin/penjualan/{idPenjualan} (DELETE)	Akses pakai token customer	-	403 Forbidden
3	/api/admin/penjualan/{idPenjualan} (PATCH)	SQL Injection di ID	idPenjualan=1 OR 1=1	400 Bad Request
4	/api/customer/penjualan/{idPenjualan} (PATCH)	Akses tanpa token	-	401 Unauthorized
5	/api/customer/penjualan/{idPenjualan} (PATCH)	Akses pakai token admin (tidak boleh)	-	403 Forbidden

n) Laporan

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/admin/laporan-penjualan (GET)	Akses tanpa token	-	401 Unauthorized
2	/api/admin/laporan-reservasi (GET)	Akses tanpa token	-	401 Unauthorized

o) Profil

No	Endpoint	Uji Keamanan	Data Uji	Expected Output
1	/api/customer/profil/{idUser} (GET)	Akses profil user lain	idUser orang lain	403 Forbidden
2	/api/customer/profil/{idUser} (PUT)	Input kosong	{}	400 Bad Request

LAMPIRAN L – DOKUMENTASI *INCREMENT*

Judul : Dokumentasi *Increment* Ke-1
 Periode : 15 September – 26 September 2025
 Developer : As Shaff, Bintang
 Tujuan : Pembuatan modul *login admin* dan manajemen *user*, modul profil klinik
Increment

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul <i>login admin</i> dan manajemen <i>user</i> , modul profil klinik (visi, misi, deskripsi, kontak, jam operasional), serta tampilan profil klinik di sisi <i>customer</i> .	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			

Judul : Dokumentasi *Increment* Ke-2
 Periode : 27 September – 6 oktober 2025
 Developer : As Shaff, Bintang
 Tujuan *Increment* : Pembuatan modul registrasi dan *login customer* serta modul profil *customer*

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul registrasi dan <i>login customer</i> serta modul profil <i>customer</i> (kelola/edit data akun).	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			

Judul : Dokumentasi *Increment* Ke-3
 Periode : 7 Oktober – 15 Oktober 2025
 Developer : As Shaff, Bintang
 Tujuan *Increment* : Pembuatan modul pengelolaan jadwal reservasi

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul pengelolaan jadwal reservasi (<i>admin</i> menambah/mengatur jadwal dokter dan <i>treatment</i>).	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			

Judul : Dokumentasi *Increment* Ke-4
 Periode : 16 Oktober – 26 Oktober 2025
 Developer : As Shaff, Bintang
 Tujuan *Increment* : Pembuatan modul pemesanan reservasi oleh *customer* berdasarkan jadwal yang tersedia

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul pemesanan reservasi oleh <i>customer</i> berdasarkan jadwal yang tersedia (hanya setelah <i>login</i>).	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			

Judul : Dokumentasi *Increment* Ke-5
 Periode : 27 Oktober – 3 November 2025
 Developer : As Shaff, Bintang
 Tujuan *Increment* : Pembuatan modul kelola informasi dokter oleh *admin* dan tampilkan profil dokter ke *customer*.

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul kelola informasi dokter oleh <i>admin</i> dan tampilkan profil dokter ke <i>customer</i> .	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			

Judul : Dokumentasi *Increment* Ke-6
 Periode : 4 November – 12 November 2025
 Developer : As Shaff, Bintang
 Tujuan *Increment* : Pembuatan modul *promo* dan *event*

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul <i>promo</i> dan <i>event</i> (admin mengelola <i>promo/event</i> dan <i>customer</i> dapat melihat informasi tersebut).	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			

Judul : Dokumentasi *Increment* Ke-7
 Periode : 13 November – 22 November 2025
 Developer : As Shaff, Bintang
 Tujuan *Increment* : Pembuatan modul distribusi *promo/event* via *WhatsApp* serta modul testimoni

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul distribusi <i>promo/event</i> via <i>WhatsApp</i> serta modul testimoni (<i>admin</i> kelola testimoni, <i>customer</i> dapat melihat testimoni).	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			

Judul : Dokumentasi *Increment* Ke-8
 Periode : 23 November – 1 Desember 2025
 Developer : As Shaff, Bintang
 Tujuan *Increment* : Pembuatan modul produk dan kategori produk

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul produk dan kategori produk (<i>admin</i> mengelola data produk, <i>customer</i> dapat melihat produk).	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			

Judul : Dokumentasi *Increment* Ke-9
 Periode : 2 Desember – 15 Desember 2025
 Developer : As Shaff, Bintang
 Tujuan *Increment* : Pembuatan modul transaksi penjualan produk yang terintegrasi dengan kode *voucher* dan diskon otomatis.

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul transaksi penjualan produk yang terintegrasi dengan kode <i>voucher</i> dan diskon otomatis.	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			

Judul : Dokumentasi *Increment* Ke-10
 Periode : 16 Desember – 29 Desember 2026
 Developer : As Shaff, Bintang
 Tujuan *Increment* : Pembuatan modul laporan reservasi dan penjualan (ekspor ke *Excel*), modul *dashboard* ringkasan data, serta tahap penyempurnaan sistem.

Modul yang Dikembangkan	Pengujian			Feedback	Perbaikan
	Jenis Testing	Hasil	Keterangan		
Modul laporan reservasi dan penjualan (ekspor ke <i>Excel</i>), modul <i>dashboard</i> ringkasan data, serta tahap penyempurnaan sistem.	<i>Unit Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Integration Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Peformance Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			
	<i>Security Testing</i>	☐ Berhasil ☐ Berhasil Sebagian ☐ Gagal			